

# 프로젝트 최종 보고서

## Pickmood

사용자가 촬영한 메뉴 사진을 활용한 카페 SNS 이미지 및 캡션 생성  
서비스

오픈소스 모델 실험 | 12 개 프리셋 | ComfyUI | 서비스 배포

### 팀원별 담당 업무

유소연 — PM · 프리셋 · ComfyUI

정설아 — 오픈소스 모델링 1

김기현 — 오픈소스 모델링 2

송우현 — 프론트엔드 · 백엔드 · 서비스 통합

codeit AI 8 기 고급프로젝트 1 팀 | 2026 년 7 월 30 일

# 목차

|                                                         |           |
|---------------------------------------------------------|-----------|
| <b>1. 서론</b>                                            | <b>6</b>  |
| 1.1 프로젝트 목표와 핵심 가치                                      | 6         |
| 1.2 주요 사용자와 사용 시나리오                                     | 6         |
| 1.3 입력과 출력                                              | 6         |
| 1.4 기존 방식의 한계와 사용자 불편                                   | 7         |
| 1.5 서비스 정의와 경쟁 서비스 비교                                   | 9         |
| <b>2. 오픈소스 모델링 1 · 생성 모델 파이프라인 실험 — 정설아</b>             | <b>12</b> |
| 2.1 1 단계 — SD1.5 Inpainting                             | 13        |
| 2.2 2 단계 — SDXL 전환 및 반복 개선                              | 14        |
| 2.3 3 단계 — FLUX.1 Kontext(v2) 도입과 실패                    | 17        |
| 2.4 4 단계 — v3: 다중 모델 조합 아키텍처                            | 18        |
| 2.5 5 단계 — v3 → v4 전환 이유                                | 22        |
| 2.6 6 단계 — v4 전체 파이프라인 개요                               | 23        |
| 2.7 7 단계 — v3-v4 사용 모델 상세                               | 25        |
| 2.8 8 단계 — v4 고도화 로직 상세                                 | 26        |
| 2.9 9 단계 — 모델·로직 선택 이유 총정리                              | 30        |
| 2.10 10 단계 — 데이터 아키텍처 재설계 (메타데이터 스키마)                   | 32        |
| 2.11 11 단계 — 노트북 구조 재설계                                 | 32        |
| 2.12 12 단계 — 학습(러닝) 우선순위                                | 32        |
| 2.13 13 단계 — 진행 일지 (개선 이력 요약)                           | 33        |
| 2.14 14 단계 — v4_final 고도화 작업                            | 33        |
| 2.15 종합 타임라인                                            | 35        |
| <b>3. 오픈소스 모델링 2 · FLUX.1 Kontext 기반 모델 파이프라인 — 김기현</b> | <b>37</b> |
| 3.1 전체 재생성 방식 검토와 로고 훼손                                 | 39        |
| 3.2 합성 방식과 이미지 두 장 입력 문제                                | 39        |
| 3.3 복잡한 템플릿에서 컵 위치 찾기                                   | 41        |
| 3.4 원래 컵 지우기와 캐싱                                        | 42        |
| 3.5 하이브리드 — 합성한 뒤 약하게 다시 그리기                            | 43        |
| 3.6 그림자와 오려내기 다듬기                                       | 44        |
| 3.7 속도 개선 — 미리 준비하기와 실시간 처리 나누기                         | 45        |
| 3.8 버전 1 정리                                             | 46        |
| 3.9 FLUX Kontext 로 전체 다시 그리기                            | 47        |
| 3.10 12 개 조합 한 번에 만들기                                   | 48        |

|                                          |           |
|------------------------------------------|-----------|
| 3.11 참조 이미지로 스타일 뽑아내기 .....              | 49        |
| 3.12 product-lock — rembg 로 제품 지키기 ..... | 50        |
| 3.13 다시 그리는 세기 조절 .....                  | 51        |
| 3.14 하드코딩 걷어내고 config 정리 .....           | 52        |
| 3.15 제품 방향 자동으로 바로잡기 .....               | 53        |
| 3.16 OpenAI 이미지 API 옵션 추가 .....          | 53        |
| 3.17 FastAPI 서버와 프론트/백엔드 연결 .....        | 55        |
| 3.18 버전 2 정리 .....                       | 55        |
| 3.19 결론 .....                            | 56        |
| <b>4. 시행착오와 공용 파이프라인 전환 .....</b>        | <b>57</b> |
| 4.1 오픈소스 실험에서 확인한 구조적 한계 .....           | 57        |
| 4.2 레퍼런스 기반 프리셋 설계로 전환 .....             | 57        |
| 4.3 레퍼런스 수집과 분류 .....                    | 58        |
| 4.4 MVP 용 12 개 대표 프리셋 확정 .....           | 59        |
| <b>5. 프리셋·장면·공용 런타임 설계 — 유소연 .....</b>   | <b>61</b> |
| 5.1 프리셋 패키지의 구성 .....                    | 61        |
| 5.2 컵 적용 방식 .....                        | 62        |
| 5.3 온도와 서빙 상태 .....                      | 63        |
| 5.4 장면 힌트의 역할 .....                      | 63        |
| 5.5 장면 설계 사례 .....                       | 64        |
| 5.6 프리셋 코드 예시 .....                      | 65        |
| 5.7 프롬프트 조립 예시 .....                     | 65        |
| 5.8 구현 및 운영 구조 .....                     | 66        |
| <b>6. 품질·비용·모델 선택 평가 — 유소연 .....</b>     | <b>71</b> |
| 6.1 구현 완료 기능 .....                       | 71        |
| 6.2 현재 확인된 구현 사실 .....                   | 72        |
| 6.3 검증 완료 범위와 해석 .....                   | 73        |
| 6.4 Langfuse 기반 비용·지연 시간 관측 .....        | 73        |
| 6.5 gpt-image-2 medium 채택 근거 .....       | 74        |
| 6.6 공용 파이프라인에서의 오픈소스 모델 재검토 .....        | 75        |
| 6.7 품질 평가 항목과 기준 .....                   | 76        |
| <b>7. 서비스 구현과 전체 아키텍처 — 송우현 .....</b>    | <b>79</b> |
| 7.1 개요 .....                             | 80        |
| 7.2 진행 타임라인과 주요 의사결정 .....               | 80        |
| 7.3 결과물 .....                            | 84        |
| 7.4 주요 의사결정 및 회의 제안 총정리 .....            | 88        |

|                               |           |
|-------------------------------|-----------|
| 7.5 오류 및 트러블슈팅 총정리 .....      | 89        |
| 7.6 회고 .....                  | 92        |
| <b>8. 현재 한계와 개선 과제 .....</b>  | <b>93</b> |
| 8.1 프리셋 제작의 수작업 의존 .....      | 93        |
| 8.2 자동 품질 평가의 미완성 .....       | 93        |
| 8.3 단일 Gateway 와 임시 저장소 ..... | 94        |
| 8.4 브랜드 로고와 편집 기능 .....       | 94        |
| 8.5 비용과 품질의 지속적인 관리 .....     | 95        |
| 8.6 검증 범위의 한계 .....           | 95        |
| 8.7 서비스 확장 방향 .....           | 96        |
| <b>9. 종합 결론 .....</b>         | <b>98</b> |

# 프로젝트 요약

| 항목        | 현재 상태                                                                                        |
|-----------|----------------------------------------------------------------------------------------------|
| 서비스 대상    | 인스타그램으로 매장을 홍보하는 카페·소규모 매장 운영자                                                               |
| 이미지 생성 모델 | OpenAI <code>gpt-image-2</code>                                                              |
| 기본 생성 품질  | <code>medium</code>                                                                          |
| 프리셋       | 화이트·우드·다크그레이 3 개 계열 × 4 개 촬영 구도 = 총 12 개                                                     |
| 촬영 구도     | 클로즈업, 미디엄샷, 항공샷, 손에 든 샷                                                                      |
| 컵 적용 방식   | 사용자 컵 재생성 / 레퍼런스 컵 채택                                                                        |
| 출력 비율     | 4:5 <code>1024×1280</code> , 1:1 <code>1024×1024</code>                                      |
| 후처리 정책    | 중앙 크롭·추가 리사이즈·LUT 색보정 미적용                                                                    |
| 실행 구조     | 하나의 ComfyUI 공용 런타임이 12 개 프리셋 설계 규칙을 해석                                                       |
| 검증 상태     | 12/12 프리셋 승인 완료, 프리셋 브랜치 회귀 테스트 78/78 통과, 통합 런타임 86 건 수행·실패 0                                |
| 배포 상태     | 기준 프리셋 12 개와 관측 코드 GCP 반영                                                                    |
| 최근 보완 작업  | 화이트·다크그레이 클로즈업의 사용자 컵 슬리브·로고 보존, 다크그레이 클로즈업 빨대·뚜껑 보존, 화이트 미디엄 축소·3 중 랜덤 힌트까지 커밋·푸시·GCP 배포 완료 |
| 검증 범위     | 12 개 프리셋 × 2 개 컵 모드 × 3 개 품질 = 총 72 개 유료 생성 조합 전수 검증                                         |

# 1. 서론

## 1.1 프로젝트 목표와 핵심 가치

Pickmood 는 사용자가 스마트폰으로 촬영한 음료 사진을 업로드하고 원하는 분위기와 구도를 선택하면, 인스타그램 게시물에 어울리는 새로운 홍보 이미지를 생성하는 서비스이다. 이름에는 내가 원하는 무드를 직접 고른다는 서비스 경험을 담았다.

단순히 화려한 이미지를 만드는 것보다 다음 세 가지를 핵심 목표로 설정했다.

1. 실제로 촬영한 듯한 자연스러움
2. 선택한 프리셋을 통해 예상할 수 있는 결과
3. 반복적인 SNS 콘텐츠 제작 부담 감소

이미지 생성 이후에는 결과 사진을 바탕으로 캡션, 해시태그, 스토리용 문구도 함께 제공한다. 사용자는 생성 결과를 확인한 뒤 문구 생성 버튼을 선택적으로 사용할 수 있으며, 최종 문구는 복사해 바로 게시물에 활용할 수 있다.

## 1.2 주요 사용자와 사용 시나리오

- 인스타그램을 주요 홍보 수단으로 활용하는 카페 사장님
- 제품 사진을 직접 촬영하지만 보정과 디자인 작업에 익숙하지 않은 사용자
- 전문 촬영이나 외주 제작에 지속적으로 비용을 쓰기 어려운 소규모 사업자
- 신메뉴·시즌 메뉴·행사 게시물을 반복적으로 제작해야 하는 운영자

## 1.3 입력과 출력

### 1.3.1 입력

- 사용자가 촬영한 음료 사진 1 장

- 12 개 프리셋 중 1 개
- 컵 적용 방식
- **사용자 컵 재생성**: 입력 사진의 컵 계열·재질·구조를 새 장면에 맞게 다시 생성
- **레퍼런스 컵 채택**: 음료의 정체성은 유지하고 프리셋에 정의된 컵 디자인 적용
- 결과 비율: 4:5 또는 1:1
- 프리셋별 보조 장면 이미지 최대 1 장(사용자 제품 1 장과 함께 provider 에 전달)
- 캡션 생성 시 메뉴명과 게시 목적 입력 가능

### 1.3.2 출력

- 인스타그램 게시용 PNG 이미지
- 게시물 캡션
- 해시태그 8 개
- 스토리용 짧은 문구



그림 1. 최신 서비스 화면의 입력 사진과 생성 결과 비교

## 1.4 기존 방식의 한계와 사용자 불편

카페 운영자가 스마트폰으로 제품을 촬영하는 것 자체는 어렵지 않다. 그러나 이 촬영본을 광고에 활용하려면 카메라 시점, 피사체 크기, 조명, 배경, 소품, 출력 비율과 문구를 다시 결정해야 한다. 결과가

의도와 다르면 같은 설명을 고쳐가며 반복 생성하거나 편집해야 하므로, 실제 불편은 생성 기능의 부재보다 광고용 결과에 도달하기까지의 판단과 반복에 있다.

Pickmood 는 이 판단을 사용자가 매번 새로 작성하게 하지 않고, 카페 SNS 촬영에서 반복되는 구도·빛·재질·배치 기준을 프리셋으로 제공한다. 서비스의 핵심은 예쁜 사진을 새로 만드는 데 그치지 않고, 사용자가 직접 촬영한 제품 사진을 제품 정체성을 고려한 광고용 이미지로 전환하는 과정을 구조화하는 것이다.

### 1.4.1 주제 선정 배경

프로젝트 초기에는 포스터나 배너를 자동으로 제작하는 서비스를 검토했다. 그러나 해당 분야는 이미 대규모 템플릿, 요소별 편집, AI 생성·수정, 인쇄물 제작까지 연결하는 상용 서비스가 자리 잡고 있다.

제한된 프로젝트 기간 안에서 기존 서비스와 같은 범위를 구현하면 기능의 폭은 넓어질 수 있지만, 사용자가 굳이 새로운 서비스를 선택해야 할 이유를 만들기 어렵다. 이에 따라 포스터 전체를 제작하는 대신, 소상공인이 반복적으로 필요로 하는 **제품 사진 제작 과정**에 집중했다.

경쟁 서비스의 기능 수를 따라가는 대신, 내부 실험에서 확인한 **결과물 품질·반복 생성 안정성·서비스화 가능성·남은 기간 내 완성 가능성**을 최종 방향의 판단 기준으로 사용했다. 출처가 확보되지 않은 시장 사례는 근거처럼 인용하지 않았다.

### 1.4.2 촬영본을 광고 이미지로 전환하는 과제

생성형 AI 는 제품을 전문 스튜디오에서 촬영한 것처럼 표현하는 데 매우 강하다. 그러나 인스타그램에서 매력적으로 보이는 사진이 반드시 고가 장비와 완벽한 조명으로 만든 광고 사진인 것은 아니다.

카페 계정에서는 다음과 같은 사진이 더 자연스럽게 받아들여지는 경우가 많다.

- 스마트폰으로 촬영한 듯한 시선
- 과도하게 정제되지 않은 생활 사진의 질감
- 제품을 압도하지 않는 자연스러운 배경

- 적당한 여백과 소품
- 실제 공간과 연결된 빛과 그림자

이에 따라 프로젝트의 방향을 다음과 같이 정의했다.

전문 스튜디오 광고 컷이 아니라, 원래 카페에서 이렇게 찍은 것처럼 보이는 인스타그램용 이미지를 만든다.

### 1.4.3 생성 이후의 경쟁력

AI 로 글, 이미지, 영상, 웹사이트를 만드는 것 자체는 더 이상 특별한 일이 아니다. 생성 기능이 보편화될수록 결과가 AI 로 만들어졌는지보다, 사용 목적에 맞고 자연스럽게 받아들여지는지가 중요해진다.

최근에는 AI 가 쓴 것처럼 보이지 않는 글, 실제 촬영처럼 보이는 영상, 인위적인 질감이 적은 이미지에 대한 관심이 커지고 있다. 이 흐름에서 Pickmood 가 집중한 지점은 생성 속도나 화려함이 아니라 AI 특유의 과장과 부자연스러움을 얼마나 줄일 수 있는가였다.

외부 사용자 반응을 정량 근거로 사용하지 않고, 실제 생성 결과에 대한 내부 시각 QA 와 재현 가능성을 품질 판단의 근거로 삼았다.

## 1.5 서비스 정의와 경쟁 서비스 비교

### 1.5.1 인스타그램용 광고 이미지의 기준

본 프로젝트에서 말하는 인스타그램 감성 이미지는 특정 필터나 색상을 의미하지 않는다. 다음 요소가 장면 안에서 자연스럽게 연결된 사진에 가깝다.

#### 1. 구도와 여백

- 피사체가 화면을 지나치게 가득 채우지 않는 구성
- 제품과 배경이 함께 보이는 시각적 여유

#### 2. 빛과 색감

- 자연광처럼 이해할 수 있는 광원

- 배경과 제품에 동일하게 적용되는 빛의 방향
- 과도한 HDR, 선명도, 색보정이 없는 톤

### 3. 카메라 시선

- 장면에 맞는 촬영 거리와 렌즈
- 피사체를 바라보는 각도와 높이
- 스마트폰 사진처럼 유한한 선명도와 자연스러운 심도

### 4. 공간과 소품

- 테이블, 벽, 의자, 접시 등 주변 요소의 관계
- 제품보다 앞서 보이지 않는 보조 소품
- 실제 공간이 프레임 밖으로 이어지는 듯한 깊이

### 5. 물리적 일관성

- 컵과 바닥이 붙어 있는 접지감
- 재질에 맞는 반사·투과·굴절
- 손과 컵의 자연스러운 접촉

## 1.5.2 범용 생성·편집 도구와의 차이

| 구분       | 작업 출발점            | 사용자가 직접 결정하는 내용               | 주요 강점                            |
|----------|-------------------|-------------------------------|----------------------------------|
| ChatGPT  | 사진 + 대화형<br>요청    | 구도·빛·소품·수정 방향을 설명하고 반복<br>지시  | 유연한 이미지 생성과<br>자연어 편집            |
| Canva    | 사진 + 디자인<br>작업    | 편집 도구·레이어·배경·레이아웃을<br>선택하고 조정 | 편집·템플릿·브랜드<br>콘텐츠 제작             |
| Pickmood | 제품 사진 + 촬영<br>프리셋 | 무드·구도·컵 방식·비율 선택              | 카페 SNS 제품 사진<br>전환에 특화된 반복<br>흐름 |

ChatGPT 와 Canva 역시 기존 사진 업로드, 이미지 생성과 부분 편집 기능을 제공한다. 따라서 Pickmood 의 차별점은 다른 서비스가 이미지를 만들지 못한다는 데 있지 않다. 범용 도구에서는 사용자가 목적과 수정 방향을 대화나 편집 동작으로 구체화하지만, Pickmood 에서는 제품 사진과 프리셋 선택만으로 카페 제품 촬영에 필요한 설계 규칙을 적용한다.

참고: OpenAI, “Images in ChatGPT” 및 Image generation guide; Canva, “AI Photo Editor”. 기능 비교는 2026 년 7 월 28 일 공식 안내를 기준으로 했다.

범용 이미지 생성 AI 에서도 사용자가 원하는 장면을 설명해 이미지를 만들 수 있다. 하지만 카페 사장님이 매번 렌즈, 조명, 피사체 크기, 여백, 소품의 위치, 그림자 방향을 자세히 작성하는 것은 현실적이지 않다.

또한 ‘인스타그램 감성’은 하나의 문장으로 정확히 정의하기 어렵다. 결과를 수정할수록 처음 의도와 멀어지는 경우도 있으며, 같은 표현을 사용해도 이미지마다 결과 편차가 발생한다.

| 구분      | 일반 이미지 생성 AI   | Pickmood              |
|---------|----------------|-----------------------|
| 장면 설정   | 사용자가 직접 상세히 설명 | 프리셋 선택으로 주요 조건 결정     |
| 프롬프트 작성 | 사용자 작성 비중이 큼   | 내부 설계 규칙과 템플릿으로 자동 조립 |
| 결과 예측   | 생성 전 예측이 어려움   | 프리셋 썸네일과 구도로 결과 예상    |
| 컵 처리    | 대화로 반복 지시      | 사용자 컵 / 레퍼런스 컵 명시적 선택 |
| 출력 비율   | 요청마다 설명        | 4:5·1:1 설계 규칙으로 고정    |
| 사용 목적   | 범용 이미지 생성      | 카페 SNS 홍보 이미지에 특화     |
| 부가 기능   | 서비스별 차이        | 캡션·해시태그·스토리 문구 생성     |

1.5.3 핵심 차별점

- 긴 프롬프트 작성 부담 감소
- 결과의 예측 가능성 향상
- 제품 정체성과 장면 스타일의 역할 분리
- 카메라·조명·배치·재질을 프리셋 단위로 관리
- 인스타그램 게시 목적에 맞는 비율과 문구 제공

개인 작업 보고서

# Pickmood

## 2. 오픈소스 모델링 1 · 생성 모델 파이프라인 실험

작성자 정설아

# ad-creator 파이프라인 개발 전체 흐름 정리

*SD1.5 → SDXL(v1) → FLUX.1 Kontext(v2) → v3(다중 모델 조합) → v4(정밀 합성 아키텍처)*

작성: 정설아 · ad-creator (Codeit AI 엔지니어링 부트캠프 팀 프로젝트, GitHub: ysy-works/ad-creator)

## 문서 개요

ad-creator는 소상공인(카페 등)이 업로드한 제품 사진을 사전에 촬영·준비된 라이프스타일 배경/템플릿에 자연스럽게 합성하여, 인스타그램에 바로 게시할 수 있는 광고 이미지와 캡션을 자동으로 생성하는 파이프라인이다. 본 문서는 프로젝트 초기 SD1.5 단계부터, SDXL 전환, FLUX.1 Kontext 기반 v1 시도, v2(다중 모델 조합 아키텍처), v3(정밀 합성 아키텍처)에 이르는 전체 개발 흐름을 노트북과 설계 문서를 근거로 시간 순 + 구조 순으로 정리한 것이다.

각 버전 전환은 '파라미터를 더 튜닝해서' 해결되지 않았던 구조적 한계에서 비롯되었다는 공통점이 있다. 이 문서는 그 구조적 한계가 무엇이었는지, 어떤 실험/디버깅을 거쳐 그것을 확인했는지, 그리고 다음 버전에서 어떤 모델·로직을 왜 선택했는지를 최대한 빠짐없이 담는 것을 목표로 한다.

## 2.1 1 단계 — SD1.5 Inpainting

프로젝트 최초 버전은 Stable Diffusion 1.5 기반 Inpainting 파이프라인으로 시작했다. rembg로 제품 배경을 제거하고, 브랜드 무드를 반영하기 위한 LoRA를 적용했으며, 캡션은 GPT-4o-mini로 생성하는 구조였다.

### 2.1.1 SD1.5 에서 SDXL 로 전환하게 된 계기

SD1.5 파이프라인은 matcha(투명·반사 유리컵) 조합에서 컵 형태가 반복적으로 왜곡되는 문제를 겪었다. 이는 파라미터 문제가 아니라 SD1.5 자체의 구조적 한계(낮은 학습 해상도 512px, 상대적으로 낮은 형태 보존 능력)로 판단되어, 더 높은 해상도(1024px)와 더 강한 형태 보존 능력을 가진 SDXL로 교체하는 결정으로 이어졌다.

| 항목    | SD1.5                                | SDXL                                             |
|-------|--------------------------------------|--------------------------------------------------|
| 파이프라인 | StableDiffusionInpaintPipeline       | StableDiffusionXLInpaintPipeline                 |
| 체크포인트 | runwayml/stable-diffusion-inpainting | diffusers/stable-diffusion-xl-1.0-inpainting-0.1 |
| 해상도   | 512px                                | 1024px                                           |
| VAE   | 기본                                   | madebyollin/sd-xl-vae-fp16-fix (오버플로우 방지)        |

## 2.2 2 단계 — SDXL 전환 및 반복 개선

### 2.2.1 파이프라인 구조

SDXL 버전 파이프라인은 노트북 기준 아래 순서로 구성된다.

| 단계                        | 기능                              | 핵심 설계 포인트                                                                                                                                           |
|---------------------------|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Background Removal     | 제품 사진 → RGBA 투명배경               | EXIF 방향 보정 + rembg alpha_matting으로 경계 halo/구멍 방지. 이미 처리된 파일은 캐싱으로 스킵. 원본이 MAX_INPUT_DIMENSION(1600px) 초과 시 자동 축소해 alpha_matting(CPU 연산) 속도 병목 완화    |
| 2. Style Manager          | Mood × Composition → Style ID   | Mood 3종(mood_wood/mood_natural_white/mood_vivid) × Composition 4종(product_large/product_center/aerial_shot/handheld_lifestyle) = 12개 Style ID 조합 관리 |
| 3. Prompt Builder         | Mood+Composition+Negative 조합    | 배경 생성 프롬프트에서는 모든 구도에서 product 설명을 제외(CLIP 77토큰 한도 확보 + Inpainting이 이미 제품 픽셀을 보고 있어 불필요). positive_prompt(캡션용)에는 product 설명 항상 포함                    |
| 4. SDXL 배경 생성             | Inpainting, 실제 스케일 마스크 +Feather | SDXL 전용 체크포인트 + fp16-fix VAE. Composition별 '최종 합성 시 제품이 차지할 비율'을 실제 스케일 그대로 마스크에 반영(참조 스케일 캡 없음) — 그림자·원근이 자연스럽도록                                   |
| 5. Image Composer         | 생성 배경 + 제품 재합성 + 그림자 후처리        | add_drop_shadow()로 인공 그림자 추가(단, SD가 이미 그림자를 그린 경우 중복 방지용 옵션 제공)                                                                                     |
| 6. Caption Generator      | OpenAI GPT-4o-mini로 캡션 생성       | 이미지 기반 캡션 자동 생성                                                                                                                                     |
| 7. 파이프라인 통합 테스트           | 0~6단계 연결, Instagram 1:1 출력      | 제품 이미지 한 장 → 최종 이미지+캡션(+메타데이터) 자동 처리. 텍스트 오버레이는 프론트엔드가 담당하도록 이 단계에서는 제외                                                                             |
| 8. Mood×Composition 조합 검증 | 12개 조합 전수 재검증                   | SD1.5→SDXL 교체 직후라 '검증된 안전 조합' 제한 없이 3×4=12개 전체를 제품별(strawberry/sand/latte/salade)로 검증                                                               |

### 2.2.2 SDXL 전환 초기 발견·수정된 버그

| 증상                | 원인                                                                     | 수정                                      |
|-------------------|------------------------------------------------------------------------|-----------------------------------------|
| 배경이 완전히 하얗게 나옴    | SDXL VAE가 fp16에서 오버플로우(NaN)                                            | VAE를 madebyollin/sd-xl-vae-fp16-fix로 교체 |
| VAE 수정 후에도 하얗게 나옴 | create_inpaint_mask()가 '알파>0'이면 무조건 제품(유지)으로 판단 → rembg의 미세 알파 노이즈까지 유 | 임계값을 '알파>10'으로 변경                       |

| 증상                            | 원인                                                     | 수정                                                                 |
|-------------------------------|--------------------------------------------------------|--------------------------------------------------------------------|
|                               | 지 영역으로 오분류                                             |                                                                    |
| 마스크 수정 후에도 하얗게 나옴             | pipe() 호출 시 strength 미지정 → 디노이징 스텝 부족                  | strength=1.0 명시 (디버그 테스트로 원인 확정)                                   |
| CLIP 'input was truncated' 경고 | aerial_shot 프롬프트가 77토큰 초과                              | 문구 축약                                                              |
| 케이크 등에서 그림자 이중으로 겹침           | SD가 이미 배경+그림자를 그리는 데 add_drop_shadow()가 인공 그림자를 추가로 얹음 | composite_product_onto_background()에 add_shadow 파라미터 추가, 기본값 False |
| 배경이 일러스트/애니메이션풍으로 나옴          | 프롬프트에 '사진'임을 명시하는 지시 부재                                | PHOTO_REALISM_PREFIX / PHOTO_REALISM_NEGATIVE 추가                   |

### 2.2.3 프롬프트 반복 히스토리 (1~5 차)

- 1 차(초안): 간단한 톤 묘사. 위 버그들 때문에 실제 품질 검증은 못함
- 2 차(버그 수정 후 재검증): 배경 생성 자체는 성공. 다만 일러스트풍이라 사실적 사진 프리픽스 추가. mood\_natural\_white(브라운/모던으로 감), mood\_vivid(배경 대신 소품 추가) 문제 확인 → MOOD\_NEGATIVE\_EXTRAS/COMPOSITION\_NEGATIVE\_EXTRAS negative 확장 구조 도입
- 3 차(팀 레퍼런스 기반 상세 프롬프트로 교체): 다량의 레퍼런스 이미지 분석 기반 상세 문구로 교체했으나, 원문이 CLIP 77 토큰 한도를 (Composition 단독으로도) 초과 — 실측(open\_clip 토크나이저) 결과 12 개 조합 중 9 개가 초과함을 확인
- 수정: 핵심 지시만 남기고 축약 + product\_description 을 모든 구도의 배경 프롬프트에서 제외(기존엔 aerial\_shot 만 제외) → 12 개 조합 전부 77 토큰 이내(최대 71 토큰)로 확인

#### 2.2.3.1 04 버전 적용 내역

| 변경                    | 내용                                                                                   | 결과                                                       |
|-----------------------|--------------------------------------------------------------------------------------|----------------------------------------------------------|
| Mood별 REALISM 프리픽스 분기 | mood_vivid만 'bold editorial product photography, photorealistic' 별도 사용               | 효과 미미 — mood_vivid는 여전히 배경이 아니라 소품(음료·리본 등)으로만 컬러풀함이 표현됨 |
| mood_wood 문구 순서 조정    | 'wood grain table surface'를 문장 맨 앞으로, muted stone/marble은 'hints of'로 격하             | 톤 자체 추가 불만은 없었으나, 소품 과다 문제는 wood 조합에도 동일하게 남음            |
| 소품 방지 negative 강화     | mood_vivid negative에 'too many objects, muted colors, gray or neutral background' 추가 | 효과 부족 — 다른 Mood(우드/화이트)의 소품 과다는 그대로였고, 와인병은 Mood 전       |

| 변경                | 내용                  | 결과              |
|-------------------|---------------------|-----------------|
|                   |                     | 체에서 반복 등장       |
| guidance_scale 조정 | 7.5 → 8.5, 파라미터로 노출 | 눈에 띄는 개선 확인 안 됨 |

결론: negative/guidance\_scale 조정만으로는 부족했고, positive 프롬프트 자체(소품 개수 제한 문구 복원, 화이트 옵션 제거, 비비드 backdrop 구체화)를 더 손봐야 했음이 확인되어 05 버전에서 반영했다. 특히 지시한 적 없는 와인병이 전 조합에 반복 등장한 것은, PHOTO\_REALISM\_PREFIX의 'professional food photography/editorial' 표현이 SDXL 학습 데이터상 스타일라이프 소품과 강하게 연결되어 있어 생기는 문제로 추정된다.

#### 2.2.3.2 05 버전 적용 내역

| 변경                             | 내용                                                                                                                                   | 결과                                                      |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|
| 소품 개수 제한 문구 복원                 | 'minimal styling with at most one simple prop'을 모든 Mood에 공통 추가(77토큰 축약 과정에서 누락됐던 팀 원본 문구)                                            | 효과 확인 — 와인병 완전히 사라짐, 소품 0~1개 수준으로 감소                    |
| 전역 소품/와인병 negative 추가          | GLOBAL_PROP_NEGATIVE = 'wine bottle, wine glass, multiple props, cluttered tablescape, extra tableware, busy composition' 전 조합 공통 적용 | 효과 확인 — 모든 Mood/제품에서 와인병 재현 안 됨                         |
| mood_natural_white 옵션 제거       | 'plain white or soft warm-gray' → 'bright white background'                                                                          | 효과 없음 — 여전히 그레이 톤                                       |
| mood_natural_white negative 강화 | 'gray background, dark background' 추가                                                                                                | 효과 없음                                                   |
| mood_vivid backdrop 구체화        | 'bold solid-color background' → 'bold colorful tablecloth or patterned backdrop'                                                     | 제품별 편차 큼 — 딸기는 4구도 전부 성공, 커피는 1/4만 성공, 샌드위치·샐러드는 반영 안 됨 |

05 버전에서도 12개 조합 전부 CLIP 77토큰 이내(최대 74토큰, negative 최대 55토큰)로 재검증했다. 소품 과다 문제는 확실히 해결됐으나, mood\_natural\_white(그레이 고착)와 mood\_vivid(제품별 편차)는 positive/negative 프롬프트 조정만으로는 해결되지 않았다.

#### 2.2.4 SDXL 프롬프트 엔지니어링의 구조적 한계 → FLUX 전환 결정

mood\_natural\_white·mood\_vivid 문제와 handheld\_lifestyle의 손 왜곡까지 종합했을 때, SDXL 텍스트 프롬프트 조합으로는 다음 두 가지 구조적 한계에 도달했다고 판단했다:

- CLIP 77 토큰 한도 — 상세한 레퍼런스 기반 프롬프트를 다 담을 수 없음

- 텍스트 기반 색감·스타일 재현력의 한계 — negative/guidance\_scale 조정으로 해결되지 않는 프롬프트-이미지 정합 문제

이에 따라 프롬프트를 더 미세조정하는 대신, FLUX.1-Fill-dev(텍스트 인코더가 T5라 토큰 한도 문제 없음, prompt adherence가 SDXL보다 강함, 손 품질 개선 여지)로 모델을 교체해 결과를 먼저 확인하기로 결정했다. 단, FLUX.1 [dev]는 Non-Commercial License로, 실제 상용 서비스 배포 전 상업 라이선스 확인이 필요하다(검증·평가 단계에서의 사용은 허용됨).

#### 2.2.4.1 참고: 검토했던 향후 튜닝 기법

| 기법         | 학습 필요 여부             | 설명                               |
|------------|----------------------|----------------------------------|
| DreamBooth | 필요 (무거운 Fine-tuning) | 모델 대부분을 소량 이미지로 재학습              |
| LoRA       | 필요 (경량 Fine-tuning)  | 작은 추가 레이어만 학습                    |
| ControlNet | 불필요 (기학습 모델 활용)      | Depth/Edge/Pose 등 구조 정보를 조건으로 추가 |
| IP-Adapter | 불필요 (기학습 모델 활용)      | 텍스트 대신 레퍼런스 이미지를 조건으로 사용         |

우선순위: ControlNet(Depth) → IP-Adapter → LoRA → DreamBooth(보류). 다만 실제로는 FLUX.1 Kontext로 먼저 전환하는 방향으로 진행되었다.

## 2.3 3 단계 — FLUX.1 Kontext(v2) 도입과 실패

v1은 FLUX.1 Kontext 모델 하나로 배경 제거·구도 배치·합성·조명 보정까지 전부 처리하려는 시도였다. 한 모델에게 여러 작업을 동시에 맡기려다 아래와 같은 구조적 문제를 반복적으로 겪었고, 이것이 v2 전면 재설계의 직접적 동기가 되었다.

| 문제                | 원인                                                                                                                                                                      |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 네이티브 멀티 이미지 입력 불가 | diffusers 라이브러리 자체 버그(huggingface/diffusers#11824). 제품+장면 두 이미지를 리스트로 넣으면 shape 오류로 실행 자체가 안 됨                                                                          |
| 스티칭(콜라주) 방식의 실패   | 제품·장면을 한 캔버스에 붙여 넣는 방식으로 우회했으나, 모델이 '왼쪽을 참고해 오른쪽에 배치' 같은 공간 지시를 이해하지 못함.<br>guidance_scale·true_cfg_scale을 올려도 장면을 무시하고 제품만 반복 출력                                       |
| 인페인팅 파이프라인 버그     | FluxKontextInpaintPipeline에 height/width를 명시하지 않으면 1024×1024로 강제 크롭되어 결과 이미지 절반이 검게 나옴.<br>from_pipe()로 컴포넌트를 공유할 때 cpu_offload 혹은 새로 등록되지 않아 dtype 불일치 RuntimeError 발생 |

| 문제                           | 원인                                                                                                                              |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| 참조 이미지 삽입 용도의 근본적 한계         | diffusers 메인테이너가 GitHub에서 직접 확인: 'Kontext 인페인팅은 텍스트 지시만으로는 잘 되지만, 참조 이미지를 마스크 영역에 삽입하는 용도로는 학습되지 않았다.' 즉 필요한 작업 자체가 모델의 설계 범위 밖 |
| 손(handheld 구도) 자연스러운 생성의 어려움 | diffusion 계열 모델 전반의 공통적 약점. 라이프스타일·자연광 콘셉트에서 특히 치명적                                                                             |

결론적으로, 하나의 범용 생성 모델에게 '배경 제거+객체 제거+정확한 위치 합성+조명 보정'을 모두 맡기는 접근 자체가 실패 원인으로 판명되어, v2에서는 각 단계를 전문 모델로 분리하는 방향으로 전면 재설계했다.

## 2.4 4 단계 — v3: 다중 모델 조합 아키텍처

### 2.4.1 설계 원칙

- 하나의 범용 모델이 전부 처리하게 하지 않고, 각 단계를 그 작업에 가장 특화된 전문 모델에게 맡긴다
- 상업적으로 이용 가능한 라이선스의 오픈소스 모델만 사용하고, 유료 API 호출 없이 전부 자체 호스팅한다(운영 비용 = GPU 사용료 외 0 원)
- Colab GPU(T4/L4) 환경에서 안정적으로 동작하는 가벼운 모델 위주로 구성한다
- 모델이 잘 못하는 영역(공간 추론, 형태 변형, 구도 판단)은 모델에게 억지로 시키지 않고, 사전 태깅된 데이터와 결정론적 이미지 처리로 우회한다

### 2.4.2 전체 파이프라인 구조 (6 단계)

흐름: 제품 사진 → ①배경 제거 / 템플릿+태깅 좌표 → 템플릿·슬롯 선택 → ②기존 물체 제거 → ③합성 배치 → ④조명·색감 조화 → ⑤LUT 색보정 → ⑥1:1 크롭 → 최종 결과물

| 단계             | 담당                            | 라이선스/성격                         |
|----------------|-------------------------------|---------------------------------|
| ① 제품 배경 제거     | BiRefNet (rembg 내장)           | 상업 이용 가능                        |
| ② 템플릿 기존 물체 제거 | LaMa (simple-lama-inpainting) | Apache 2.0                      |
| ③ 합성 배치        | 자체 코드 (PIL)                   | 모델 아님                           |
| ④ 조명·색감 조화     | 자체 코드 (1차: 클래식 컬러 트랜스퍼 +그림자)  | 모델 아님 (추후 IC-Light v1 업그레이드 예정) |
| ⑤ LUT 색보정      | 자체 코드 (톤 커브)                  | 모델 아님                           |
| ⑥ 인스타그램 1:1 크롭 | 자체 코드                         | 모델 아님                           |

## 2.4.3 단계별 상세 — 기능, 인풋·아웃풋, 선정 이유

### 2.4.3.1 ① BiRefNet — 제품 배경 제거

인풋: 제품 사진 원본(RGB) → 아웃풋: 배경이 투명한 RGBA 컷아웃(여백 크롭 포함). rembg 기본 모델(u2net)보다 유리컵·투명 재질, 복잡한 경계에서 훨씬 정확해 채택. 제품 사진 상당수가 유리컵 음료라 정확도가 특히 중요했고, rembg 패키지 안에서 모델명만 바꾸면 되어 통합 비용도 낮았다.

### 2.4.3.2 ② LaMa — 템플릿의 기존 물체 제거

인풋: 원본 템플릿 + 제거 영역 마스크(빨대·가니시 포함 넉넉하게) → 아웃풋: 마스크 영역이 자연스럽게 메워진 빈 장면. 텍스트 지시 없이 순수 '제거'만 수행해 결과가 안정적이고 예측 가능하며, 커뮤니티에서 수년간 검증되어 FLUX Kontext 인페인팅 때 겪은 신생 버그 위험이 낮다.

### 2.4.3.3 ③ 합성 배치 — 자체 코드(PIL)

인풋: 제품 컷아웃 + 정리된 장면 + 태깅된 배치 좌표(slot) → 아웃풋: 제품이 지정 위치에 비율 유지한 채 놓인 합성 이미지. 모델이 아니라 순수 이미지 처리로, slot 높이에 맞춰 리사이즈 후 가로 중앙 정렬해 붙여 찌그러짐이 없다.

### 2.4.3.4 ④ 조명·색감 조화 — 1 차 버전(모델 없음), IC-Light 업그레이드 예정

본래 계획은 IC-Light(V1, 상업 이용 가능 — V2는 비상업 라이선스라 제외)였으나, SD1.5 UNet 구조를 직접 패치해야 하는 복잡한 통합이라, 나머지 파이프라인의 안정성을 먼저 검증하기 위해 1차 버전은 모델 없이 구현했다: (1) LAB 색공간 Reinhard 색전이로 제품 색 톤을 배경에 맞추고, (2) 제품 아래 타원형 그라데이션 그림자를 합성. 골격이 검증되면 이 함수만 IC-Light로 교체할 예정이었다.

### 2.4.3.5 ⑤ LUT 색보정 / ⑥ 인스타그램 1:1 크롭 — 자체 코드

톤 커브 프리셋(warm\_caramel/cool\_moody/soft\_pastel) 적용. 크롭은 단순 중앙 크롭 대신 제품이 실제 배치된 위치(paste\_box)를 중심으로 계산해, 제품이 한쪽에 치우친 사진에서도 잘려나가지 않도록 했다.

## 2.4.4 템플릿·슬롯 선택 로직 & container\_type 태깅

카테고리(drink/food 등)가 일치하는 모든 템플릿·슬롯을 후보로 모은 뒤, 업로드된 제품 사진의 가로세로 비율과 가장 비슷한 슬롯을 우선하되, 상위 약 40%(diversity\_fraction) 후보군 안에서 무작위로 선택한다. 완전 최적값만 매번 고르면 레퍼런스가 적은 단계에서 특정 비율마다 항상 같은 사진이 반복 선택되는 문제가 있어 절충한 것이다.

제품이 유리컵 음료인데 매칭된 레퍼런스가 테이크아웃컵이면 부자연스러운 결과가 나온다. 이를 모델이 용기 형태를 변형해 해결하면 v1에서 실패한 '모델에게 형태 변형을 맡기는' 위험을 재도입하게 되므로, 태깅 단계에서 용기 종류(glass/takeout\_cup/mug 등)를 함께 기록하고 제품의 용기 종류와 일치하는 레퍼런스를 우선 매칭하는 방식으로 해결했다.

## 2.4.5 API 기반 모델(OpenAI GPT-image, FLUX.2)을 채택하지 않은 이유

- gpt-image-1.5/2: 정부 발급 신분증+실시간 셀피 기반 '조직 인증'이 필요한데 현재 진행이 어려운 상황이라 접근 자체가 막혀 있음

- gpt-image-1-mini: 조직 인증 없이 사용 가능하나, OpenAI 공식 가이드상 '고객에게 보여줄 결과물·포토리얼리즘·편집 위주 작업'에는 상위 모델을 권장하고 mini 는 탐색적·저중요도 대량 생성에만 권장됨 — 우리 작업 범위를 벗어남
- FLUX.2: 네이티브 멀티 이미지 입력으로 FLUX.1 Kontext 의 스티칭 문제는 해결하지만, 로컬 무료 버전([dev]/[Klein] 9B)은 비상업 라이선스, 상업 라이선스 버전([pro])은 유료 API, Klein 4B(Apache 2.0)는 경량이라 품질 제한적

'운영자 입장에서 API 호출은 계속 나가는 변동비'라는 문제의식 아래, 위 두 계열 모두 제외하고 전부 상업 이용 가능한 오픈소스 모델을 자체 호스팅하는 v2 아키텍처(BiRefNet, LaMa, 추후 IC-Light V1)로 확정했다.

#### 2.4.6 '라이팅 시트' 아이디어를 채택하지 않은 이유

팀원 제안: 복잡한 프롬프트보다 원하는 무드를 가장 잘 보여주는 이미지 한 장을 먼저 만들고, 그 이미지를 기준점 삼아 라이팅 시트를 확장하면 텍스트 프롬프트만 쓸 때보다 일관되고 영화적인 결과가 나온다는 아이디어. 검토 결과, 이 기법은 배경 자체를 텍스트로부터 새로 생성하는 워크플로우(예: GPT-image로 배경까지 생성)에서 특히 유효하다는 결론에 이르렀다.

| 구분       | 라이팅 시트 방식            | v2 파이프라인                                   |
|----------|----------------------|--------------------------------------------|
| 무드 기준점   | 무드를 대표하는 근사 이미지 1장   | 실제 타겟 배경 사진 그 자체 (④ 단계가 진짜 배경을 직접 보고 조화시킴) |
| 톤 일관성 확보 | 이미지 기반 프롬프트로 유도(확률적) | LUT 색보정 — 수학적 연산으로 100% 결정론적               |

라이팅 시트의 핵심('근사치 텍스트보다 이미지 기준점이 낫다')을, v2는 '근사치 이미지도 아니고 실제 배경 그 자체+결정론적 색보정'으로 더 강하게 확보하고 있어 별도 도입이 불필요하다고 판단했다. 다만 향후 ④번 단계를 텍스트 프롬프트 기반 생성형 모델로 교체할 경우 재검토 대상이다.

#### 2.4.7 레퍼런스 이미지 큐레이션 전략

초기 287~298장이었던 레퍼런스를 카테고리당 5장 내외로 축소했다. (1) 태깅은 사람이 위치·카테고리·용기종류를 직접 지정하는 수작업이라 장수에 비례해 비용이 늘어나고, (2) 기울어진 각도·물체가 겹친 사진 등 저품질 레퍼런스가 합성 결과 품질을 직접 깎아먹는 것을 실험으로 확인했기 때문이다. 실제 서비스 배포 시에는 반복 노출 문제를 줄이기 위해 레퍼런스를 늘리는 것이 유리하나, 이는 (1) 파이프라인 안정성 검증, (2) 태깅 자동화/인력 투입, (3) 레퍼런스 사진 상업적 이용 권리 확인 이후 단계적으로 진행하는 것이 맞다고 판단했다.

#### 2.4.8 로컬 도구

원본 사진 전처리(preprocess.py)와 배치 좌표 태깅(template\_placement\_tool.ipynb)을 하나의 로컬 실행 노트북(template\_prep\_and\_tagging.ipynb)으로 통합했다. GPU·Colab 없이 로컬에서 실행하며, EXIF 방향 보정·RGB 통일·리사이즈 전처리, 배경색 차이 기반/YOLO 기반 자동 배치 검출, 카테고리별 contact sheet 검수, 마우스 드래그 인터랙티브 태깅(위치+카테고리+용기 종류) 기능을 포함한다.

## 2.4.9 v3 실행 중 발견·해결된 버그

| 문제                                   | 원인                                                                                           | 해결                                                                                                            |
|--------------------------------------|----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| 템플릿이 전부 0장으로 반환됨                     | 실제 이미지가 든 폴더명이 templates_lib가 아니라 오타(templates_lib)로 압축되어 있었음                                | 폴더명을 되돌리는 이름 변경 셀 추가. 근본적으로는 로컬 전처리 도구의 경로 변수 오타를 고쳐야 함                                                       |
| 카테고리 태그가 있는데도 슬롯을 못 찾음               | templates_lib_placement.json의 key가 로컬(Windows) 절대경로, Colab은 Linux 절대경로라 문자열이 완전히 달라 항상 매칭 실패 | 경로 대신 파일명(basename) 기준 매칭으로 수정 — 즉시 해결됐으나 파일명 중복 시 다른 폴더의 슬롯을 잘못 가져올 위험이 남아 장기적으로 상대경로 /template_id 기반 구조가 필요 |
| pick_template_and_slot()이 항상 None 반환 | 함수 수정 과정에서 마지막 return random.Random(seed).choice(candidates)문이 누락됨(Python은 자동으로 None 반환)     | 누락된 return 문 추가                                                                                               |
| rembg импорт 시 CUDA 관련 ImportError   | onnxruntime-gpu가 요구하는 CUDA 버전과 Colab 실제 설치 버전 불일치                                            | rembg[gpu]+onnxruntime-gpu 대신 rembg[cpu]로 전환. BiRefNet은 가벼운 모델이라 CPU로도 충분                                     |
| LaMa 제거 후 제품 주변이 넓게 블러 처리됨           | 빨대 등 돌출 요소까지 지우려 padding_ratio=0.5의 큰 사각형 마스크 사용 — 주변 테이블·질감까지 LaMa가 재생성                     | padding_ratio를 0.1 수준으로 축소했으나 이번엔 빨대·장식이 슬롯 밖에 남는 문제 재발 — 사각형 마스크 방식 자체의 구조적 한계로 확인, v3에서 정밀 마스크로 전환 필요       |

## 2.4.10 v3 출력 결과물에서 확인된 부족한 점

### 2.4.10.1 (1) 합성물이 '붙어 있는' 느낌 (스티커·부유감)

- 접촉 그림자 부재 — 제품과 배경이 맞닿는 지점의 짧고 진한 그림자가 없어 물체가 떠 보임
- 원근 불일치 — 제품 촬영 각도와 배경 각도가 서로 다름
- 조명 방향 불일치 — 배경은 좌측 자연광인데 제품 명암은 평평하거나 다른 방향
- 경계가 과도하게 선명함 — 배경 제거 후 PNG 경계가 실제 카메라 촬영보다 훨씬 깨끗함
- 색온도·선명도 차이 — 제품 컷아웃은 차갑고 선명한데 배경은 따뜻하고 부드러움

### 2.4.10.2 (2) 색감·조명 조화가 제품 고유색을 손상시킴

전역 통계 기반 Reinhard color transfer를 강하게 적용한 결과, 딸기·복숭아·민트 등 제품 고유색이 탁해지고 유리컵 내부 명암이 평평해지며 전체적으로 회색 막이 씌워진 듯한 결과가 관찰됐다. 저채도 배경의 평균값을 제품 전체에 그대로 적용하는 방식의 근본적 한계다.

2.4.10.3 (3) 정밀하지 않은 제거 마스크로 인한 트레이드오프

사각형 마스크의 여백을 넓히면 블러가 발생하고 좁히면 돌출 요소가 남는 상충 관계 — 숫자 조정으로 해결할 수 없는 마스크 형태 자체의 한계.

2.4.11 v3의 예상 품질 수준

| 평가 항목    | v2 예상 점수(10점 만점) | 판단                            |
|----------|------------------|-------------------------------|
| 배경 제거    | 9.0~9.5          | BiRefNet으로 상당히 양호             |
| 제품 형태 보존 | 9.0              | PIL 배치 방식이라 왜곡 위험 낮음          |
| 구도       | 8.0~8.5          | 사전 태깅된 슬롯 기반이라 안정적            |
| 배경 선택    | 8.0              | 고품질 템플릿 큐레이션에 좌우됨             |
| 색감       | 6.5~7.0          | 전역 컬러 트랜스퍼의 부작용 존재            |
| 조명       | 5.5~6.0          | 광원 방향·재조명 처리 부족               |
| 그림자      | 4.5~5.5          | 단일 타원 그림자는 접촉감이 약함            |
| 실사감      | 5.5~6.5          | 스티커·부유 현상 발생 가능               |
| 인스타그램 감성 | 6.0~7.0          | 템플릿 자체 품질은 좋으나 합성의 물리적 사실감 부족 |

종합 판단: v2는 약 65~75점 수준의 MVP를 만들 수 있는 구조다. IC-Light, Depth 정보, contact shadow, 재질별 처리, 카메라 매칭, 정량 평가 체계를 추가하면 75~85점 수준까지 상승할 가능성이 있다 — 이것이 v3 설계의 출발점이다.

2.5 5 단계 — v3 → v4 전환 이유

2.5.1 핵심 병목 6 가지

| 병목                   | 설명                                                   |
|----------------------|------------------------------------------------------|
| Lighting consistency | 배경 광원 방향과 제품의 광원 방향이 다르면, 색이 맞아도 합성 티가 즉시 남          |
| Contact relation     | 제품이 테이블·접시에 실제로 닿아 있다는 접촉 그림자와 ambient occlusion이 없음 |

| 병목                              | 설명                                             |
|---------------------------------|------------------------------------------------|
| Depth and occlusion             | 바닥 평면, 앞뒤 관계, 크기·원근이 충분히 반영되지 않음               |
| Transparent/reflection material | 유리컵·얼음·음료는 알파 채널 하나만으로는 자연스럽게 처리하기 어려움         |
| Camera matching                 | 초점, 선명도, 렌즈감, 카메라 높이와 촬영 각도가 제품과 배경 사이에서 서로 다름 |
| Template quality & metadata     | 결과 품질의 상당 부분은 모델보다 템플릿 큐레이션과 태깅 정밀도에 좌우됨       |

## 2.5.2 왜 파라미터 조정만으로는 한계인가

- 마스크 여백을 넓히면 블러가 생기고, 좁히면 돌출 요소가 남는다 — 사각형 마스크라는 '형태' 자체의 한계
- 전역 색보정 강도를 낮추면 조화가 약해지고, 높이면 제품 고유색이 죽는다 — '전역 통계 기반 보정'이라는 '접근 방식' 자체의 한계
- 그림자를 진하게 하면 인공적이고, 얇게 하면 접촉감이 안 산다 — '그림자 1 개로 접촉+캐스트를 동시에 처리'하는 '역할 미분리'의 한계

즉 지금까지의 문제는 값 하나를 잘못 골라서가 아니라, 애초에 그 작업에 필요한 정보(깊이, 재질, 조명 방향, 카메라 각도)를 파이프라인이 갖고 있지 않아서 생긴 구조적 한계였다. v3는 이 정보들을 명시적으로 추정·활용하는 방향으로 설계한다.

## 2.5.3 v4의 목표 수준

v2의 65~75점(MVP) 수준에서, 사람이 실제로 촬영한 것처럼 느껴지는 75~85점 수준의 Instagramable 결과물을 목표로 한다. 이를 위해 조명 일치, 접촉 관계, 깊이·가림, 재질별 처리, 카메라 매칭, 정량 평가 체계를 v3에 추가한다.

## 2.6 6 단계 — v4 전체 파이프라인 개요

사용자 제품 사진 한 장이 입력되어 다음 순서로 처리된다: BiRefNet 배경 제거 → SAM2 선택적 마스크 보정(얇은 부분·투명 경계 보완) → container/material/camera\_angle classifier(CLIP 제로샷) → 템플릿 매칭(category·container\_type·camera\_angle 엄격 일치, 안 맞으면 생성 중단) → TemplateCleaner(SAM2 정밀 마스크+겹치는 부속물 자동 병합+LaMa/API Edit) → SurfaceAligner(제거 마스크 기울기로 제품 회전 보정, roll) → PitchApproximator/PhysicalSizeApproximator(부감·실물크기 근사 보정) → Depth Anything V2 Small로 배경의 바닥 평면·원근·가림 분석 → perspective-aware placement → IC-Light V1 재조명 → contact shadow+cast shadow 엔진(역할 분리) → 투명 재질·반사 처리 → local LAB harmonization(제품 고유색 보존형 색감 조화) → EdgeHarmonizer(합성 경계 국소 재 생성) → smart crop(rule of thirds/safe margin) → quality evaluator(정량 평가) → 상위 후보 3장 출력.

v2와 비교하면, ①~③(배경 제거·템플릿 정리·합성 배치)의 기본 골격은 유지하되 각 단계 사이에 깊이·재질·조명 정보를 추정하는 단계가 추가되고, 마지막에 정량 평가로 여러 후보 중 상위 결과만 걸러내는 구조로 확장된다.

### 2.6.1 v4 컴포넌트별 역할 (v3 대비 변경점)

| 컴포넌트                                           | 역할 (v2 대비 변경점)                                                                                                                          |
|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| BackgroundRemover                              | BiRefNet 그대로 유지 + SAM2로 얇은 부분·투명 경계 보정 추가                                                                                               |
| TemplateMatcher                                | 파일명 매칭 → 상대경로/template_id 기반으로 강화,<br>container_type-camera_angle까지 매칭 조건에 포함                                                           |
| TemplateCleaner                                | 사각형 마스크 대신 본체+extras 정밀 마스크, cleaned-template 캐시 도입. LaMa 결과가 기준 미달일 때만 OpenAI Image Edit로 국소 복원. 겹치는 다른 슬롯(빨대·손잡이 등)도 카테고리 무관하게 자동 마스크 |
| Compositor                                     | 단순 중앙 배치 → 깊이 기반 perspective-aware 배치로 확장                                                                                               |
| LightingHarmonizer                             | 전역 Reinhard → local LAB 기반 선택적 조화(제품색 보존) + IC-Light V1 재조명으로 확장                                                                        |
| ShadowEngine                                   | 단일 타원 그림자 → contact/cast shadow 역할 분리 + ambient occlusion                                                                               |
| MaterialProcessor (신규)                         | 유리·반사·투명 액체 등 재질별 규칙 적용                                                                                                                 |
| Cropper                                        | 중심 크롭 유지 + rule of thirds/safe margin 고려 확장                                                                                             |
| Evaluator (신규)                                 | 규칙 기반 평가 + 추후 랭킹 모델, 상위 후보 3장 선별                                                                                                        |
| Logger/Config (신규)                             | print()/함수 인자 산재 → logging + config.yaml/dataclass로 통합                                                                                  |
| Container/Material/CameraAngle Classifier (신규) | 학습 데이터 없이도 즉시 쓸 수 있도록 CLIP 제로샷으로 자동 분류. manual_* 입                                                                                      |

| 컴포넌트                                                           | 역할 (v2 대비 변경점)                                                                                    |
|----------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
|                                                                | 력 시 그 값을 항상 우선                                                                                    |
| SurfaceAligner/PitchApproximator/PhysicalSizeApproximator (신규) | 제거 마스크 모양으로 촬영 각도 (roll·pitch)를 근사 추정해 제품을 회전·세로압축, container_type 평균 실물 크기표로 상대 크기 보정 (전부 근사 버전) |
| EdgeHarmonizer (신규)                                            | 제품 실루엣 링+접촉부+LaMa 흔적을 합친 마스크만 국소 재생성. 편집 면적 상한과 품질 평가 기반 롤백으로 이중 안전장치                             |

## 2.7 7 단계 — v3·v4 사용 모델 상세

### 2.7.1 BiRefNet (v3 부터 유지)

기능: 고정밀 배경 제거(salient object segmentation). 장점: rembg 내장으로 통합 간편, u2net보다 유리컵·투명 재질·섬세한 경계에서 정확, 상업 이용 가능. 단점: 완전히 투명하거나 반사가 심한 재질은 알파 채널만으로는 표현 한계 — MaterialProcessor로 보완. 워크플로우: RGB → 세그멘테이션 마스크 → RGBA 컷아웃. 라이선스: 상업 이용 가능.

### 2.7.2 LaMa (v3 부터 사용, v4 에서 마스크 정밀화)

기능: 마스크 지정 영역을 주변 텍스처 참고해 자연스럽게 채우는 인페인팅(객체 제거) 전용 모델. 장점: 텍스트 조건 불필요라 결과가 안정적, Apache 2.0으로 상업 이용 자유, 가볍고 빠름. 단점: 사각형 마스크만 쓰면 여백 확대→블러, 축소→잔여물 트레이드오프. 정밀 실루엣 마스크가 없으면 한계를 넘기 어려움. 워크플로우: 배경+제거 마스크 → 인페인팅 결과. v3에서는 SAM2 정밀 마스크를 입력으로 사용. 라이선스: Apache 2.0.

### 2.7.3 SAM2 (v4 신규)

정밀 실루엣 마스크 생성에 사용. (a) 태깅 시 사람이 지정한 박스를 프롬프트로 입력 → 정밀 실루엣 마스크 반환(LaMa 제거 마스크 정밀화), (b) 제품 컷아웃 후 빨대·손잡이 등 얇은 잔여 영역 보정에도 사용. 라이선스: Apache 2.0.

### 2.7.4 Depth Anything V2 (Small) (v4 신규)

기능: 이미지 한 장으로 장면의 상대적 깊이(원근) 지도를 추정하는 단안 깊이 추정 모델. 장점: Small 모델은 Apache 2.0로 완전 상업 이용 가능(Base-Large-Giant는 CC-BY-NC-4.0 비상업이라 반드시 Small만 사용), 확산 모델 기반 깊이 추정 대비 10배 이상 빠르고 경량(약 25M 파라미터). 단점: 정밀도가 낮고, 절대 거리가 아닌 상대적 깊이이라 실제 배치 계산에는 별도 스케일 보정 필요. 워크플로우: 배경 사진 →

깊이 지도 → 바닥 평면 추정, 배치 위치의 원근에 맞는 크기·위치 보정, 그림자 방향·형태 계산에 활용.  
라이선스: Apache 2.0(Small 한정).

### 2.7.5 IC-Light V1 (v4 신규 도입 시도)

기능: 전경(제품) 이미지의 조명을 배경 조건에 맞게 재계산(relighting)하는 SD1.5 기반 확산 모델. 장점: '조명 재계산'이라는 좁은 임무만 수행해 예측 가능성이 높고, V1은 상업 이용 가능(V2는 비상업이라 제외), SD1.5 기반이라 FLUX 계열 대비 훨씬 가벼움. 단점: SD1.5 UNet 입력 채널 구조를 직접 패치해야 하는 통합 난이도, 완전히 결정론적이지 않아 v2에서 겪은 예측 불가능성 문제가 부분 재도입될 위험. 라이선스: V1만 상업 이용 가능.

※ 후술하듯 v3\_final 세션(2026-07-20)에서 forward hook 누락이라는 구조적 버그로 실제 동작한 적이 없었음이 확인되어, 최종적으로 비활성화(기본값 False)로 확정되었다(9.2절 참고).

### 2.7.6 Container/Material Classifier (v4 신규)

제품 이미지가 어떤 용기·재질인지 분류하는 경량 이미지 분류 모델(궁극적으로는 학습 예정, 현재는 CLIP 제로샷으로 대체). 라벨링된 학습 데이터를 확보해야 하며, 학습 전까지는 태깅 도구의 수동 입력 값 또는 CLIP 제로샷 예측으로 대체된다.

### 2.7.7 Evaluator/Ranking (v4 신규)

여러 생성 후보 중 어떤 것이 더 자연스러운지 정량 평가하고 순위를 매기는 컴포넌트. 초기엔 규칙 기반(경계 부드러움, 색 편차, 그림자 유무 등 통계 지표)으로 시작하고, 사람이 좋음/나쁨을 라벨링한 데이터가 쌓이면 학습 기반 랭킹 모델로 교체 예정.

### 2.7.8 OpenAI gpt-image-2 (Image Edit) (v4 신규, 조건부 폴백 전용)

LaMa가 실패하는 복잡한 텍스처(반복 무늬, 곡면 등)에서 시도해볼 수 있는 유일한 대안. 조직 인증이 필요할 수 있고 종량제 API라 원칙 (2)와 충돌하므로 반드시 LaMa 실패 시에만 호출하는 폴백으로 한정한다. 공식 모델 페이지에 deprecated로 표시돼 있어 장기 고정 모델이 아니라 교체 가능한 provider로 취급한다. 마스크는 정확한 픽셀 경계 명령이 아니라 프롬프트 기반 안내일 뿐이므로 결과를 그대로 쓰지 않고 원본 픽셀 잠금 후처리가 반드시 필요하다.

### 2.7.9 CLIP (v4 신규)

이미지·텍스트를 같은 임베딩 공간에 놓고 유사도를 비교, 제로샷 분류에 사용. 파인튜닝·라벨링 데이터 없이 바로 사용 가능하다는 점이 핵심 — container\_type·material\_type·camera\_angle 태깅 데이터가 각각 5%, 0%뿐인 상황에서 '학습 없이 구현 가능한 것부터' 시작하는 로드맵에 정확히 부합. MIT 라이선스로 상업 이용 가능, 모델 크기(~600MB)도 가볍다. 단점: 전용 학습 분류기보다 정확도가 낮을 수 있고, 후보 문장끼리 의미가 겹치면 확률이 애매하게 갈릴 수 있다.

## 2.8 8 단계 — v4 고도화 로직 상세

### 2.8.1 Local LAB 기반 선택적 색감 조화 (전역 Reinhard 대체)

색감 조화를 '제품을 배경색으로 물들이는 작업'이 아니라, 제품 고유색 대비 채도는 보존하고 밝기·색온도·환경광만 제한적으로 맞추는 작업으로 재정의했다:

- 배경 전체가 아니라 배치 위치(paste\_box) 주변 컨텍스트 영역만 색상 통계 샘플로 사용
- LAB 색공간에서 L(명암) 채널은 20~35%, a/b(색상) 채널은 5~15%만 배경 쪽으로 조정
- 조정 결과를 원본과 15~35%만 혼합(blend)해 과도한 보정 방지
- 채도 높은 영역(과일·라벨 등)은 보호 마스크로 원본 색을 강하게 유지
- 원본 채도의 70~85% 밑으로 떨어지지 않도록 하한선
- 원본의 미세 대비(고주파 성분)를 15~30% 복원
- 제품·배경 색 차이 크기에 따라 강도를 자동 조정(차이가 작으면 약하게, 크면 강하게)

권장 기본값: luminance\_strength 0.28 / chroma\_strength 0.08 / global\_blend\_strength 0.25 / saturation\_preserve\_ratio 0.80 / detail\_restore\_strength 0.20

### 2.8.2 Contact Shadow · Cast Shadow 분리 + Ambient Occlusion

V3의 단일 타원 그림자를 두 역할로 분리했다. 하나의 그림자로 두 역할을 동시에 수행하면 제품마다 부자연스러워지기 때문이다.

| 종류                | 역할                                     | 권장 특성                                                         |
|-------------------|----------------------------------------|---------------------------------------------------------------|
| Contact shadow    | 제품과 표면이 맞닿는 지점의 그림자 — '실제로 놓여 있다'는 접촉감 | 짧고 진하게, blur 작게(opacity 60~100, blur 3~7)                     |
| Cast shadow       | 광원 반대 방향으로 넓게 드리우는 그림자                 | 넓고 흐리게, 광원 반대 방향 offset(opacity 25~45, blur 8~14)             |
| Ambient Occlusion | 접촉부의 아주 좁은 영역을 전체 그림자보다 더 진하고 덜 흐리게 처리 | 제품이 표면에 눌러앉은 감각 부여 — IC-Light 적용 후에도 AO가 없으면 '불인 느낌'이 남을 수 있음 |

### 2.8.3 Depth 기반 Perspective-aware Placement

Depth Anything V2 Small로 추정된 배경 깊이 지도로 바닥 평면·원근을 분석하고, 제품 배치 크기·위치·기울기를 그 원근에 맞게 조정한다. 촬영 각도 차이가 큰 경우 약한 세로 압축(vertical\_scale 0.88~0.94)으로 보정하되, 차이가 지나치게 크면 이 보정만으로 완전히 자연스럽게 만들 수 없어 템플릿 매칭 단계부터 제품·슬롯의 camera\_angle을 비교해 각도 차이가 작은 조합을 선택하는 것이 근본적 해결책이다.

### 2.8.4 Smart Crop (rule of thirds · safe margin · negative space)

paste\_box 중심 크롭은 유지하되, 인스타그램 감성을 고려해 삼분할 구도(rule of thirds), 여백 안전 영역(safe margin), 네거티브 스페이스까지 고려한 크롭 로직으로 확장했다.

## 2.8.5 배경 편집 하이브리드: 국소 API Edit 풀백

### 2.8.5.1 1 배경을 통째로 재생성하지 않는 이유

V3의 '붙어 있는 느낌'과 실사감 부족 문제를 해결하려 배경 자체를 확산 모델로 새로 생성하면, 같은 종류의 위화감이 배경 쪽에서도 재현된다. 확산 모델은 방대한 이미지의 통계적 분포를 학습해 '가장 그럴듯한' 픽셀 배치를 생성하므로, 실제 사진의 무작위성(센서 노이즈, 미세한 초점 흐림, 불균일한 조명)이 오히려 지나치게 균일하게 다듬어지는 경향이 있고, 빛의 반사·그림자를 물리적으로 시뮬레이션하는 게 아니라 학습된 패턴을 모사하는 방식이라 광원 방향·그림자 경도가 미묘하게 부자연스러운 경우가 잦다.

실제 A/B 테스트(동일 제품·구도, GPT-4o-mini 비전 채점, 7개 항목 합계 35점)에서도 배경 전체 생성 하이브리드는 21점으로 실사 템플릿+LaMa(30점) 대비 낮았고, 특히 제품 접지감·원근 일치도에서 격차가 컸다. 국소 Edit는 편집 마스크를 전체 이미지의 15% 이하로 제한하고 마스크 바깥은 원본 픽셀을 그대로 잠그므로, 배경의 90% 이상이 실제 카메라로 촬영된 픽셀로 유지된다.

### 2.8.5.2 2 TemplateCleaner 내부 처리 흐름

SAM2 정밀 제거 마스크 → LaMa 1차 제거 → BackgroundRepairEvaluator 품질 평가 → (통과: cleaned background 확정 / 미달: OpenAI Image Edit 국소 복원) → 원본 픽셀 잠금 → 구조 보존 검사 → cleaned-template 캐시. LaMa 우선, API Edit는 후순위 풀백이라는 점이 핵심이다.

### 2.8.5.3 3 원칙: 전체 이미지 Edit 금지

템플릿 전체와 프롬프트만 보내 편집을 요청하면 벽·테이블 색 변화, 렌즈 심도·노이즈 변화, 기존 슬롯·광원·깊이 메타데이터 무효화 문제가 생긴다. OpenAI 공식 문서도 마스크는 정확한 픽셀 경계 명령이 아니라 프롬프트 기반 안내일 뿐이라고 설명하므로, 마스크를 최소 범위로 제한한 복원 전용 프롬프트만 사용한다(핵심 표현은 'Create a beautiful table'이 아니라 'Reconstruct the original surface' — '예쁘게'는 창작 허가증이고, 필요한 건 창작이 아니라 수선이다).

### 2.8.5.4 4 원본 픽셀 잠금 및 구조 보존 검사

API 결과를 그대로 쓰지 않고 편집 영역만 잘라 원본 템플릿 위에 다시 합성한다 (BackgroundPixelLocker: 핵심 제거 영역은 API 결과 100%, 주변 연결부는 20~80% 점진 혼합, 그 외는 원본 100%). 합성 전 구조 보존 점수(line\_continuity 0.30 + texture\_direction 0.25 + local\_color\_match 0.20 + focus\_match 0.15 + artifact\_free 0.10)를 계산해 기준 미달이면 재시도하거나 LaMa 결과로 되돌린다.

### 2.8.5.5 5 풀백 사용 조건과 한계

- 편집 마스크가 전체 이미지의 15% 이하 → API Edit 허용 / 15~25% → 구조 검사 강화 / 25% 초과 → 다른 템플릿 선택
- 결과 재현성이 낮아 후보 2 장 생성 → 평가 → 최고 결과만 cleaned-template 캐시에 저장해 재사용, 동일 템플릿에 API 반복 호출 방지
- 템플릿 저작권 조건은 API Edit 사용 여부와 무관하게 별도 확인 필요 — API Edit 는 저작권 해결책이 아니라 복원 기술일 뿐

## 2.8.6 구도(camera\_angle) 자동 판정 및 매칭

실제 운영 데이터 확인 결과 container\_type은 131개 슬롯 중 7개(5%), camera\_angle은 0개(0%)만 태깅되어 있었다. '제품·슬롯의 camera\_angle을 비교해 각도 차이가 작은 조합을 선택'하는 근본 해결책은 태깅 데이터가 없으면 적용할 수 없어, 태깅 도구 보완과 CLIP 제로샷 자동 분류를 함께 도입했다.

### 2.8.6.1 1 태깅 도구 보완

template\_prep\_and\_tagging.ipynb에 camera\_angle 입력을 추가했다. 자유 텍스트로 받으면 표기가 갈려 매칭에 못 쓰게 되므로 번호 선택 방식(1: eye\_level, 2: top\_down, 3: high\_angle, 4: handheld → 이후 low\_angle로 명칭 정정)으로 강제했다. 다만 이 보완은 새로 태깅하는 템플릿부터만 적용되고 기존 131개 슬롯에는 소급되지 않는다.

### 2.8.6.2 2 CLIP 제로샷으로 제품·배경 양쪽 모두 자동 분류

처음엔 제품 쪽만 CLIP으로 분류하고 배경(슬롯) 쪽은 태깅 메타데이터만 읽도록 구현했는데, 이러면 배경 쪽 값이 항상 빈 문자열이라 비교가 절대 참이 될 수 없어 반쪽짜리 자동화가 되는 문제를 발견했다. 이후 슬롯에 camera\_angle이 없으면 배경 이미지의 슬롯 주변을 크롭해 CLIP으로 추정하도록 수정했다. CLIP은 각도를 물리적으로 측정하는 것이 아니라 사전학습된 이미지-텍스트 유사도로 추정하는 것이므로, 태깅 데이터가 전혀 없는 현재 0보다 나은 근사치를 얻기 위한 임시 대체 수단으로 도입한 것이다.

### 2.8.6.3 3 mismatch 자동 판정과 Compositor 반영

제품·슬롯의 camera\_angle을 '정면(eye\_level-low\_angle)'과 '부감(top\_down-high\_angle)' 두 그룹으로 나눠 그룹이 다르면 mismatch로 판정한다. 슬롯 각도를 전혀 알 수 없는 경우는 안전하게 mismatch=False로 처리해 기존처럼 세로 압축 없이 진행한다.

### 2.8.6.4 4 매칭 단계 엄격 정책

매칭 필터가 태깅 부재로 사실상 비활성 상태였던 한계는, '합성에 어색함이 있어서는 안 되고 구도가 어긋나면 후보정으로도 소용없다'는 판단에 따라 다음과 같이 해결했다: product\_camera\_angle과 정확히 일치하는 슬롯이 후보군에 없으면 다른 각도로 대체하지 않고 (None, None)을 반환해 생성 자체를 중단시킨다. 같은 '부감' 그룹이어도 실제로는 전혀 다른 사진이 되므로 그룹 단위가 아니라 문자열이 완전히 같을 때만 매칭을 허용한다(그룹 기반 판정은 mismatch 사후 보정에서만 사용). 이 정책의 트레이드오프는 명확하다: 태깅 커버리지가 낮은 지금은 생성이 자주 실패할 수 있다 — 이는 의도된 동작이며, 근본 해결책은 여전히 태깅 보완이다.

### 2.8.6.5 5 CLIP 분류 신뢰도(confidence margin) 처리

1등 라벨과 2등 라벨의 유사도 점수 차이(margin)를 신뢰도 지표로 사용한다(low\_confidence\_margin 기본 0.03). camera\_angle은 margin이 임계값 미만이면 예외를 던져 생성을 중단시키는 엄격 정책, container\_type/material\_type은 margin이 낮아도 경고 로그만 남기고 예측값을 그대로 사용하는 관대한 정책으로 나눠 적용한다 — 이 두 필드는 틀려도 파이프라인이 안전하게 폴백하도록 이미 설계돼 있기 때문이다. 임계값(0.03)은 검증된 값이 아니라 임시 기준치다.

## 2.8.7 EdgeHarmonizer — 합성 경계 국소 재생성

LaMa/API Edit(8.5)가 '배경에서 예전 물체를 지우는' 역할이라면, EdgeHarmonizer는 반대로 '제품을 배경에 얹은 뒤 그 경계(alpha 컷아웃 테두리)가 스티커처럼 도드라져 보이는 부분'만 국소적으로 다시 그리는 역할이다. ShadowEngine 다음, smart crop 이전 단계에서 동작한다.

- 마스크 설계: 링(제품 실제 alpha 실루엣 기준 안쪽/바깥쪽 차집합, 로고·텍스트가 있는 제품 중심부는 제외) + 접촉부(제품 하단 일부와 그 아래, 그림자가 배경 질감과 자연스럽게 이어지도록) + LaMa 흔적(TemplateCleaner 제거 마스크 중 제품이 덮지 않은 부분)을 union 으로 결합. 유리 등 투명 재질은 material\_type='glass'일 때 inner\_px 를 1.5 배로 키워 내부 보호 강화
- 안전장치: 편집 마스크가 max\_area\_ratio(기본 20%) 초과 시 LaMa 흔적을 제외하고 재계산, 그래도 초과하면 경계 조화 자체를 스킵. API 편집 후에는 evaluate\_repair\_quality()로 재채점해 min\_quality\_score(기본 0.5) 미만이면 편집 전 합성 결과로 롤백. 편집 영역은 lock\_original\_pixels()로 원본 위에 다시 합성해 마스크 밖 픽셀은 항상 보존
- 프롬프트 설계: '그림자를 맞춰라'는 표현이 ShadowEngine 이 만든 그림자를 새로 그리라는 뜻으로 읽힐 수 있어 '이미 있는 그림자의 톤 전환만 다듬어라'로 수정. 보존 지시와 편집 지시가 충돌하지 않도록 금지 문구(resize/rotate/repaint 금지 등)를 구체적으로 나열

## 2.8.8 제품-배경 기하학적 정합 근사: SurfaceAligner / PitchApproximator / PhysicalSizeApproximator

실측 검증 중 (1) 제품 촬영 각도(roll/pitch)가 배경과 달라도 원본 각도 그대로 얹혀 어색해 보이는 경우, (2) 슬롯 박스 크기만 기준으로 제품 크기를 정해 배경 속 다른 소품과 실물 크기 비율이 안 맞는 경우가 추가로 확인됐다. 둘 다 태깅 없이 지금 있는 데이터로 근사하는 방식을 우선 도입했다.

- SurfaceAligner(roll 보정): TemplateCleaner 의 제거 마스크를 타원으로 근사(cv2.fitEllipse)해 장축 회전각을 구하고 그만큼 제품을 회전. 마스크가 거의 원형이면 회전을 적용하지 않음. SAM2 가 실패해 마스크가 사각형 폴백된 경우 회전 정보 자체가 소실되는 한계가 있음
- PitchApproximator(부감 근사 보정): 배경(제거 마스크)-제품 각각의 단축/장축 비율을 비교해 부감 정도 차이를 근사. 원형 상단(rim)이 뚜렷한 용기에서만 유효, 근사 불가능하면 camera\_angle\_mismatch 기반 고정 압축으로 자동 폴백. 압축 배율은 0.5~1.0 범위로 clip
- PhysicalSizeApproximator(실물 크기 근사 보정): container\_type 별 평균 실물 크기(cm) 참고표로 슬롯-제품의 container\_type 을 비교해 상대 배율을 추정(scale = clip(product\_size/slot\_size, 0.6, 1.6)). 슬롯에 container\_type 이 태깅되어 있어야만 동작하며(태깅 비율 약 5%), 없으면 배율 1.0 으로 안전 폴백

셋 다 '학습 없이 지금 있는 데이터로 즉시 쓸 수 있는 근사치'이지, 정밀한 카메라 파라미터 추정이 아니다. 근본적으로 정확하게 하려면 태깅 시 촬영 각도(도 단위)와 실물 크기(cm)를 직접 입력받는 것이 맞다.

## 2.9 9 단계 — 모델·로직 선택 이유 총정리

전체 선택 기준은 v3 때부터 일관된 세 가지 원칙을 그대로 따른다: (1) 상업 이용이 가능한 라이선스만 사용, (2) 유료 API 호출 없이 자체 호스팅해 운영 비용을 GPU 사용료 수준으로 유지, (3) Colab에서 돌아갈 수 있을 만큼 가벼운 모델을 우선한다. 이 원칙에 따라 GPT-image 계열(조직 인증 필요·종량제 API)과 FLUX.2 무료 버전(비상업 라이선스)은 채택하지 않았다. 다만 SAM2+LaMa로도 해결되지 않는 일부 복잡한 텍스처 복원에 한해, cleaned-template 캐시로 재호출을 최소화하는 조건부 폴백으로만 원칙 (2)의 예외를 둔다. 배경 전체를 API로 재생성하는 방식은 이 예외에 포함되지 않는다.

| 채택 항목                              | 검토했던 대안                               | 채택 이유                                                                                                     |
|------------------------------------|---------------------------------------|-----------------------------------------------------------------------------------------------------------|
| BiRefNet (유지)                      | rembg 기본 모델(u2net)                    | 투명·반사 재질과 미세 경계에서 정확도가 확연히 높고, 교체 비용이 거의 없음                                                               |
| LaMa (유지, 마스크만 정밀화)                | FluxKontextInpaintPipeline            | 텍스트 조건이 불필요한 순수 제거 전용이라 결과가 안정적이고, 오래 검증되어 신생 라이브러리 버그 위험이 낮음                                             |
| SAM2 (신규)                          | 수동 태깅만 계속 사용                          | 정밀 실루엣 마스크를 자동으로 만들어 LaMa 마스크의 블러·잔여물 트레이드오프를 근본적으로 해결. Apache 2.0                                        |
| Depth Anything V2 Small (신규)       | Marigold(확산 기반), MiDaS                | 확산 기반 대비 10배 이상 빠르고 경량이며, Small 모델만 Apache 2.0이라 라이선스 원칙에 정확히 부합                                          |
| IC-Light V1 (신규)                   | GPT-image API, FLUX.2, 전역 Reinhard 유지 | API 비용 없이 자체 호스팅 가능하고 V1 한정 상업 이용 가능, '조명 재계산'이라는 좁은 임무만 수행                                               |
| Container/Material Classifier (신규) | 사용자 수동 입력 유지                          | 장기적으로 태깅·입력 비용을 줄이고 자동 매칭 정확도를 높이기 위함. 데이터가 쌓이기 전까지는 수동 입력과 병행                                            |
| Evaluator/Ranking (신규)             | 사람이 매번 결과 확인 후 선택                     | 다수 후보 생성 후 자동으로 상위 결과만 남겨 운영 효율을 높임                                                                       |
| Local LAB harmonization            | 전역 Reinhard color transfer 유지         | 제품 고유색·채도·대비를 보존하면서 조명만 맞출 수 있어 색감 손상 문제를 해결                                                              |
| Contact/Cast shadow 분리             | 단일 타원 그림자 유지                          | 접촉감(짧고 진한 그림자)과 광원 방향(넓고 흐린 그림자)이라는 서로 다른 역할을 각각 제대로 표현 가능                                                |
| OpenAI Image Edit (조건부, 신규)        | SAM2+LaMa만 계속 사용 / 배경 전체 API 재생성      | SAM2+LaMa로 해결 안 되는 복잡한 텍스처에 한해 국소적으로만 사용(마스크 15% 이하, 원본 픽셀 90%+ 잠금). 배경 전체 재생성은 A/B 실험에서 실사감 점수가 크게 낮아 채택 |

| 채택 항목     | 검토했던 대안                                      | 채택 이유                                                         |
|-----------|----------------------------------------------|---------------------------------------------------------------|
|           |                                              | 하지 않음                                                         |
| CLIP (신규) | 경량 분류 모델 학습(데이터 부족으로 보류) / 태깅 완료까지 수동 입력만 사용 | 태깅 데이터가 5%, 5%, 0%뿐인 현재 상태에서 학습 없이 즉시 자동 분류를 시작할 수 있는 유일한 선택지 |

## 2.10 10 단계 — 데이터 아키텍처 재설계 (메타데이터 스키마)

태깅 도구가 절대경로를 그대로 key로 저장하면 로컬 PC·Colab·Docker·클라우드 환경을 옮길 때마다 매칭이 깨진다. V4에서는 template\_id를 기준으로 하고, 이미지 경로는 상대경로로만 저장하는 구조로 전환한다.

기존 대비 달라지는 점: (1) 경로는 절대경로가 아닌 상대경로만 저장, (2) removal\_mask가 사각형 좌표가 아니라 SAM2로 생성한 정밀 마스크 이미지 파일을 가리킴, (3) extras 배열로 빨대 등 부속 요소를 본체와 분리해서 관리, (4) container\_type·material\_type·camera\_angle·light\_direction 등 v4 매칭·보정에 필요한 필드가 추가됨, (5) cleaning/cleaned\_cache 필드로 배경 복원 방식과 품질 점수를 기록해 동일 템플릿에 대한 API 재호출을 방지, (6) camera\_angle은 태깅값이 있으면 그대로 쓰고 없으면 실행 시점에 CLIP으로 추정하므로 태깅 완료 전까지는 스키마상 정의만 되어 있고 비어 있을 수 있다.

## 2.11 11 단계 — 노트북 구조 재설계

기존 노트북 끝에 TODO나 함수를 하나씩 덧붙이는 방식이 아니라, 실제 실행 가능한 클래스·컴포넌트 구조로 재작성했다. AdCreatorPipeline 아래 BackgroundRemover → TemplateMatcher → TemplateCleaner → SurfaceAligner → Compositor → LightingHarmonizer → ShadowEngine → EdgeHarmonizer → MaterialProcessor → Cropper → Evaluator → Logger → Config 순서로 구성되며, LUT 관련 모듈은 v3에서 완전히 제거됐다. Word 설계 문서도 Executive Summary / Problem Definition / Design Philosophy / v1 Failure Analysis / v2·v3 Architecture / Data Architecture / 단계별 Input·Output·Algorithm·Replaceability / Model Selection / Failure Cases / Evaluation / Optimization / Production Architecture / Future Work / Design Decision Log 순으로 코드와 짝이 맞도록 재구성했다.

## 2.12 12 단계 — 학습(러닝) 우선순위

모든 기능을 한 번에 학습으로 해결하려 하지 않고, 학습이 필요 없는 것부터 단계적으로 진행한다:

- 1) 학습 없이 구현: IC-Light, Depth Anything, BiRefNet, SAM2, 그림자·harmonization 규칙 — 전부 사전학습된 모델과 결정론적 규칙으로 즉시 사용 가능
- 2) 경량 분류 모델 학습: container\_type, material\_type, camera\_angle, light\_direction, surface\_type — 데이터 라벨링 후 가벼운 분류기 학습
- 3) 랭킹 모델 학습: 사람이 선택한 좋음·나쁨 합성 데이터를 이용해 후보 순위화 모델 학습

- 4) 도메인 특화 fine-tuning: 데이터가 충분히 쌓인 이후, relighting-harmonization-shadow-material-specific 모델을 도메인에 맞게 미세조정

## 2.13 13 단계 — 진행 일지 (개선 이력 요약)

### 2.13.1 v3 골격 구축 및 실측 기반 1 차 개선

BiRefNet+SAM2 배경 제거, TemplateCleaner(SAM2 마스크+LaMa+API 폴백), Depth Anything V2 Small, Compositor, IC-Light, ShadowEngine, Cropper, Evaluator 등 v4 핵심 골격을 구현하고, CLIP 제로샷 Classifier와 태깅 도구의 camera\_angle 입력 필드를 도입했다.

실측 중 발견한 문제와 조치: SAM2가 iou\_score만 보고 마스크를 골라 컵 모양이 왜곡되는 문제 → 박스 대비 커버리지 검증(60% 미만이면 사각형 폴백)+팽창(dilate)으로 개선. 일반 타원 그림자에서 뿔 등이 '떠 보이는' 문제 → 제품 alpha 실루엣 하단 띠 기반 접촉 그림자로 교체. 빨대·손잡이 등 부속물을 자동으로 함께 마스크하는 get\_removal\_targets 도입(태깅 박스를 그냥 키우면 Compositor의 배치 크기 계산에 부작용이 있어, '겹치는 슬롯을 찾아 마스크만 합치는' 방식으로 결정). EdgeHarmonizer 신규 도입(링+접촉부+LaMa혼적 마스크, 면적상한+품질롤백 이중 안전장치). camera\_angle 매칭을 '엄격' 정책으로 전환. 'handheld' 명칭이 composition\_id와 혼동을 줘 'low\_angle'로 정정. 노트북 셀 순서 의존성 버그(import 위치, 변수 재정의) 발견·수정.

### 2.13.2 CLIP 신뢰도 문제 해결, 이중 검증 구조 도입, 분류 체계 단순화

transformers 5.x부터 CLIPModel.get\_text/image\_features가 순수 텐서 대신 BaseModelOutputWithPooling을 반환하도록 바뀐 라이브러리 breaking change를 3단계에 걸쳐 수정(1차: projection 재적용 시도 → 차원 불일치 에러, 2차: projection 재적용 없이 pooler\_output 그대로 사용 → 정상 동작, 3차: 원시 코사인 유사도가 좁은 범위에 몰려 margin이 항상 작게 나오는 문제 → logit\_scale 기반 softmax 확률화로 재설계).

confidence margin 정책을 확고하고(camera\_angle 엄격/container\_type-material\_type 관대), 항목별 threshold로 분리했다. 전체 파이프라인을 딸기라떼로 실측하고 외부 피드백을 반영해 다음을 개선했다: EdgeHarmonizer가 실제로는 매번 mimetype 에러로 API 호출에 실패해 원본 합성으로 폴백되고 있었음을 발견(BytesIO에 파일명 미지정이 원인). TemplateCleaner를 'LaMa 0.9 미만일 때만 API 시도'에서 'LaMa 0.9 미만이면 API도 함께 만들어 더 나은 쪽 채택'하는 이중 검증 구조로 전환. SurfaceAligner의 신뢰도 임계값 완화(0.85→0.7). ShadowConfig opacity 20~30% 상향.

material\_type(6갈래)이 실제로는 'glass인지 아닌지'만 쓰이고 있음을 코드 전수 조사로 확인 → is\_transparent(이진)로 단순화. container\_type CLIP 후보를 9개에서 4개(glass/paper\_cup/mug/other)로 축소(음료 위주 서비스 특성 반영). MaterialProcessor 위치를 BackgroundRemover 직후에서 LightingHarmonizer 직후로 재배치(색감 조화 과정에서 재질 효과가 씻겨나갈 위험 방지). PhysicalSizeApproximator에 종횡비 보정(estimate\_aspect\_ratio\_scale) 추가.

## 2.14 14 단계 — v4\_final 고도화 작업

V4\_final(범용 자동 합성) 파이프라인을 계속 사용하는 것을 전제로 실측 기반 디버깅을 진행한 세션이다. 이번 세션에서 발견한 버그와 수정 내역, 그리고 코드로 더 이상 해결되지 않는 구조적 한계를 정리한다.

### 2.14.1 처리한 항목 요약

- 구도 매칭 완화(B 안): camera\_angle 이 정확히 일치하지 않아도 인접 각도 그룹(front: eye\_level/low\_angle, overhead: top\_down/high\_angle) 안에서는 대체 매칭을 허용하도록 정책 완화. 완전히 다른 그룹끼리는 여전히 매칭 안 됨
- IC-Light 비활성화 확정: 실제 GPU 에서 재검증한 결과, conv\_in 만 8 채널로 패치하고 forward hook(공식 구현의 hooked\_unet\_forward)이 빠져 있어 구조적으로 한 번도 성공한 적이 없었음을 확인. 효과도 미미하고 인물 조명 재현용 모델이라 도메인도 안 맞아, use\_ic\_light 기본값을 False 로 전환하고 Local LAB 만으로 조명 조화를 담당하도록 확정
- EdgeHarmonizer 아키텍처 교체(생성형 → 결정론적): 생성형 API 기반 국소 재생성이 반복 실측 결과 '마스크 바깥 픽셀 보존을 보장 못 하는 생성 모델의 근본 한계'로 제품이 두 겹으로 겹쳐 보이는 고스팅을 반복 재현함(마스크 폭·사이즈·종횡비 조정으로도 해결 안 됨). OpenCV seamlessClone(Poisson 블렌딩) 기반 결정론적 경계 블렌딩으로 기본 경로를 교체. 생성형 경로는 옵션으로 남겨둠. (seamlessClone 자체도 center 파라미터 계산 버그가 2 차례 있었음 — 무게중심 기준에서 바운딩박스 중심 기준으로 수정)
- openai\_local\_edit() 종횡비(aspect ratio) 처리 버그 수정: 템플릿 실제 종횡비와 무관하게 size="1024x1024"를 고정 사용해 API 가 원본을 크롭/재구성하는 문제 발견(OpenAI 개발자 포럼에도 동일 이슈 보고됨). 입력 종횡비에 맞는 프리셋을 고르고 결과를 비율 유지 확대+중앙 crop 으로 되돌리도록 수정. 다만 이 수정만으로는 EdgeHarmonizer 의 고스팅이 해결되지 않아, API 자체의 신뢰성 한계가 더 근본 원인으로 판단됨
- PitchApproximator 를 균일 세로 압축에서 사다리꼴 원근 왜곡으로 확장: 세로 길이만 균일하게 줄여서는 위에서 내려다볼 때 물체 위쪽이 아래쪽보다 좁아 보이는 원근 수렴 효과를 표현하지 못하던 한계를 보완, squash\_factor 로부터 근사한 수렴 강도만큼 4 점 원근 변환을 추가 적용(단, '위아래 수렴'만 반영, 좌우 비대칭/키스톤 방향은 미반영)
- PitchApproximator 배경/제품 측정 기준 불일치 버그 수정: 배경은 제거된 물체의 전체 실루엣에 타원을 맞추고, 제품은 top\_band\_ratio 로 윗부분만 보고 있어 서로 다른 기준을 비교하던 문제 — squash factor 가 실제 체감 차이보다 항상 미약하게 나오던 원인(실측: 기준 통일 전 0.897 → 통일 후 0.656 로 대폭 강화). 양쪽에 동일한 top\_band\_ratio 를 적용하도록 통일하고, 세로로 긴 용기는 밴드를 자동으로 좁히는 적응형 로직 추가
- 템플릿 재고 확인 스크립트 도입: 스타일×구도×카테고리×각도 조합별 슬롯 개수를 확인한 결과, product\_large/product\_center 구도에는 스타일 불문 front 그룹(eye\_level/low\_angle) 템플릿이 0 개, 전체 85 슬롯 중 overhead 계열이 84%로 편중, container\_type 태깅도 36%만 채워져 있음을 확인
- 노트북 부분 실행으로 인한 재현 문제 다수 발생 및 최종 정리본 재구성: 함수 수정이 파일에는 반영됐지만 커널에 재실행이 안 돼 옛날 동작이 재현된 경우, 편집 스크립트가 중간에 실패해

특정 함수 정의가 통째로 유실된 경우 등 같은 유형의 원인 규명에 여러 차례 시간이 소요됨.  
 ad\_creator\_pipeline\_v4\_final.ipynb 를 최종 정리본으로 재구성(첫 셀에 파이프라인  
 개요·유의사항, 마지막 셀에 작업 이력·냉철한 평가·남은 과제)

- 문서 정리: 설계 문서에서 검토만 하고 채택하지 않은 Stable Pipeline 전환 관련 내용을 제거하고, v4\_final 을 단일 진행 라인으로 정리

## 2.14.2 구조적 한계로 확인된 것 — 2D 기하 보정의 한계

PitchApproximator의 측정 기준을 바로잡고 사다리꼴 원근 왜곡까지 추가한 뒤에도, 원본 템플릿이 제품 사진보다 훨씬 가파른 부감으로 촬영된 조합에서는 시각적으로 구도가 맞지 않는 현상이 남았다. 원인은 얇은 각도로 찍힌 원본 사진에는 '더 위에서 봤을 때 보이는 정보(음료 표면 안쪽 등)'가 애초에 존재하지 않기 때문이다 — 2D 워핑은 있는 정보를 변형할 뿐, 없는 정보를 만들어낼 수는 없다. 압축 배율을 더 세게 주는 것도 해결책이 아니다 — '구도가 안 맞음' 대신 '물체가 부자연스럽게 눌러 보임'이라는 다른 종류의 어색함이 생길 뿐이다.

근본적으로 해결하려면 코드가 아니라 다음 중 하나가 필요하다: (a) 템플릿 촬영각도와 더 가까운 제품 사진 확보, (b) 템플릿 재고를 실제 pitch 값 기준으로 더 정밀하게 매칭(현재는 4단계 각도 버킷 매칭만 함), (c) 생성형 보정(다만 이는 EdgeHarmonizer에서 이미 겪은 것과 같은 신뢰성 리스크를 다시 끌어오는 선택이라 비권장).

## 2.14.3 다음 우선순위 제안

- TemplateCleaner 의 API 폴백 경로(openai\_local\_edit 공유)도 EdgeHarmonizer 와 동일한 신뢰성 리스크가 있는지 재검토
- product\_large/product\_center 구도에 front 그룹(eye\_level/low\_angle) 템플릿 추가 촬영·태깅
- container\_type 태깅 커버리지 확대(현재 36%) — PhysicalSizeApproximator 정확도에 직결
- EdgeHarmonizer 가 결정론적 블렌딩 게이트를 통과할 만큼 편집 전 단계(마스크·프롬프트) 품질을 높이는 방향 검토

## 2.15 종합 타임라인

| 단계             | 시기/구분    | 핵심 사건                                                                                                                                                     |
|----------------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| SD1.5          | 초기       | rembg+LoRA+GPT-4o-mini caption. matcha(유리컵) 조합 컵 왜곡 → 구조적 한계 판단                                                                                           |
| SDXL           | SD1.5 이후 | 1024px+fp16-fix VAE로 교체. VAE 오버플로우/마스크 임계값/strength/CLIP 토큰/이중그림자/일러스트풍 버그 수정. 04·05 버전 프롬프트 반복 끝에 소품 문제는 해결, 그레이 고착·제품별 편차 문제는 프롬프트만으로 해결 불가 → 구조적 한계 도달 |
| FLUX.1 Kontext | SDXL 이후  | 단일 모델로 전 과정 처리 시도 → 멀티 이미지 입력 불가, 스티칭                                                                                                                     |

| 단계          | 시기/구분      | 핵심 사건                                                                                                                            |
|-------------|------------|----------------------------------------------------------------------------------------------------------------------------------|
| (v2)        |            | 실패, 인페인팅 강제크롭 버그, 참조 이미지 삽입 용도로 미학습이라는 근본 한계 확인 → 실패                                                                             |
| V3          | FLUX 이후    | BiRefNet+LaMa+PIL+색보정+그림자+크롭의 6단계 전문 모델 조합. 경로/매칭 버그 다수 해결. MVP 65~75점 수준 확인, '붙어있는 느낌'·색감 손상·마스크 트레이드오프 한계 확인                   |
| V4 설계       | V3 이후      | 6대 병목(조명/접촉/깊이/재질/카메라/템플릿) 정의, SAM2·Depth Anything·IC-Light·CLIP Classifier·Evaluator·EdgeHarmonizer·SurfaceAligner 등 신규 컴포넌트 설계 |
| V4 1차 구현    | 2026-07-15 | 핵심 골격 구현, SAM2 커버리지 검증, 접촉그림자 교체, EdgeHarmonizer 도입, camera_angle 엄격 매칭 도입                                                       |
| V4 개선       | 2026-07-16 | CLIP transformers 5.x 호환성 수정, confidence margin 정책, TemplateCleaner 이중 검증, material_type→is_transparent 단순화                      |
| V4_final 세션 | 2026-07-20 | 구도 매칭 완화, IC-Light 비활성화 확정, EdgeHarmonizer를 결정론적 seamlessClone으로 교체, PitchApproximator 개선, 2D 기하보정의 구조적 한계 확인, 템플릿 재고 공백 확인      |

개인 작업 보고서

# Pickmood

## 3. 오픈소스 모델링 2 · FLUX.1 Kontext 기반

### 모델 파이프라인

작성자 김기현

## 개요

사용자가 올린 음료·제품 사진을 인스타 감성의 제품 사진으로 자동으로 바꿔주는 모델 파이프라인을 만들고 개선한 과정을 정리한 것이다.

버전 1 은 파이프라인을 처음 만든 단계다. 제품을 올려서 배경에 붙이는 합성 방식을 쓰고, 여기에 약한 재생성을 더한 하이브리드 방식까지 만들어서 일단 쓸 만한 결과물을 얻기까지의 기록이다.

버전 2 는 이 파이프라인을 더 자연스럽게 범용적으로 바꿔보려던 단계다. 전체를 다시 그리는 재생성 방식으로 방향을 틀고, 참조 이미지 분석, 제품 보존, 방향 보정, 외부 API 연동 등을 시도했지만 결국 원하는 품질까지는 가지 못했다.

프로젝트의 공통 목표는 아래 두 가지를 조합한 12 가지 스타일 결과를 만드는 것이었다.

- 구도: closeup, medium, aerial, handheld

배경 스타일: white, wood, vivid

작업 환경은 GCP VM(NVIDIA L4, VRAM 약 24GB, 시스템 RAM 15GB)이고, 모델은 FLUX.1 Kontext-dev 를 NF4 4 비트 양자화와 CPU 오프로드로 돌렸다.

## 버전 1 — 합성 기반 파이프라인 구축

### 3.1 전체 재생성 방식 검토와 로고 훼손

#### 3.1.1 실험 목적

가장 먼저 시도한 건 이미지 편집 모델인 FLUX Kontext 에 제품 사진과 프롬프트를 같이 넣어서 배경과 구도를 통째로 바꾸는 방식이었다. 제품과 배경을 한 화면 안에서 같이 다시 그리면 자연스럽게 나올 거라고 봤다.

#### 3.1.2 실패 현상

배경과 조명은 그럴듯하게 바뀌었는데, 제품의 로고와 글자가 원본과 다르게 나왔다. 대표적으로 "Compose Coffee" 로고가 "COSCO WAYC" 같은 알아볼 수 없는 가짜 글자로 바뀌었다. 브랜드를 알아보게 해주는 로고와 글자가 망가지니 실제로 쓸 수가 없었다.

#### 3.1.3 원인 분석

이런 생성 모델은 이미지를 노이즈에서부터 다시 그려내는 식으로 만든다. 이 과정에서 로고나 작은 글씨처럼 세밀한 부분은 원본 그대로 살리지 못한다. 모델이 "둥글고 검은 로고가 있다" 정도는 기억하지만, 그 안의 글자는 매번 새로 지어낸다. 프롬프트로 "로고를 그대로 뒀라"고 아무리 강하게 말해도 소용이 없었다.

#### 3.1.4 코드 수정 방향

로고가 망가지는 건 재생성 방식으로는 어쩔 수 없는 한계라고 봤다. 그래서 제품을 다시 그리지 않고, 원본 픽셀을 그대로 가져다 쓰는 합성 방식으로 방향을 바꿨다. 이 결정이 버전 1 전체의 뼈대가 됐다.

### 3.2 합성 방식과 이미지 두 장 입력 문제

#### 3.2.1 실험 목적

제품의 원본 픽셀을 그대로 살리기 위해, 제품을 배경에서 오려낸 다음 템플릿 배경에 얹는 합성 방식을 만들었다.

### 3.2.2 초기 구현과 실패

처음에는 제품 이미지와 배경 이미지를 각각 모델에 넘겨서 역할을 나눠주려고 했다(image=[제품, 배경]). 그런데 우리가 쓰는 diffusers 의 FluxKontextPipeline 은 이미지를 리스트로 두 장 받는 걸 지원하지 않았다. 실제로 돌려보니 이런 오류가 났다.

```
RuntimeError: shape '[1, 16, 64, 2, 64, 2]' is invalid for input of size 524288
```

그래서 다음으로 제품과 배경을 좌우로 이어 붙여서 한 장으로 넣어봤는데(stitch 방식), 이번엔 모델이 어느 쪽이 제품이고 어느 쪽이 배경인지 구분을 못 하고 제품을 새로 그려버렸다. 방식 자체가 역할을 헷갈리게 만드는 문제였다.

### 3.2.3 코드 수정 방향

모델한테 두 이미지를 합치게 하는 걸 포기하고, rembg 로 배경을 지운 다음 PIL 로 직접 붙이는 방식으로 바꿨다. 합성의 핵심 코드는 아래와 같다.

```
from rembg import remove, new_session
_session = new_session()

def cutout(image):
    # 배경 제거 -> RGBA (제품 픽셀 그대로 보존)
    return remove(image.convert("RGB"), session=_session).convert("RGBA")

def compose(product_image, template_image, scale=1.0):
    template = template_image.convert("RGBA")
    product = cutout(product_image)
    # 제품 실루엣 bbox 를 구해 템플릿의 컷 위치에 배치
    ...
    template.alpha_composite(product_scaled, (paste_x, paste_y))
    return template.convert("RGB")
```

### 3.2.4 결과

다시 그리는 과정이 없으니 제품 픽셀이 그대로 남아서 로고가 망가지는 문제가 사라졌다. 속도도 빨라졌다. 다만 이때부터 합성 티가 나는 것과 위치를 잡는 것이 새로운 숙제로 남았다.

## 3.3 복잡한 템플릿에서 컵 위치 찾기

### 3.3.1 실험 목적

합성을 하려면 템플릿에서 원래 컵이 어디 있는지 찾아서, 그 자리에 사용자 제품을 놓아야 했다.

### 3.3.2 실패 현상

처음에는 rembg 가 잡아주는 "가장 큰 덩어리"를 컵이라고 봤다. 그런데 컵이 여러 개거나 접시와 케이크가 같이 있는 복잡한 템플릿에서는 엉뚱한 걸 잡았다. 실제로 접시 위 케이크 조각을 컵 자리로 잡아서, 케이크 위치에 음료를 붙여버리는 일이 있었다.

### 3.3.3 코드 수정 방향

"가장 큰 덩어리"라는 애매한 기준 대신, 글자로 무엇을 찾을지 지정할 수 있는 Grounding DINO(grounding-dino-tiny)를 붙였다.

```
from transformers import AutoProcessor, AutoModelForZeroShotObjectDetection

_MODEL_ID = "IDEA-Research/grounding-dino-tiny"

def detect_cups(image, text_prompt="a cup. a drink. a glass. a beverage."):
    inputs = processor(images=image, text=text_prompt, return_tensors="pt")
    outputs = model(inputs)
    # 신버전 인자(threshold) 우선, 실패 시 구버전(box_threshold) 폴백
    try:
        results = processor.post_process_grounded_object_detection(
            outputs, inputs["input_ids"], threshold=0.3,
            text_threshold=0.25, target_sizes=[image.size::-1])[0]
    except TypeError:
        results = processor.post_process_grounded_object_detection(
            outputs, inputs["input_ids"], box_threshold=0.3,
            text_threshold=0.25, target_sizes=[image.size::-1])[0]
    return results
```

붙이는 과정에서 transformers 버전이 올라가면서 함수 인자 이름이 box\_threshold 에서 threshold 로 바뀐 문제가 있었다. 그래서 위처럼 새 이름을 먼저 써보고, 안 되면 옛 이름으로 넘어가게 처리했다.

### 3.3.4 결과

복잡한 장면에서도 "cup"이라는 글자로 컵만 정확히 잡게 됐다. 컵이 여러 개면 그중 가장 큰 걸 골랐다.

## 3.4 원래 컵 지우기와 캐싱

### 3.4.1 실험 목적

사용자 제품을 엮기 전에 템플릿에 원래 있던 컵을 지워야 했다. 안 지우면 원래 컵이 새 제품 뒤로 빠져나와서 지저분하게 남았다.

### 3.4.2 코드 수정 방향

컵을 지우는 전용 모델(FLUX Fill, 15GB)을 따로 받는 대신, 디스크가 부족한 상황이라 이미 올려둔 Kontext 한테 컵을 지우라고 시켰다. 그리고 템플릿은 바뀌지 않으니, 한 번 지운 결과는 저장해두고 다시 썼다.

```
ERASE_PROMPT = (
    "Remove the drink cup and any glass, straw or beverage from the table, "
    "leaving a clean empty surface. Keep the exact same background, table, "
    "lighting, shadows and composition unchanged. Fill the spot where the cup "
    "was with the natural continuation of the table surface. Photorealistic."
)

def erase_product(pipe, template_image, template_path, cache_dir, ...):
    cache_file = _cache_path(template_path, cache_dir)
    if os.path.exists(cache_file):
        # 같은 템플릿은 재사용
        return Image.open(cache_file).convert("RGB")
    empty = pipe(image=template_image, prompt=ERASE_PROMPT, ...).images[0]
    empty.save(cache_file)
    return empty
```

### 3.4.3 결과

모델을 더 받지 않고도 원래 컵을 지웠고, 저장해둔 걸 재사용하니 같은 템플릿을 매번 다시 처리할 필요가 없어졌다. 다만 지우라는 지시가 약하면 컵을 지우는 대신 다른 물건으로 바꿔버려서, "빈 바닥만 남기고 아무것도 새로 만들지 마라"고 세계 못 박아야 했다.

## 3.5 하이브리드 — 합성한 뒤 약하게 다시 그리기

### 3.5.1 실험 목적

합성으로 로고는 지켰지만, 오려 붙인 제품의 조명이 배경과 안 맞아서 오려 붙인 티가 났다. 이걸 줄이려고, 합성한 이미지를 Kontext 로 아주 약하게만 다시 그려서 조명과 경계만 배경에 녹이는 하이브리드 방식을 넣었다.

### 3.5.2 초기 구현

Kontext 에는 img2img 처럼 재생성 정도를 조절하는 strength 값이 없다. 그래서 두 가지로 세기를 조절했다. 하나는 스텝 수를 낮추는 것, 다른 하나는 다시 그린 결과를 원본과 섞는 비율을 조절하는 것이다.

```
def harmonize(pipe, composite_image, product_box, product_silhouette,
              blend_strength=0.35, product_protect=0.85,
              num_inference_steps=12, ...):
    regen = pipe(image=work, prompt=HARMONIZE_PROMPT,
                 num_inference_steps=num_inference_steps, ...).images[0]
    # 전역 블렌딩: 재생성 결과를 blend_strength 만큼 반영
    blended = orig * (1 - blend_strength) + regen * blend_strength
    # 제품 영역은 원본을 되살려 로고 보호
    mask = _silhouette_mask(size, product_box, product_silhouette)
    blended = blended * (1 - mask * product_protect) + orig * (mask * product_protect)
    return blended
```

### 3.5.3 실패 현상과 해결 (줄줄이 나온 버그들)

하이브리드를 넣으니 자잘한 버그가 여러 개 나왔고, 하나씩 코드로 잡았다.

(1) 회색 사각형 자국. 제품을 지키는 마스크를 사각형으로 만들었더니, 컵 옆 빈 공간까지 원본이 되살아나서 회색 사각형이 남았다. 마스크를 제품의 실제 모양(rembg 알파)대로 바꾸고, 가장자리를 살짝 안쪽으로 줄여서 경계는 다시 그리는 쪽이 자연스럽게 처리하도록 했다.

```
def _silhouette_mask(size, box, silhouette, feather=20):
    m = Image.new("L", size, 0)
    a = silhouette.split()[-1].resize((box_w, box_h))
    m.paste(a, (box_left, box_top))
    m = m.filter(ImageFilter.MinFilter(5)) # 살짝 수축
    return m.filter(ImageFilter.GaussianBlur(feather // 3))
```

(2) 없던 로고가 생김. 프롬프트에 "로고를 그대로 뒤라"라고 넣었더니, 로고가 없는 유리컵에 없던 로고를 그려 넣었다. 프롬프트에서 로고를 언급한 것 자체가 "로고를 그려라"는 신호가 된 셈이다. 그래서 제품에 로고가 있는지(has\_logo)에 따라 문구를 나눴다.

```
def build_harmonize_prompt(has_logo=False):
    if has_logo:
        return BASE + " Keep the product's existing label, logo and text exactly."
    return BASE + (" The product surface is plain with no logo, no label, no text. "
                  "Do not add any logo, text or marking to the product.")
```

(3) 제품이 하나 더 생김. 다시 그리는 세기를 높이면, 모델이 발밑 그림자를 보고 제품을 하나 더 그렸다. 프롬프트에 제품은 딱 하나여야 하고 그림자 안에는 아무것도 없어야 한다고 못 박아서 해결했다("exactly ONE product", "the shadow must stay a plain soft shadow with no object inside it").

### 3.5.4 결과

로고는 지키면서 조명과 경계도 어느 정도 녹인 결과를 얻었다. 이 하이브리드가 버전 1의 기본 결과물이 됐다.

## 3.6 그림자와 오려내기 다듬기

### 3.6.1 실패 현상과 해결

합성 품질을 더 올리려고 그림자와 오려내기의 자잘한 문제를 차례로 잡았다.

(1) 오려낼 때 그림자가 따라옴. rembg가 원본 그림자를 반투명하게 남겨서 회색 얼룩이 생겼다. 투명도가 일정 기준보다 낮은 부분은 아예 잘라냈다. 다만 유리컵처럼 원래 반투명한 제품은 기준을 너무 높이면 몸통까지 사라지니, 낮은 값부터 조금씩 올리며 맞췄다.

```
def cutout(image, alpha_threshold=140):
    rgba = remove(image, session=_session).convert("RGBA")
    arr = np.array(rgba)
    a = arr[..., 3].astype(np.float32)
    a[a < alpha_threshold] = 0 # 반투명 그림자 제거
    a[a >= alpha_threshold] = 255
    arr[..., 3] = a.astype(np.uint8)
    return Image.fromarray(arr, "RGBA")
```

(2) 그림자가 사각형으로 나옴. 타원 그림자를 그리는 판을 너무 작게 잡아서, 흐림 효과가 퍼질 공간이 없어 타원이 잘려 사각형처럼 보였다. 흐림이 퍼질 만큼 판을 넉넉하게 잡아서 해결했다.

```
def _directional_shadow(size, blur, opacity=110, skew=0.35):
    w, h = size
    ell_w, ell_h = int(w * 1.05), max(14, int(h * 0.10))
    margin = blur * 3 + 4 # 블러가 퍼질 여유
    cw, ch = ell_w + abs(int(w*skew)) + margin*2, ell_h + margin*2
    s = Image.new("RGBA", (cw, ch), (0,0,0,0))
    ...
    return s.filter(ImageFilter.GaussianBlur(blur))
```

(3) 그림자 방향이 구도랑 안 맞음. 위에서 내려다보는 항공샷인데도 그림자가 옆으로 누웠다. 항공샷은 옆으로 안 눕게 하고, 나머지는 배경의 좌우 밝기를 비교해 빛이 어디서 오는지 짐작한 뒤 그 반대쪽으로 눕혔다.

```
def estimate_light_dir(image):
    g = np.asarray(image.convert("L"), np.float32)
    lower = g[g.shape[0]//2:, :] # 아래쪽(테이블 면)
    left, right = lower[:, :w//2].mean(), lower[:, w//2:].mean()
    return (right - left) / (left + right + 1e-6) * 2.0

# 항공샷은 skew=0, 그 외는 빛 반대 방향
skew = 0.0 if composition == "aerial_shot" else -light * abs(base_skew) * 1.5
```

## 3.6.2 결과

합성 결과가 배경에 좀 더 잘 어울리게 됐다. 그래도 오려 붙인 티가 완전히 없어지지는 않았다. 이 한계 때문에 버전 2 로 넘어가게 됐다.

## 3.7 속도 개선 — 미리 준비하기와 실시간 처리 나누기

### 3.7.1 실험 목적

처음 파이프라인은 한 장에 3 분쯤 걸렸다(컵 지우기 2 분, 다시 그리기 50 초 등). 실시간으로 쓰기엔 너무 느렸다.

### 3.7.2 코드 수정 방향

템플릿은 어차피 정해져 있다는 점을 이용했다. 오래 걸리는 컵 지우기는 서비스 전에 미리 해두고, 그 결과(빈 배경과 위치 정보)를 저장해뒀다. 실시간에는 미리 만들어둔 배경을 불러와서 제품만 붙이면 된다.

```
# 오프라인: 컵 지운 빈 배경과 배치 정보를 placements.json 에 저장
place = {"box": [l, t, r, b], "bottom_y": b,
         "background": bg_path, "scene": scene_stats(bg)}
```

```

# 런타임(fast): 준비된 배경 로드 -> 제품 합성 -> 그림자. 모델 미사용
def run_fast_swap(product_path, style, comp, quality="fast", ...):
    picked = random.choice(list_prepared(style, comp))
    background = Image.open(picked["background"]).convert("RGBA")
    # ... 제품 합성 ...
    if quality == "high": # 선택적 재생성
        result = harmonize(pipe, result, ...)

```

그리고 시스템 RAM(15GB)이 모자라서 메모리가 넘치면 SSH 가 끊기는 일이 있었다. 그래서 스왑을 8GB 잡아두고, 오래 걸리는 작업은 nohup 으로 백그라운드에서 돌렸다.

### 3.7.3 결과

빠른 모드에서는 4 초 정도가 나왔다. 12 조합을 빠르게 훑어보고, 마음에 드는 것만 고품질(다시 그리기 포함, 30 초쯤)로 다시 뽑는 2 단계 방식으로 설계했다.

## 3.8 버전 1 정리

버전 1 은 제품을 오려 붙이고 약하게 다시 그려서 녹이는 하이브리드 방식이었다. 로고를 지켜야 한다는 가장 중요한 조건을 만족했고, 속도도 빠른 모드 4 초로 쓸 만한 수준까지 왔다. 다만 합성 방식의 한계는 끝까지 남았다. 오려 붙인 제품의 조명, 원근, 질감이 배경과 딱 맞지는 않아서 오려 붙인 티가 났다. 특히 배경이 복잡하거나 우드 톤처럼 조명이 까다로운 템플릿에서 이질감이 더 눈에 띄었다. 이 한계를 넘어보려고 다시 전체 재생성 쪽으로 방향을 틀어 개선을 시도한 게 버전 2 다.

## 버전 2 — 전체 재생성으로 개선 시도

### 3.9 FLUX Kontext 로 전체 다시 그리기

#### 3.9.1 실험 목적

버전 1의 합성 방식은 제품과 배경이 따로 노는 게 문제였다. 그래서 모델이 제품과 배경을 한 화면에서 같이 다시 그리면 더 자연스러울 거라고 다시 판단했다. `generate_beverage_image()`를 중심으로 입력 이미지와 프롬프트를 넣어 전체를 다시 그렸다.

#### 3.9.2 초기 구현

```
result = generate_beverage_image(
    input_image_path,
    composition=composition,
    background_style=background_style,
    strength=strength,
    seed=seed,
)
```

이 함수는 `app/inference.py`에서 FLUX Kontext를 부르고, `app/prompt_builder.py`가 만든 프롬프트를 넘겨줬다.

#### 3.9.3 실패 현상

처음 테스트한 망고 스무디는 그럴듯했는데, 다른 제품을 넣으면 문제가 생겼다.

넣은 제품과 나온 제품의 모양이 달라졌다.

빈 컵을 넣어도 음료가 든 컵으로 나왔다.

손잡이 컵을 넣어도 손잡이가 사라지거나 다른 컵으로 바뀌었다.

제품을 지키기보다 새 제품을 만들어내는 경향이 강했다.

#### 3.9.4 원인 분석

초기 프롬프트가 "예쁜 인스타 제품 사진을 만들어라"에 초점이 맞춰져 있었다. 그래서 모델이 입력 이미지를 그대로 베껴야 할 기준이 아니라, 그냥 참고할 영감 정도로 받아들였다. 결국 제품을 지키기보다

스타일과 배경을 예쁘게 만드는 쪽으로 세게 작동한 것이다. (버전 1 처음에 겪은 로고 훼손과 뿌리가 같은 문제다.)

## 3.10 12 개 조합 한 번에 만들기

### 3.10.1 실험 목적

실제 서비스를 위해 4 구도 × 3 배경 = 12 개를 한꺼번에 만드는 실험을 `scripts/generate_matrix.py` 로 돌렸다.

### 3.10.2 실패 현상

`closeup` 과 `medium` 의 제품 크기 차이가 거의 없었다.

`aerial` 이 진짜 위에서 본 게 아니라 살짝 높은 데서 찍은 정면샷처럼 나왔다.

`handheld` 도 손이 어색하거나 제품 옆에 손 일부만 붙어 있었다.

### 3.10.3 원인 분석

프롬프트에서 배경 스타일 설명이 구도 설명보다 세게 먹혔다. 모델이 "어떤 분위기냐"는 잘 따랐는데 "어떤 각도냐"는 약하게 따랐다.

### 3.10.4 코드 수정 방향

`presets/category_prompts.py` 와 `presets/compositions.py` 에서 구도 지시를 더 세게 줬다. 항공샷에는 이런 내용을 넣었다.

```
Strict          top-down          flat          lay          composition.
Camera          is          directly          above          the          product.
90-degree          overhead          view.
Do not use front-facing, eye-level, three-quarter, portrait, macro, or close-up camera angle.
```

미디어샷에는 제품이 너무 크게 나오지 않도록 이렇게 넣었다.

```
The          product          should          appear          smaller          in          frame.
The          camera          is          one          step          farther          away.
Show more surrounding environment and background context.
```

### 3.10.5 결과

항공샷 각도는 전보다 나아졌다. 그래도 제품 자체가 바뀌는 문제는 그대로였다.

## 3.11 참조 이미지로 스타일 뽑아내기

### 3.11.1 실험 목적

내가 직접 쓴 프롬프트만으로는 인스타 감성의 조명, 색감, 소품 배치를 다 담기가 어려웠다. 그래서 스타일별로 참조 이미지를 모으고, OpenAI API 로 그 공통점을 뽑아내는 방식을 시도했다.

참조 이미지는 이렇게 정리했다.

```
templates_lib/  
  neutral_white_minimal/{aerial_shot,    handheld_lifestyle,    product_center,    product_large}/  
  vivid_color/{...}/  
  wood/{...}/
```

### 3.11.2 코드 수정 방향

analyze\_templates\_openai.py 로 각 폴더의 여러 이미지를 분석해서 공통점만 뽑아 presets/style\_sheets.json 에 저장했다. 분석 항목은 카메라 각도, 조명 방향, 배경 재질, 색감, 소품 배치, 제품과 배경의 거리감, 인스타 감성 요소였다. (한 장만 보면 그 사진의 소품에만 맞춰지니, 여러 장의 공통점만 뽑았다.)

### 3.11.3 실패 현상

참조 이미지에 빨대, 음료, 커피콩, 시럽, 크림 같은 게 들어 있으면, 모델이 그걸 스타일이 아니라 "만들어야 할 것"으로 이해했다. 그래서 빈 컵을 넣어도 망고색 음료, 시럽, 휘핑크림, 빨대, 플라스틱 컵이 계속 딸려 나왔다.

### 3.11.4 원인 분석

style\_sheets.json 안에 "clear plastic cup", "iced liquid", "black straw", "beverage", "drink" 같은 표현이 남아 있었다. 모델이 입력 제품보다 이 설명을 더 세게 따른 것이다.

## 3.12 product-lock — rembg 로 제품 지키기

### 3.12.1 실험 목적

Kontext 가 제품을 너무 바꾸니까, 제품만 따로 떼어내서 새 배경 위에 다시 붙이는 product-lock 방식을 시도했다. (버전 1 의 합성 아이디어를 재생성 파이프라인 안에 다시 넣은 셈이다.)

### 3.12.2 초기 구현

app/preserve\_mode.py 에서 rembg 로 제품을 떼어내되, OpenAI Vision 으로 제품 위치를 먼저 찾은 다음 그 부분에만 rembg 를 쓰도록 했다.

```
from rembg import remove

bbox = detect_product_bbox_with_openai(image)
crop = image.crop(bbox)
cutout = remove(crop)
canvas.alpha_composite(cutout, (x, y))
```

### 3.12.3 실패 현상

제품을 떼어내다 실패하면 원본 배경 일부가 같이 떨어져 나왔다. 특히 입력이 지저분하거나 제품이 나무 바닥이랑 색이 비슷하면, 컵과 함께 책상 무늬가 잘려 나오거나, 원본 배경 조각이 새 배경에 붙거나, 제품이 반달 모양으로 잘려서 오히려 티가 더 났다.

### 3.12.4 원인 분석

프롬프트 문제가 아니라 제품을 떼어내는 게 문제였다. 제품을 깔끔하게 못 떼어내면 배경이 아무리 좋아도 결과가 망가졌다.

### 3.12.5 코드 수정 방향

app/inference.py 에서 preserve\_mode.run\_preserve() 호출을 빼고 product-lock 경로를 껐다. 다시 FLUX Kontext 에 입력 이미지를 그대로 넘기는 방식으로 되돌렸다.

```
result = pipe(
    image=input_image,
    prompt=prompt,
    guidance_scale=guidance,
    num_inference_steps=steps,
```

```
generator=generator,  
) .images[0]
```

### 3.12.6 결과

원본 조각이 붙는 문제는 없어졌지만, 제품이 바뀌는 문제는 다시 남았다. (버전 1 에서 봤던 제품 떼어내기의 약점이 버전 2 에서도 똑같이 나타난 셈이다.)

## 3.13 다시 그리는 세기 조절

### 3.13.1 실험 목적

제품이 바뀌는 걸 줄이려고 strength=low 로 다시 그리는 세기를 낮춰봤다.

### 3.13.2 초기 구현

strength 를 숫자로 바꿔서 image\_guidance\_scale 같은 값에 넘기려 했다.

```
fallback_table = {"low": 0.25, "medium": 0.35, "high": 0.5}
```

### 3.13.3 실패 현상

strength=low 를 넣으니 오히려 입력 제품을 덜 반영하고, 모델이 배경 스타일을 더 세게 따라가며 새 제품을 만들었다.

### 3.13.4 원인 분석

이 값이 "얼마나 바꿀지"가 아니라 "입력 이미지를 얼마나 참고할지"로 작동하는 것 같았다. 값이 낮으면 제품을 더 지키는 게 아니라, 입력을 덜 보게 만들었을 가능성이 크다.

### 3.13.5 코드 수정 방향

strength 를 모델 값으로 직접 넘기지 않고, 프롬프트를 얼마나 보수적으로 쓸지로만 조절하게 바꿨다.

```
(strength=low)  
Very conservative edit.  
Keep the product almost unchanged.  
Change mainly the background, lighting, and framing.
```

### 3.13.6 결과

모델 값을 억지로 건드리기보다, 제품을 지키라는 프롬프트를 확실히 주는 쪽이 더 안정적이었다.

## 3.14 하드코딩 걷어내고 config 정리

### 3.14.1 실험 목적

급하게 문제를 막다 보니 임시 코드가 쌓였고, 일부 설정값이 실제 config.py 와 안 맞아서 서버 오류가 났다.

```
AttributeError: module 'app.config' has no attribute 'OUTPUT_WIDTH'
AttributeError: module 'app.config' has no attribute 'GUIDANCE_SCALE'
```

### 3.14.2 코드 수정 방향

설정값을 하나로 고정하지 않고, 여러 이름을 차례로 확인한 뒤 없으면 기본값을 쓰는 \_config\_value()를 만들었다.

```
def _config_value(*names, default=None):
    for name in names:
        if hasattr(config, name):
            return getattr(config, name)
    return default

width = _config_value("OUTPUT_WIDTH", "DEFAULT_WIDTH", default=None)
height = _config_value("OUTPUT_HEIGHT", "DEFAULT_HEIGHT", default=None)
```

입력 이미지가 찌그러지지 않게, 비율을 지키면서 여백을 채우는 함수도 넣었다.

```
def _fit_with_padding(image, size):
    target_w, target_h = size
    src_w, src_h = image.size
    scale = min(target_w / src_w, target_h / src_h)
    new_w = max(8, int(src_w * scale) // 8 * 8)
    new_h = max(8, int(src_h * scale) // 8 * 8)
    resized = image.resize((new_w, new_h), Image.LANCZOS)
    canvas = Image.new("RGB", (target_w, target_h), (245, 245, 245))
    canvas.paste(resized, ((target_w - new_w)//2, (target_h - new_h)//2))
    return canvas
```

### 3.14.3 결과

서버가 더 안정적으로 돌아갔고, 입력 비율이 깨지는 문제도 줄었다.

## 3.15 제품 방향 자동으로 바로잡기

### 3.15.1 실험 목적

입력 제품이 옆으로 눕거나 돌아가 있으면 모델이 제대로 이해하지 못하고 이상하게 그렸다.

### 3.15.2 코드 수정 방향

OpenAI Vision 으로 제품이 똑바로 서려면 얼마나 돌려야 하는지 판단하는 `app/orientation_openai.py` 를 넣었다.

```
def detect_upright_rotation(image_path) -> int:
    """
    Returns clockwise rotation degrees to make the main product upright.
    Allowed values: 0, 90, 180, 270.
    """
```

API 요청 내용은 이랬다.

```
Look at the main product in the image.
Decide how many degrees clockwise the entire image should be rotated
so the main product appears naturally upright for a product photo.
Return only JSON with rotate_degrees.
```

`app/inference.py`에서는 이미지 불러오는 부분을 방향 보정 버전으로 바꿨다.

```
from app.orientation_openai import load_upright_image

def _load_image(path):
    return load_upright_image(path)
```

### 3.15.3 결과

제품 종류를 미리 정해두지 않고 방향만 판단하는 전처리를 넣었다. 다만 OpenAI API 를 부르는 만큼 비용과 시간이 더 드는 단점이 있었다.

## 3.16 OpenAI 이미지 API 옵션 추가

### 3.16.1 실험 목적

FLUX 결과가 계속 아쉬울 수 있으니, 대안으로 OpenAI 이미지 API 도 쓸 수 있게 했다. 엔드포인트를 나눴다.

```
POST      /generate                                ->      FLUX      Kontext
POST /generate-openai  -> OpenAI image API
```

### 3.16.2 코드 수정 방향

app/openai\_image\_provider.py 를 만들었다.

```
response = client.images.edit(
    model=model,
    image=image_file,
    prompt=prompt,
    size=size,
    quality=quality,
)
```

app/api.py 에 /generate-openai 엔드포인트를 넣었다.

```
@app.post("/generate-openai")
async def generate_openai(...):
    result = await asyncio.to_thread(
        _generate_openai_image,
        image_path=temp_path,
        background_style=background_style,
        output_dir=output_dir,
        composition=composition,
        strength=strength,
    )
    return result
```

### 3.16.3 모델 선택 문제

gpt-image-1 을 쓰려 했는데 이런 오류가 났다.

```
RatelimitError: gpt-image-1 RPM Limit 0
```

사용량을 확인해보니 실제로 쓸 수 있는 건 gpt-image-1-mini 였다. .env 를 고쳤다.

```
OPENAI_IMAGE_MODEL=gpt-image-1-mini
OPENAI_IMAGE_SIZE=1024x1024
OPENAI_IMAGE_QUALITY=high
```

input\_fidelity="high" 옵션은 gpt-image-1 에서만 되니, 그 모델일 때만 쓰도록 나눴다.

```
if model == "gpt-image-1":
    kwargs["input_fidelity"] = "high"
```

### 3.16.4 결과

FLUX 와 OpenAI 이미지 API 를 한 서버에서 골라 쓸 수 있는 구조가 됐다.

## 3.17 FastAPI 서버와 프론트/백엔드 연결

### 3.17.1 실험 목적

프론트엔드와 백엔드는 다른 팀원이 맡으니, 모델 파이프라인은 API 로 넘겨줘야 했다.

### 3.17.2 서버 구조

```
uvicorn app.api:app --host 0.0.0.0 --port 8001
```

주요 엔드포인트와 요청 항목은 이렇다.

|      |                     |
|------|---------------------|
| GET  | /health             |
| POST | /generate           |
| POST | /generate-openai    |
| GET  | /outputs/{filename} |

|                                                                                                    |      |
|----------------------------------------------------------------------------------------------------|------|
| image:                                                                                             | file |
| composition:        closeup                 medium                 aerial                 handheld |      |
| background_style:        white                 wood                 vivid                          |      |
| strength: low   medium   high                                                                      |      |

### 3.17.3 연결 구조

프론트가 Cloudflare Pages 로 배포돼 있어서, 모델 서버를 직접 부르면 mixed content 문제가 생길 수 있었다. 그래서 모든 생성 요청이 API 서버를 거치도록 했다.

```
Frontend -> Backend -> Model API Server -> FLUX or OpenAI Image API
```

## 3.18 버전 2 정리

버전 2 는 전체 재생성으로 방향을 틀어서 더 자연스러운 결과를 얻으려 했다. 프롬프트 정리, 참조 스타일 분석, 방향 보정, 외부 API 같은 여러 개선을 시도했다. 서버 구조와 코드 안정성은 많이 좋아졌지만, 정작 가장 중요한 "제품을 안 바꾸면서 자연스러운 장면을 만드는 것"은 끝내 안정적으로 되지 않았다. 전체 재생성에서는 제품이 계속 바뀌었고, 이걸 막으려고 다시 넣은 product-lock 은 버전 1 과 똑같이 제품 떼어내기에서 막혔다. 세기 조절 값도 기대와 반대로 움직였다. 결국 버전 2 는 완성된 해법이라기보다, 여러 개선을 시도해보고 그 한계가 어디인지 분명히 확인한 실험이었다.

## 3.19 결론

### 3.19.1 제품 보존과 자연스러움은 서로 부딪힌다

이 프로젝트에서 가장 크게 배운 건, 제품 이미지 생성에서는 "예쁜 이미지"보다 "입력 제품을 얼마나 그대로 두느냐"가 더 중요하고, 이 둘이 서로 부딪힌다는 점이다. 전체 재생성은 조명과 장면이 자연스럽지만 제품(로고, 모양)을 바꾼다. 반대로 오려 붙이는 합성은 제품은 지키지만 조명이 안 맞아 티가 난다. 버전 1 과 버전 2 는 각각 이 양쪽 끝을 시도해본 셈이고, 어느 한쪽만으로는 답이 되지 못했다.

### 3.19.2 프롬프트는 "만들어라"가 아니라 "그대로 뒀라" 중심으로

버전 2 의 실패들은 대부분 프롬프트가 "예쁘게 만들어라", "이런 걸 넣어라"에 쏠려서 생겼다. 모델은 프롬프트에 나온 것(빨대, 음료, 로고)을 만들라는 뜻으로 받아들였다. 해결의 핵심은 "입력에 있으면 두고 없으면 만들지 마라"로 쓰고, 제품을 영감이 아니라 그대로 베껴야 할 기준으로 다루게 한 것이었다.

```
Treat the uploaded product as the exact product reference, not as an inspiration.  
Only the surrounding scene, background, lighting, shadows, and camera framing may change.  
Do not redesign, simplify, replace, reinterpret, or invent a new version of the product.
```

### 3.19.3 제품을 잘 떼어내야 합성도 산다

버전 1 과 버전 2 모두에서, 제품을 오려 붙이는 방식의 품질은 결국 제품을 얼마나 깔끔하게 떼어내느냐에 달려 있었다. 떼어내기에 실패하면(배경이 딸려 오거나 제품이 잘리면) 배경이 아무리 좋아도 결과가 망가졌다. 이게 합성 방식이 넘기 힘든 한계였다.

### 3.19.4 모델 값보다 프롬프트가 더 예측 가능하다

`image_guidance_scale` 같은 값은 이름과 실제 동작이 반대일 때가 있었다(세기를 낮췄더니 오히려 제품이 더 바뀐). 그래서 이런 값을 억지로 만지기보다, 프롬프트로 "얼마나 보수적으로 할지"를 표현하는 쪽이 더 안정적이었다.

### 3.19.5 안정성 개선은 그 자체로 성과

비록 최종 이미지 품질은 목표에 못 미쳤지만, config 정리(`_config_value`), 비율 유지 여백 채우기, 방향 보정, 엔드포인트 분리, FastAPI 서버화 같은 작업은 파이프라인을 훨씬 튼튼하고 넓게 쓸 수 있게 만들었다.

## 4. 시행착오와 공용 파이프라인 전환

앞선 오픈소스 모델 실험과 초기 LoRA 학습은 제품 보존과 자연스러운 장면 생성의 가능성을 서로 다른 방식으로 검증했다. 결과를 함께 비교한 뒤, 반복해서 나타난 한계를 최종 파이프라인 설계의 기준으로 삼았다.

### 4.1 오픈소스 실험에서 확인한 구조적 한계

SD1.5-SDXL-FLUX.1 Kontext 기반의 전체 재생성은 장면을 자연스럽게 만들 수 있었지만 컵 형태·문자·서빙 상태가 달라질 수 있었다. LoRA 는 무드 적용 가능성을 빠르게 확인하는 데 의미가 있었으나 저해상도 모델의 형태 왜곡과 제품 보존 문제를 해결하지 못했다.

배경 제거 후 합성하는 다중 모델 방식은 원본 제품을 유지했지만 경계·원근·조명·그림자의 불일치가 남았고, 투명컵과 액체처럼 굴절과 반사가 중요한 재질에서 한계가 컸다. 모델별 실행 흐름이 분리돼 반복 생성 시간과 운영 부담도 커졌다.

### 4.2 레퍼런스 기반 프리셋 설계로 전환

최종 프리셋 설계에서는 레퍼런스 이미지를 그대로 복사하거나 단순 합성하지 않고 장면의 원리를 설계 규칙으로 변환하는 방식을 채택했다.

#### 4.2.1 전체 과정

무드와 구도 선정

→ 유사 레퍼런스 수집

→ 카메라·빛·구도·재질 분석

→ 비식별 장면 힌트 제작

→ 프리셋 설계 규칙 작성

→ 사용자 이미지와 결합

→ 생성 결과 검수

이 구조의 핵심은 다음과 같다.

- 사용자 제품 사진은 **음료와 제품 정체성의 기준**
- 장면 힌트는 **카메라, 배치, 조명, 재질 관계의 기준**
- 레퍼런스 속 음료, 로고, 인물, 정확한 그림자는 전달 금지
- 최종 결과의 모든 픽셀은 새 장면에 맞게 다시 생성

#### 4.2.2 방향 전환의 판단 기준

최종 실행 구조는 남은 개발 기간 안에서 결과 품질과 반복 실행 가능성을 함께 확보하기 위해 다음 네 기준으로 설계했다.

1. 제품 정체성과 서빙 상태를 안정적으로 유지할 수 있는가
2. 구도·조명·여백을 프리셋으로 반복 재현할 수 있는가
3. 12 개 장면을 개별 워크플로 복제 없이 운영할 수 있는가
4. 실패 원인을 프롬프트·자산·실행 기록 단위로 추적할 수 있는가

프리셋 설계 규칙 방식은 사용자 제품, 장면 힌트, 런타임 정책의 권한을 분리할 수 있고 하나의 공용 실행 구조로 검증 범위를 통제할 수 있었다. 이에 따라 최종안은 gpt-image-2 기반 재생성 + 규칙 기반 프리셋 + 사람의 시각 QA 로 확정했다.

### 4.3 레퍼런스 수집과 분류

웹사이트와 인스타그램에서 카페 제품 사진 약 700 장을 수동으로 수집했다. 수집 이미지는 학습 데이터로 직접 사용하기보다, 카페 SNS 사진에서 반복되는 장면 원리를 찾기 위한 분석 자료로 활용했다.

1차 분류에서는 화이트·우드·다크그레이처럼 색감과 공간 분위기가 유사한 사진을 묶었다. 2차 분류에서는 카메라 시점, 촬영 거리, 피사체가 화면에서 차지하는 비율을 기준으로 클로즈업·미디엄·손에 든 장면·오버헤드로 나눴다. 이후 빛과 소품의 사용 방식까지 비교해 결이 가까운 사진끼리 다시 정리했다.

특정 사진 한 장을 그대로 복제하지 않기 위해 최종 프리셋에는 원본 이미지를 직접 사용하는 대신, 여러 사진에서 반복된 카메라·배치·빛·재질 정보만 통합한 장면 힌트와 설계 규칙을 사용했다. 필요한 영역은 크롭하고 직접 복제될 수 있는 배경과 세부 요소는 블러 처리했다.

## 4.4 MVP 용 12 개 대표 프리셋 확정

프로젝트 기간과 검증 비용을 고려해 1 차 MVP 는 3 개 무드와 4 개 촬영 구도의 조합인 12 개 프리셋으로 범위를 제한했다. 이는 가능한 모든 감성을 포괄한다는 의미가 아니라, 반복 수요가 높고 서로 구분되는 대표 장면을 안정적으로 제공하기 위한 최소 단위이다. 향후에는 하나의 무드 안에서도 각도·여백·소품이 다른 세부 프리셋을 확장할 수 있다.

| 무드    | 클로즈업 | 미디엄 | 손에 든 장면 | 오버헤드 |
|-------|------|-----|---------|------|
| 화이트   | 1    | 1   | 1       | 1    |
| 우드    | 1    | 1   | 1       | 1    |
| 다크그레이 | 1    | 1   | 1       | 1    |

개인 작업 보고서

# Pickmood

PM · 프리셋 · 장면 · ComfyUI 공용 런타임 설계

작성자 유소연

# 5. 프리셋·장면·공용 런타임 설계 — 유소연

## 5.1 프리셋 패키지의 구성

프리셋은 긴 프롬프트 한 문장이 아니라, 실행에 필요한 조건을 여러 파일로 나눈 데이터 패키지이다.

| 구성 파일                              | 역할                                          |
|------------------------------------|---------------------------------------------|
| registry.json                      | 12 개 슬롯, 공개 상태, 기본 프리셋, provider profile 연결 |
| preset.json                        | 컵 모드, 온도 정책, 힌트 라우팅, 출력 비율, QA 기준           |
| prompt-template-*.txt              | 제품 정체성, 장면, 금지 조건, 모드별 프롬프트                 |
| lighting-sheet.json                | 광원 종류, 방향, 크기, 그림자, 화이트밸런스                  |
| container-design.json              | 레퍼런스 컵 디자인이 필요한 경우의 구조                      |
| scene-recipe.json·scene-graph.json | 사물 구성과 관계, 카메라와 공간 규칙                       |
| 힌트 이미지                             | 배치, 포즈, 재질, 저주파 장면 구조 전달                    |
| qa_criteria                        | 필수 통과 항목, 실패 조건, 비교 평가 항목                   |
| SHA-256 해시                         | 프롬프트와 자산이 배포 후 바뀌지 않았는지 검증                  |

프리셋별 분기 로직을 코드에 계속 추가하는 대신, 공용 런타임이 이 JSON 설계 규칙을 해석하도록 구성했다.

새로운 프리셋을 추가할 때 실행 그래프를 복제하지 않고 패키지를 등록하는 방식이다.

## 프리셋은 '장면 계약'이다

한 문장 프롬프트가 아니라, 재현 가능한 조건을 역할별로 분리한다

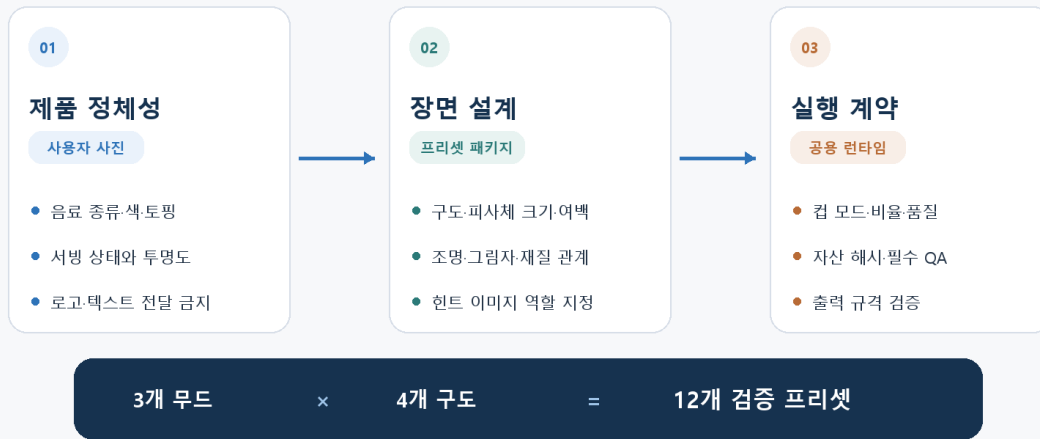


그림 1. 사용자 사진은 제품 정체성을, 프리셋은 장면을, 공용 런타입은 실행 조건을 담당하도록 권한을 분리한 구조

## 5.2 컵 적용 방식

### 5.2.1 사용자 컵 재생성: `reconstruct_source`

- 입력 사진의 컵 계열, 재질, 실루엣, 손잡이, 뚜껑 등 주요 구조를 참고
- 원본 픽셀을 잘라 붙이지 않고 목표 카메라와 빛에 맞게 다시 생성
- 원본 장면의 배경, 조명, 그림자, 카메라 각도는 폐기
- 프리셋에 따라 접시·스푼 등 실제 서빙 구성도 함께 보존

### 5.2.2 레퍼런스 컵 채택: `adopt_reference`

- 사용자 음료의 종류, 색, 층, 얼음, 거품, 토픽과 온도는 유지
- 입력 사진의 컵 형태는 버리고 프리셋에 정의된 컵 디자인 적용
- 장면과 컵 디자인의 통일감을 높일 수 있음

### 5.2.3 현재 브랜드 정책

별도의 사용자 로고 파일 입력과 자동 합성 기능은 현재 비활성화 상태이다. 제품 사진에 보이는 문자와 로고도 프리셋 설계 규칙과 생성 모델 특성에 따라 완전한 보존을 보장하기 어렵기 때문에, 품질 평가에서는 OCR 과 잘못된 글자 생성 여부를 별도로 확인한다.

## 5.3 온도와 서빙 상태

`serving_temperature` 는 생성 자유도를 조절하는 값이 아니라, 뜨거운 음료와 차가운 음료가 뒤바뀌는 문제를 방지하기 위한 설계 규칙이다.

- `auto`: 사용자 이미지에서 보이는 상태를 우선 보존
- `iced`, `cold`, `ambient`, `hot`: 명시적으로 상태 지정
- 레퍼런스와 힌트 이미지는 음료의 온도와 레시피에 대한 권한이 없음
- 입력 사진에 없는 얼음, 김, 결로, 거품을 임의로 추가하지 않도록 제한

## 5.4 장면 힌트의 역할

프리셋별 보조 이미지는 완성된 레퍼런스 사진을 그대로 전달하기 위한 것이 아니다. 다음과 같이 필요한 정보만 분리해 전달한다.

- 카메라 거리와 각도
- 피사체의 상대 크기와 위치
- 테이블·트레이·의자·소품의 관계
- 빛의 대략적인 분포
- 나무, 유리, 도자기 등 핵심 재질
- 손과 컵의 포즈

반대로 다음 항목은 전달하지 않는다.

- 레퍼런스 속 음료와 레시피
- 로고와 문자
- 특정 인물의 정체성
- 정확한 가구와 소품
- 원본의 정확한 그림자와 픽셀 배치
- 보드의 패널 경계와 마스크 형태

### 5.4.1 입력 수를 줄인 2 분할 컨트롤 보드

컵 디자인과 장면 구도가 모두 필요한 프리셋은 정보를 별도 이미지로 각각 보내지 않고, 한 장의 2 분할 컨트롤 보드로 묶었다.

- 왼쪽 패널: 컵의 림·바닥·재질·세로 홈 등 디자인 확대
- 오른쪽 패널: 같은 컵이 실제 배경에 놓인 크기·위치·원근·접지감
- 실행 시 입력: 사용자 제품 1 장 + 통합 컨트롤 보드 1 장
- 패널 경계와 빈 컵의 내용물은 결과로 복제하지 않고, 사용자 음료를 오른쪽 장면에 재구성

이 구조는 컵 디자인과 완성 구도를 동시에 전달하면서 provider 입력 이미지 수를 억제한다. 다만 비용은 이미지 수뿐 아니라 해상도와 provider 의 과금 방식에도 영향을 받으므로, 입력 수 감소를 비용 절감률과 동일하게 해석하지 않는다. 화이트 미디엄처럼 배경 변형이 여러 개인 프리셋은 후보 보드 중 한 장만 실행 시 무작위로 선택해 전달한다.

## 5.5 장면 설계 사례



그림 2. 컵 디자인 확대와 같은 컵이 장면에 놓인 상대 크기를 한 장에 결합한 2 분할 컨트롤 보드. 보조 입력 수를 한 장으로 유지하면서 모드별 역할을 분리한다.

우드 항공샷은 단순히 나무 테이블 위에 음료를 올리는 프롬프트가 아니다. 다음 요소가 함께 정의된다.

- 45~55mm 환산 렌즈
- 높은 오버헤드 카메라
- 상단 좌측에서 들어오는 자연광
- 하단 우측으로 이어지는 그림자
- 우측 원형 테이블과 좌측 상단 의자의 관계
- 음료와 브라우니·스푼의 상대 배치
- 제품과 소품이 잘리지 않는 최종 4:5 캔버스

## 5.6 프리셋 코드 예시

아래는 실제 구조를 이해하기 쉽도록 축약한 예시이다.

```
{
  "slot_id": "wood__product_center",
  "preset_id": "tokyo_a6_relational_scene_hint_v4",
  "runtime_policy": {
    "default_container_mode": "reconstruct_source",
    "supported_container_modes": ["adopt_reference", "reconstruct_source"],
    "temperature_policy": {"mode": "preset_primary"}
  },
  "hint_images": [
    {
      "role": "source_container_scene_hint",
      "container_mode_scope": ["reconstruct_source"],
      "send_to_provider": true
    },
    {
      "role": "reference_container_scene_hint",
      "container_mode_scope": ["adopt_reference"],
      "send_to_provider": true
    }
  ],
  "aspect_ratio_criterias": {
    "4:5": {"generation_size": "1024x1280", "bbox_qa": {"crop_allowed": false}},
    "1:1": {"generation_size": "1024x1024", "bbox_qa": {"crop_allowed": false}}
  },
  "qa_criteria": {
    "hard_gates": ["identity", "container_mode", "crop", "subject_count"]
  }
}
```

## 5.7 프롬프트 조립 예시

실제 프롬프트는 제품 정체성, 장면, 빛, 비율, 금지 조건을 블록으로 나눠 조립한다.

```
[EXACT PRODUCT - HARD LOCK]
Image 1 is the sole exact user-product identity and the only authority
for the featured beverage, serving temperature, liquid layers,
toppings, ice state and container policy.
```

```
[LIGHTING SHEET - AUTHORITATIVE]
Use the resolved key-light direction, shadow attachment,
white balance and material response from the preset lighting sheet.
```

[SCENE HINT SAFETY]  
 The scene hint may control only camera distance, subject scale, object relationships, broad light distribution and material family. It must not supply beverage identity, text, logo or exact pixels.

[RUNTIME POLICY - AUTHORITATIVE]  
 Preserve the source serving state. Generate no undeclared product, logo, label or watermark. Return one aspect-native image only.

프롬프트의 목적은 설명을 길게 늘리는 것이 아니라, 서로 충돌하기 쉬운 권한을 명확히 나누는 것이다.

## 5.8 구현 및 운영 구조

### 5.8.1 통합 구현 범위

최종 서비스는 12 개 프리셋과 힌트 자산, ComfyUI 공용 런타임, 프론트엔드·백엔드 연결, GCP 배포와 Langfuse 관측까지 하나의 실행 흐름으로 통합했다.

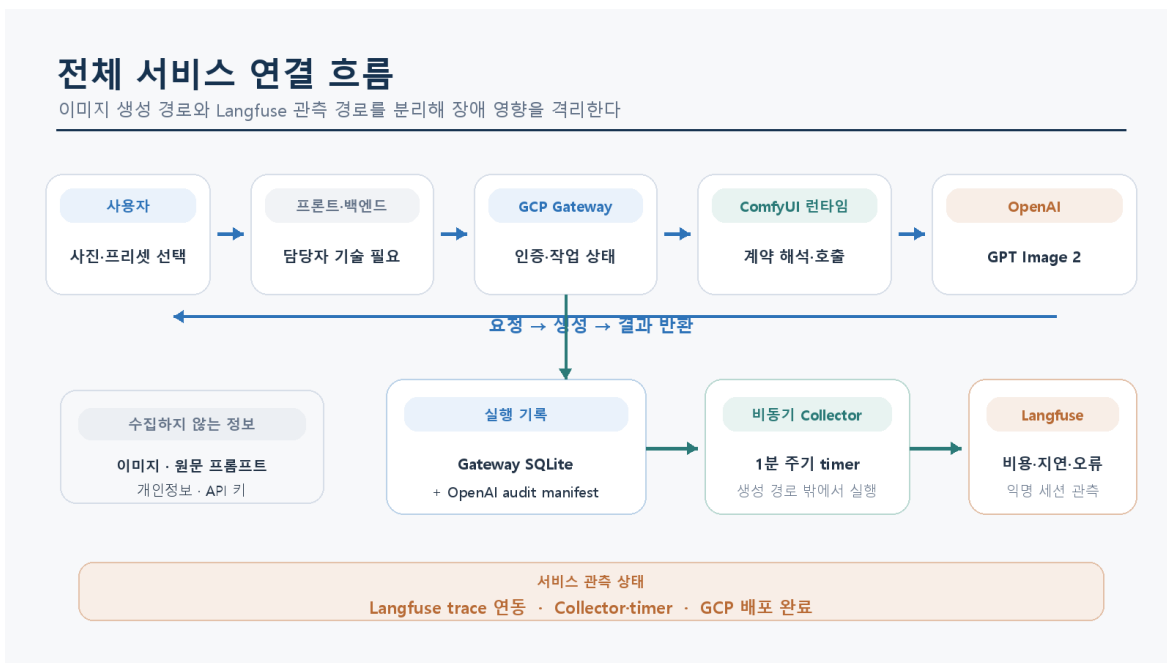


그림 3. 사용자 요청부터 OpenAI 이미지 생성·결과 반환까지의 주 경로와, Gateway SQLite-audit manifest 를 읽어 Langfuse 로 보내는 비동기 관측 경로

### 5.8.2 최종 제출 기준 브랜치 흐름

```

backend ————┐
frontend ————┤
presets ————┤
serving ————┤ → deploy(통합·E2E 검증) → PR #4 → main
langfuse ————┘
model-c ————— 독립 모델 실험 이력
  
```

- frontend·backend·presets·serving·langfuse 는 deploy 에서 통합 E2E 검증 후 PR #4 로 main 에 병합했다.
- deploy 는 병합 후 자동 삭제되었으며, main 이 최종 제출·배포 기준이다.
- model-a·model-b 는 삭제했고 model-c 만 독립 실험 이력으로 유지했다.
- backend·frontend·langfuse·presets·serving 원격 브랜치는 담당 영역의 이력 보존용이며 운영 소스는 main 을 기준으로 한다.

### 5.8.3 최종 제출 기준 저장소 구조

아래 트리는 최종 서비스의 저장소 구성을 요약한다.

```
Pickmood/
├── README.md
├── frontend/           # 사용자 화면
├── app/                # 백엔드 API·캡션 생성
├── comfyui/
│   ├── config/providers/ # 모델 제공자 설정
│   ├── presets/         # 12 개 프리셋·힌트 자산
│   ├── workflows/       # 공용 실행 워크플로
│   ├── custom_nodes/ad_creator/ # 프리셋 해석·모델 호출
│   ├── orchestrator/    # 실행 조립·라우팅
│   ├── gateway/         # 생성 요청·상태·결과 API
│   ├── observability/   # Langfuse 수집·비용 관측
│   ├── deploy/         # GCP 배포 구성
│   ├── scripts/        # 운영·검증 도구
│   └── tests/          # 설계 규칙·회귀 테스트
├── comfyui_nodes/      # 노드 패키지·호환 코드
└── requirements.txt
```

### 5.8.4 ComfyUI 공용 런타임

초기 제작 워크플로는 프롬프트 조립, 힌트 선택, QA, 리사이즈, LUT, 저장 단계가 각각 독립 노드로 노출돼 있었다. 이 구조는 프리셋을 제작하고 문제를 찾는 데 유용했지만, 실서비스에서는 다음 문제가 있었다.

- 12 개 프리셋별 워크플로 복제 가능성
- UI 와 Gateway 의 실행 로직 차이
- 사용자의 내부 설정 오조작 위험
- 프롬프트와 힌트 변경 추적 어려움
- 노드 변경에 따른 회귀 범위 확대

최종 서비스 워크플로는 다음 세 노드로 단순화했다.

```
LoadImage
→ AdCreatorOpenAIImageGenerate
→ SaveImage
```

## 복잡성은 제작 단계에, 조작면은 단순하게

프리셋별 워크플로를 복제하지 않고 하나의 공용 런타임이 계약을 해석한다

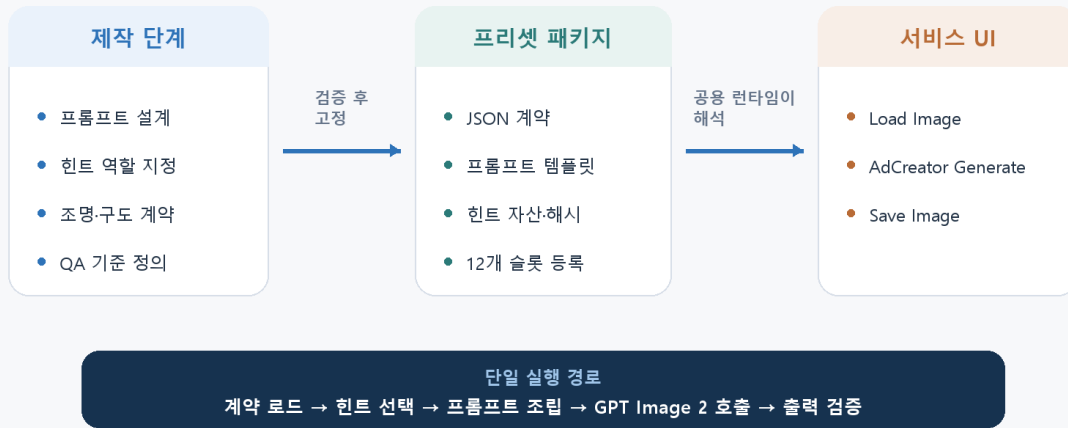


그림 4. 프리셋 제작·검증용 복합 그래프에서 실제 서비스용 3 노드 공용 런타임으로 전환한 구조

화면에는 세 노드만 보이지만, 공용 생성 노드 내부에서는 다음 작업이 수행된다.

1. 프리셋과 컵 모드 해석
2. 온도 정책 확인
3. 프롬프트·조명·힌트 자산 로드
4. 상대경로·파일 크기·SHA-256 검증
5. 모드에 맞는 보조 이미지 1 장 선택
6. 최종 프롬프트 조립
7. OpenAI Images API 호출
8. 출력 형식과 크기 확인
9. provider 원본과 실행 기록 저장

즉, 단순화는 기능 삭제가 아니라 **실행 책임을 공용 런타임 안으로 캡슐화한 결과**이다.

### 5.8.5 후처리를 적용하지 않은 이유

현재 결과에는 중앙 크롭, 추가 리사이즈, LUT 또는 색보정을 적용하지 않는다.

- 4:5 는 처음부터 1024×1280 으로 생성

- 1:1 은 처음부터 1024×1024 로 생성
- 반환 비율이 설계 규칙과 다르면 임의로 자르지 않고 오류 처리
- `grade_profile.apply_in_pilot=false`
- 생성 결과를 그대로 `SaveImage` 로 전달

후처리를 적용하면 다음 문제가 발생할 수 있다.

- 프리셋에서 설계한 제품 크기와 여백 훼손
- 유리와 액체의 미세 질감 손실
- 색온도와 음료 색상 변화
- 그림자 또는 소품 잘림

따라서 현재 단계에서는 모델이 최종 비율의 이미지를 직접 생성하도록 하고, 원본 결과를 그대로 보존하는 방식을 선택했다.

## 5.8.6 GCP 배포 구조

GCP 의 ComfyUI 는 로컬 이미지 모델을 추론하는 GPU 서버가 아니라, 프리셋을 해석하고 외부 이미지 생성 API 를 호출하는 CPU 기반 오케스트레이션 계층으로 운영된다.

### 5.8.6.1 실행 구성

- ComfyUI: 127.0.0.1:8188
- Generation Gateway: 127.0.0.1:8002
- 외부 진입점: Caddy HTTPS 443
- ComfyUI 와 Gateway 는 외부에 직접 공개하지 않음
- ComfyUI 와 Gateway 는 별도 systemd 서비스
- Gateway worker 1 개
- 작업 상태: SQLite 저장
- 입력 제한: JPEG·PNG·WebP, 최대 20MiB

### 5.8.6.2 보안과 중복 과금 방지

- OpenAI API 키는 GCP ComfyUI 프로세스에만 주입
- Gateway API 키와 결과 서명 키 분리

- 사용자 생성 동작마다 `Idempotency-Key` 발급
- 같은 요청의 네트워크 재시도에는 동일 키 사용
- 상태가 불명확한 작업을 자동으로 새 유료 생성하지 않음

### 5.8.7 실행 기록과 비용 관측

각 실행은 다음 정보를 audit sidecar 에 기록한다.

- provider 와 모델명
- quality
- preset ID 와 slot ID
- 컵 모드와 온도 해석 결과
- 입력 이미지 역할.크기.해시
- 프롬프트.bundle.provider profile 해시
- OpenAI request ID 와 client request ID
- 출력 크기와 결과 해시
- API usage
- 처리 시간 `elapsed_ms`

프롬프트 본문, 비밀키, 사용자 원본 경로는 audit 파일에 저장하지 않는다.

#### 5.8.7.1 Langfuse 관측 연동

Langfuse 관측 코드를 GCP 에 배포했다. 생성 요청 경로에서 직접 전송하지 않고, 1 분 주기의 systemd timer 가 완료 작업만 읽는 Collector 를 실행한다.

```
Gateway SQLite + OpenAI audit manifest
→ 읽기 전용 Collector
→ Langfuse v4
```

- 수집: workflow, 성공·실패 상태, 모델 생성 시간, 사용자 체감 완료 시간, 모델, 품질, preset ID, 토큰, 공식 단가 기준 USD 비용, 익명 세션 ID
- 미수집: 업로드·결과 이미지, 원문 프롬프트, 파일 경로·해시, 이메일, 이름, IP, API 키
- 장애 격리: Collector 는 이미지 생성 요청 경로 밖에서 동작하므로 Langfuse 장애가 생성 성공 여부와 응답 시간에 영향을 주지 않음
- 중복 방지: 정상 전송한 job ID 를 별도 state DB 에 기록

2026 년 7 월 29 일 GCP 점검에서 Collector 와 timer 의 active/enabled 상태, 익명 세션 전달, 실제 완료 작업 trace 수집을 확인했다. 요청 수락·대기·실행·성공·프론트엔드 표시 완료를 같은 익명 세션에 기록해 모델 생성 시간과 사용자 체감 완료 시간을 함께 비교할 수 있음을 확인했다.

## 6. 품질·비용·모델 선택 평가 — 유소연

### 6.1 구현 완료 기능

#### 6.1.1 이미지 생성

- 화이트·우드·다크그레이 3 개 무드 계열
- 4 개 촬영 구도
- 총 12 개 프리셋
- 사용자 음료 사진 반영
- 사용자 컵 재생성 / 레퍼런스 컵 채택
- 4:5 / 1:1 선택
- 프리셋별 장면 힌트 적용
- 무크롭 PNG 결과 제공

#### 6.1.2 게시물 문구

- 결과 이미지 기반 캡션 생성
- 해시태그 8 개
- 스토리용 짧은 문구
- 메뉴명과 게시 목적 반영
- 한국어·영어 출력 지원

#### 6.1.3 운영 구조

- ComfyUI 공용 런타임
- Gateway 인증과 작업 큐
- 요청 중복 방지
- 작업 상태 조회
- provider 원본과 audit 기록 저장

- GCP systemd 서비스 운영
- Langfuse 완료 작업 Collector 와 1 분 주기 timer 배포

## 6.2 현재 확인된 구현 사실

- 이미지 생성 모델: `gpt-image-2`
- 서비스 기본 품질: `medium`
- ComfyUI 내부 선택 가능 품질: `low`, `medium`, `high`
- 출력 형식: PNG
- 4:5: `1024×1280`
- 1:1: `1024×1024`
- 기본 워크플로: `gpt-image-2-v1`
- 12 개 프리셋 모두 `published` 상태로 등록
- 12/12 프리셋 대표 결과에 대한 최종 시각 검수 완료
- 12 개 프리셋 모두 4:5·1:1 설계 규칙과 두 컵 모드 해석 가능
- 프리셋 브랜치 회귀 테스트 78/78 통과
- GCP 통합 런타임 테스트 86 건 수행, 실패 0, 선택 의존성 테스트 1 건 건너뛴
- 기준 프리셋 12 개 GCP 런타임 반영 완료
- 프리셋과 자산의 상대경로·크기·SHA-256 검증
- 최신 프리셋 GCP 배포, ComfyUI·Gateway health 정상
- 프론트엔드·백엔드 익명 세션 전달과 Langfuse 실제 trace E2E 확인 완료
- 화이트·다크그레이 클로즈업 사용자 컵 슬리브·로고 보존과 다크그레이 빨대·뚜껑 보존 배포 완료
- 화이트 미디엄 힌트 보드 3 종을 명시적으로 추적하고 요청마다 1 종만 무작위 선택하도록 배포 완료

### 6.3 검증 완료 범위와 해석

| 검증<br>계층       | 범위                                  | 결과                                               |
|----------------|-------------------------------------|--------------------------------------------------|
| 프리셋<br>등록      | 3 개 무드 × 4 개 구도                     | 12/12 공개 상태 확인                                   |
| 설계<br>규칙<br>검증 | 경로, JSON 구조, 프롬프트·자산 해시, 입력 수<br>제한 | 12/12 통과                                         |
| 런타임<br>검증      | 두 컵 모드, 4:5:1:1, Medium 기본값, 오류 처리  | 프리셋 브랜치 78/78 통과, GCP 통합 런타임<br>86 건·실패 0        |
| 시각 QA          | 각 프리셋의 대표 실제 생성 결과                  | 12/12 승인                                         |
| GCP 반영         | 공용 ComfyUI 런타임에서 기준 프리셋 로드·호출       | 12/12 기준본 반영 완료                                  |
| 관측<br>연동       | Collector·timer·익명 세션·Langfuse 설정   | 실제 trace E2E·모델 생성 시간·사용자 체감 완료 시간<br>확인 및 배포 완료 |

유료 전수 검증은 12 개 프리셋 × 2 개 컵 모드(사용자 컵·레퍼런스 컵) × 3 개 품질(Low·Medium·High)의 총 72 개 조합을 대상으로 완료했다. 각 조합을 실제 유료 API 로 생성해 프리셋 적용, 컵 모드 해석, 품질 단계별 결과와 비용·시간 로그 수집 여부를 확인했다. 다만 이는 기준 입력을 사용한 통제된 검증이며, 모든 음료와 촬영 상태에서 동일한 성공률을 보장한다는 뜻은 아니다.

장기간 운영 로그가 아직 충분하지 않아 평균 생성 시간, p95, 1 회 평균 비용과 전체 요청 성공률을 확정 지표로 제시하지 않았다. 대신 Langfuse 에 모델 생성 시간, 사용자 체감 완료 시간, usage 와 비용을 분리 기록해 향후 운영 지표의 기준으로 삼는다.

### 6.4 Langfuse 기반 비용·지연 시간 관측

초기 운영 표본으로 2026 년 7 월 28 일 Langfuse 에서 확인한 gpt-image-2 medium 생성 기록 12 건의 평균 비용은 1 회당 약 0.0791 달러였다. 입력 비용은 평균 0.0301 달러, 출력 비용은 평균 0.0490 달러였고, 총비용 범위는 0.0684~0.0974 달러였다. 서버 비용을 제외하면 월 100 회 생성은 약 7.91 달러, 1,000 회 생성은 약 79.13 달러로 추정된다.

Langfuse에서는 모델 생성 시간과 사용자 체감 완료 시간을 분리해 기록한다. 모델 생성 시간은 AI가 이미지를 생성하는 데 걸린 시간이며, 사용자 체감 완료 시간은 클릭 후 결과를 확인하기까지 걸린 전체 시간이다. 최신 실제 요청에서는 각각 약 61초와 84초가 기록되어, 약 23초의 차이가 대기·상태 조회·결과 로드 등 서비스 구간에서 발생했음을 확인했다.

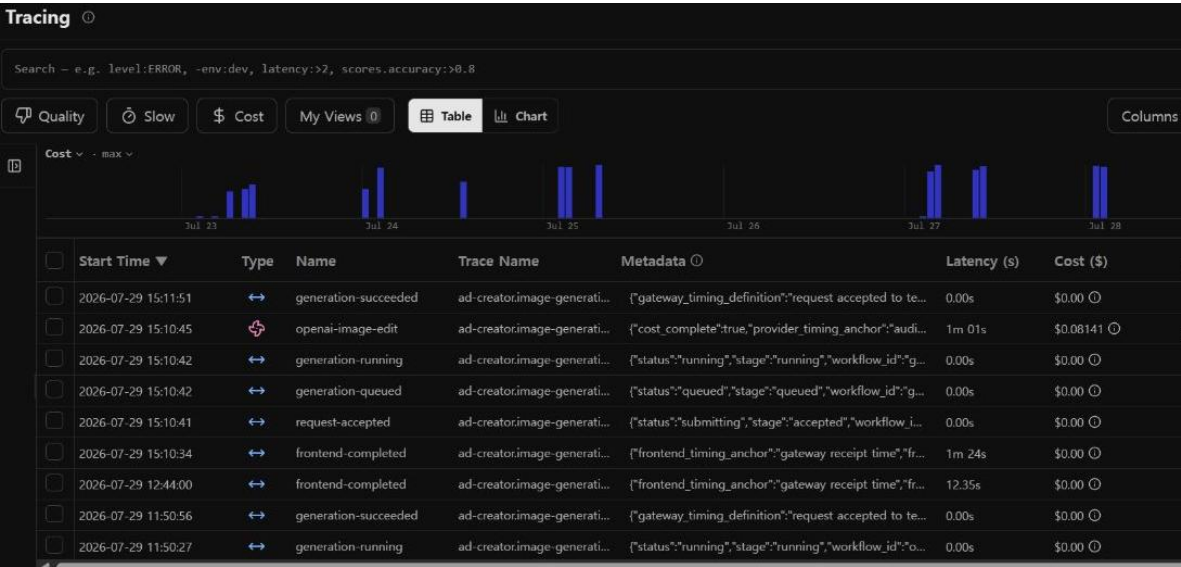


그림 3. Langfuse의 모델 생성 시간(61초)과 사용자 체감 완료 시간(84초) 동시 추적 화면(식별 정보 제거)

## 6.5 gpt-image-2 medium 채택 근거

서비스 기본 품질은 medium으로 설정했다. low는 빠르고 저렴하지만 제품의 세부 질감과 손·컵 경계가 단순화될 수 있고, high는 세부 묘사를 강화하는 대신 비용과 대기 시간이 증가한다. medium은 제품 정체성, 카페 사진의 자연스러움, 반복 생성 비용 사이의 균형을 위한 기본값이다. Low·Medium·High 비교는 12개 프리셋과 사용자 컵·레퍼런스 컵 두 방식으로 모두 유료 생성해 확인했다.

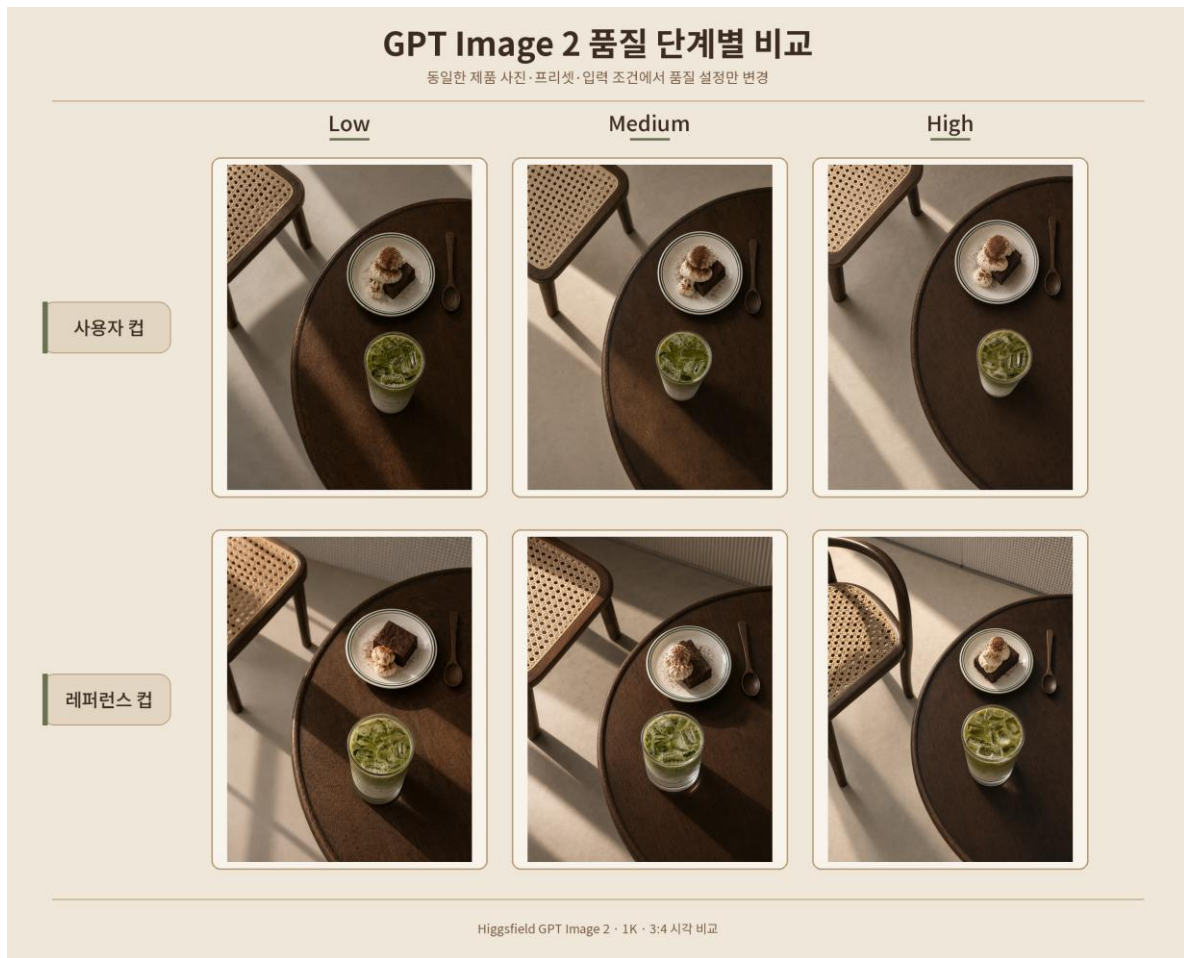


그림 4. 동일 입력·프리셋에서 gpt-image-2 품질 단계별 결과 비교(1K·3:4 시각 비교)

## 6.6 공용 파이프라인에서의 오픈소스 모델 재검토

프리셋과 입력 역할이 정리된 뒤 FLUX.2-klein-4B 와 Mage-Flow 계열을 같은 이미지 조건으로 다시 비교했다. 두 모델은 각각 속도, 제품 반영, 장면 생성에서 장단점이 있었지만, 현재 설정만으로는 Pickmood 가 목표로 하는 자연스러운 카페 SNS 사진 품질에 충분히 도달했다고 보기 어려웠다.

다만 이 결과는 오픈소스 모델 자체의 가치가 낮다는 의미가 아니다. 다른 이미지 과제에서는 ComfyUI 워크플로와 후처리 설계에 따라 비용 효율적인 선택이 될 수 있다. 이번 재검토를 통해 오픈소스 또는 상용 API 를 일률적으로 고집하기보다, 서비스의 목표와 제품 보존 요구에 맞춰 모델을 선택하고 고도화해야 한다는 점을 확인했다.

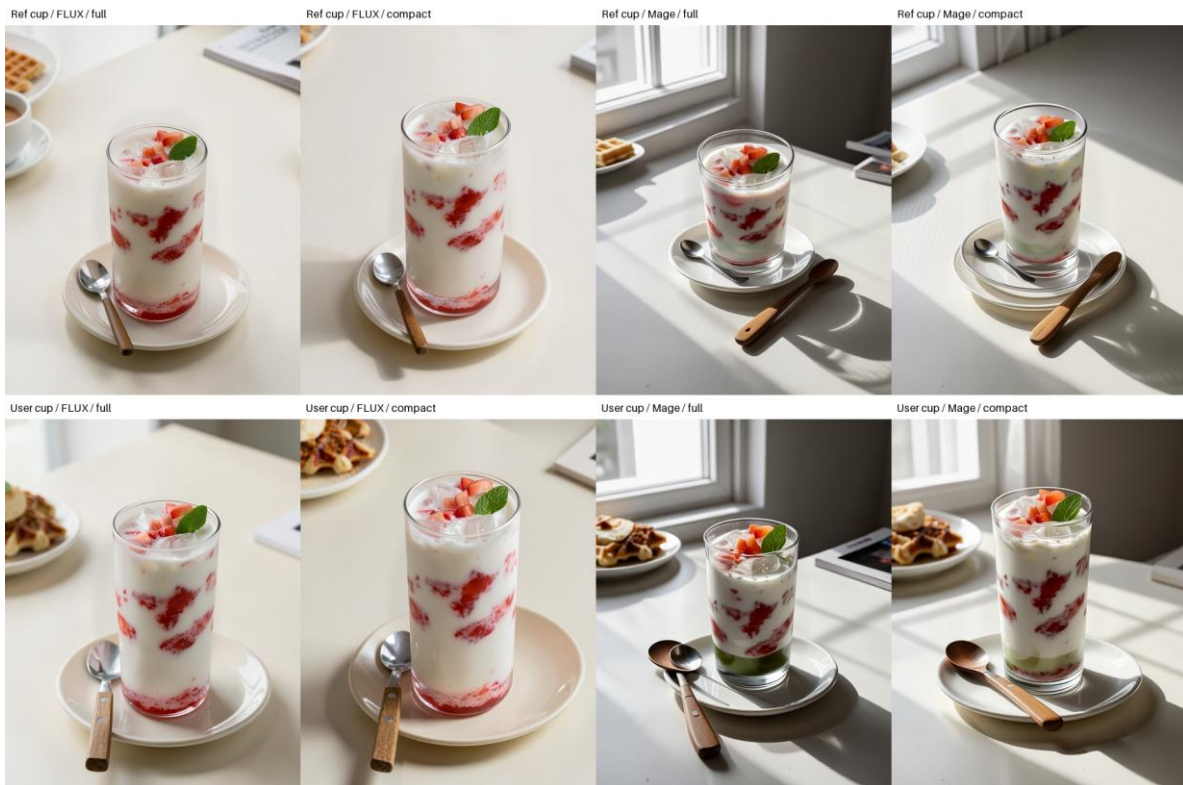


그림 5. 공용 파이프라인에서 재검토한 오픈소스 모델 8-way 비교

## 6.7 품질 평가 항목과 기준

### 6.7.1 평가 목적 및 검증 결과

정상 반환만으로 게시 활용성을 판단할 수 없어 제품 정체성·프리셋 의도·자연스러움·운영 성능을 함께 평가했다. 최종 검증에서는 12 개 프리셋 × 2 개 컵 모드 × 3 개 품질의 72 개 유료 조합을 모두 생성하고, 대표 결과를 시각 QA 해 12/12 프리셋을 승인했다.

## 72개 유료 조합 전수 검증 완료

12개 프리셋 × 2개 컵 모드 × Low-Medium-High

72 / 72

유료 생성 조합 완료

12 / 12

프리셋 최종 승인

회귀 검증 통과

공용 런타임-라우팅-출력 규칙

### 유료 생성 검증

각 조합의 생성 성공-컵 모드-품질 단계 결과를 확인  
동일한 공용 런타임에서 프리셋과 품질만 바뀌 실행  
실패 호출은 재시도 기록과 함께 별도로 분리

### 시각 QA

제품 정체성-장면 의도-자연스러움을 사람의 눈으로 확인  
구도-빛-재질-손-문자 오류와 실제 게시 활용성 검토  
필수 기준과 대표 결과를 함께 확인해 12개 모두 승인

검증 경계 · 전수 생성 완료 ≠ 모든 사용자 입력에서의 통계적 품질 보장

그림 6. 12개 프리셋 × 2개 컵 모드 × 3개 품질의 72개 유료 생성 조합 전수 검증 결과

검증 경계: 72개 조합은 품질 단계와 컵 모드의 실행-결과를 전수 확인한 범위이며, 모든 사용자 입력과 음료 상태에서 동일한 품질을 통계적으로 보장한다는 뜻은 아니다.

### 6.7.2 평가 범위

- 유료 생성 조합: 12개 프리셋 × 2개 컵 모드 × Low-Medium-High = 72개
- 출력 비율: 4:5:1:1은 공용 런타임과 자동 회귀 검증으로 확인했으며 72개 조합에는 중복 산입하지 않았다.
- 시각 QA: 투명·유리컵, 온도·층·토픽, 손·다중 피사체, 항공샷 등 오류 가능성이 높은 입력을 대표 사례로 확인했다.
- 평가 방식: 필수 통과 기준을 우선 적용하고 5점 척도는 프리셋 간 비교와 개선을 위한 보조 지표로 사용했다.

### 6.7.3 승인 기준

아래 항목 중 하나라도 치명적으로 실패하면 해당 결과는 승인하지 않는다.

| 평가 항목        | 통과 기준                                     |
|--------------|-------------------------------------------|
| 제품 정체성·서빙 상태 | 음료 종류·색·층·얼음·거품·토픽과 hot/cold 상태가 입력과 일치   |
| 컵 모드·형태      | 사용자 컵 또는 레퍼런스 컵 정책에 맞고 실루엣·손잡이·접지면이 자연스러움 |

| 평가 항목         | 통과 기준                                       |
|---------------|---------------------------------------------|
| 구도·잘림·사물 수    | 제품 크기·중심 위치·여백과 필수 사물이 프리셋 허용 범위 안에 있음      |
| 손·접촉·물리 관계    | 손가락 수, 잡는 각도, 컵과 소품의 접촉 및 원근 관계에 치명적 오류가 없음 |
| 빛·그림자·재질      | 광원 방향과 그림자가 일치하고 유리·도자기·나무 질감이 과장되지 않음      |
| 문자·로고·레퍼런스 유출 | 가짜 문자·로고·워터마크 및 힌트 이미지의 직접 복제가 없음           |
| 출력 형식         | 요청 비율과 파일 형식을 만족하고 깨짐·빈 이미지·중복 결과가 없음       |

#### 6.7.4 구도·정성 평가

프리셋별 좌표·비율 규칙과 사람의 시각 판단을 함께 사용한다. 5 점 척도는 비교용이며, 최종 승인은 필수 기준과 실제 게시 활용성을 우선한다.

| 평가 영역    | 판단 기준                                | 검증 방식         |
|----------|--------------------------------------|---------------|
| 제품 크기·위치 | 프리셋별 허용 범위 안에서 주 피사체가 의도한 위치와 크기를 유지 | 좌표·비율 + 시각 확인 |
| 여백·사물 관계 | 필수 소품, 상대 위치, 잘림 금지와 의미 있는 여백을 충족    | 설계 규칙 + 시각 확인 |
| 자연스러움    | 실제 카페나 일상 공간에서 촬영한 사진처럼 보임           | 5 점 보조 척도     |
| AI 인위성   | 반복 질감, 과도한 매끈함, 비현실적 형태·빛이 두드러지지 않음  | 5 점 보조 척도     |
| 무드 일관성   | 프리셋이 의도한 색·빛·재질과 구도가 반복 결과에서 유지      | 프리셋 간 비교      |
| 게시 활용성   | 치명적 오류 없이 카페 SNS 게시물로 바로 활용 가능한 수준   | 팀 시각 검수       |

#### 6.7.5 운영 지표

| 운영 지표        | 기록·해석 기준                                                            |
|--------------|---------------------------------------------------------------------|
| 모델 생성 시간     | AI 가 이미지를 생성하는 모델 호출 구간의 소요 시간                                      |
| 사용자 체감 완료 시간 | 생성 클릭부터 사용자가 결과를 확인하기까지의 전체 시간                                      |
| 비용·성공 상태     | 프리셋·품질별 usage 와 비용, 성공·오류 유형을 함께 기록                                 |
| 추적·개인정보      | trace 와 익명 session 으로 요청을 연결하되 입력 이미지·프롬프트·개인정보는 Langfuse 에 전송하지 않음 |

## 개인 작업 보고서

### 7. 서비스 구현과 전체 아키텍처 — 송우현

사진 업로드 한 장으로 인스타그램 무드컷을 만들어주는 서비스, Picmood

담당: 서비스 개발 · 모델 서빙 (프론트엔드 · 백엔드 · 배포)

**작성자** 송우현

**작업 기간** 2026. 7. 1. ~ 2026. 7. 27.

**기술 스택** React(Vite) · FastAPI · ComfyUI Gateway · OpenAI gpt-image-2 · Docker · Render · Cloudflare Pages

## 7.1 개요

이 보고서는 팀 프로젝트 Picmood(카페 음료 사진 → 인스타그램 무드컷 생성 서비스)에서 서비스 개발과 모델 서빙을 담당하며 진행한 작업을 정리한 개인 보고서다.

담당 범위는 프론트엔드(React/Vite), 백엔드(FastAPI), 모델 서빙 연동, 배포(Docker · Render · Cloudflare Pages)까지로, 팀 내에서 실제 서비스가 "작동하게 만드는" 부분 전체를 책임졌다. 모델 자체(이미지 생성 로직)는 다른 두 팀원과 팀장님이 개발했고, 나는 그 결과물을 서비스에 실제로 연결하고 사용자에게 보여주는 파이프라인을 설계·구현·운영했다.

### 7.1.1 전체 여정 요약

프로젝트는 크게 세 번의 방향 전환을 거쳤다. 처음엔 "제품 사진에 광고 문구를 엮는 서비스"로 시작했으나(7/1~7/2), 회의를 통해 "레퍼런스 스타일을 골라 사진을 재보정하는 서비스"로 전면 전환했고(7/6), 모델이 자체 개발한 로컬 모델(Flux, 7/15)에서 팀장이 구축한 ComfyUI Gateway(7/20), 그리고 최종적으로 OpenAI의 gpt-image-2 기반 신규 파이프라인(7/23~7/27)으로 모델 서빙 경로 자체도 세 차례 바뀌었다. 이런 잦은 변경 속에서 서비스 코드가 매번 크게 흔들리지 않도록 인터페이스를 먼저 고정해두는 설계를 일관되게 유지한 것이 이 기간 동안의 핵심 전략이었다.

### 7.1.2 시스템 구조 (최종 형태)

```
사용자 → 프론트(React, Cloudflare Pages)
      → 백엔드(FastAPI, Render/Docker)
        → ComfyUI Gateway (제출 → 폴링 → 결과 다운로드)
          → workflow_id 별 실제 모델(gpt-image-2-v1 / 레거시 model-c-v1)
        → 캡션 생성(OpenAI GPT-4o-mini, Vision 기반)
```

## 7.2 진행 타임라인과 주요 의사결정

시간 순서대로, 각 국면에서 실제로 무엇을 만들었고 왜 그렇게 결정했는지를 정리한다.

### 7.2.1 Phase 1 · 7/1~7/2 — 초기 뼈대 구축과 방향 탐색

이미지 생성 모델 코드가 없는 상태에서도 서비스 전체 흐름을 검증할 수 있도록, FastAPI 백엔드 + Streamlit 프론트엔드로 최소 구조를 먼저 완성했다. 모델 호출 부분은 입력 이미지를 그대로 반환하는 더미 함수로 비워두고, 나머지(웹 화면, API 서버, 인스타그램 비율 리사이즈)를 먼저 동작하게 만드는 방식을 택했다.

▶ **내 판단/제안:** 모델·프론트·백엔드가 동시에 개발되는 상황에서는, 실제 로직이 없어도 입출력 형식(PIL Image in/out)만 팀원과 미리 합의해두면 각자 독립적으로 작업을 진행할 수 있다고 판단해 이 구조를 구현했다.

이들째(7/2)에는 "광고 이미지 위 문구를 사용자가 직접 편집할 수 있게 하자"는 기획에 맞춰, Streamlit을 버리고 React + react-konva 기반 캔버스 편집기로 전면 재구축했다. Streamlit은 상호작용마다 스크립트 전체가 재실행되는 구조라 드래그 같은 연속 인터랙션 구현에 부적합하다고 판단한 것이 전환의 이유였다. 백엔드

응답도 PNG 한 장에서, 이미지 레이어와 텍스트 레이어(좌표는 0~1 비율)를 함께 담은 JSON 구조로 바꿨다 — 캔버스 크기가 달라져도 좌표 계산이 깨지지 않도록 하기 위한 설계였다.

### 7.2.2 Phase 2 · 7/6~7/8 — 서비스 방향 전환과 UI 재구축

팀 회의에서 "텍스트를 얹는 광고 배너"는 차별성이 약하다는 결론이 나와, "사용자가 서투르게 찍은 카페 음료 사진을 업로드하면 레퍼런스 스타일 중 하나를 골라 그 분위기로 재보정해주는 서비스"로 완전히 전환했다. 전날 만든 텍스트 레이어 편집기는 새 서비스에 텍스트 자체가 없어 그대로 폐기했다.

▶ **내 판단/제안:** 무드·구도 이름과 개수가 모델팀 진행 상황에 따라 계속 바뀔 것으로 예상해, 이를 코드에 하드코딩하지 않고 references.json 이라는 별도 데이터 파일로 분리하는 구조를 설계했다. 실제로 이후 무드 10 개 × 구도 3 개 → 3 개 × 4 개로 구조가 완전히 바뀌었을 때도 프론트·백엔드 코드는 건드리지 않고 데이터 파일만 갱신해 대응할 수 있었다.

```
{
  "id": "warm_cafe_top_view",
  "mood_id": "warm_cafe",          // 코드가 그루핑할 때 쓰는 값
  "mood_label": "따뜻한 카페 감성", // 화면 표시용
  "composition_id": "top_view",
  "thumbnail_url": "..."
}
```

7/7 에는 모델 제작 담당(정설아)이 만든 config 파일과 내 references.json 의 키 이름이 서로 달라(neutral\_white vs mood\_natural\_white 등) 생기는 문제를 발견해, 두 쪽이 어느 이름으로 통일할지 직접 조율하고 데이터·이미지 파일명을 일괄 변경했다. 또한 형용사 위주 프롬프트 ("익스트림 클로즈업" 등)가 매번 다른 결과를 낳는 것을 실제 이미지 비교로 확인하고, 화면 채움 비율(%)과 카메라 각도를 숫자로 명시하는 방식으로 구도 프롬프트를 다시 쓸 것을 모델 제작 담당(정설아)에게 제안했다.

7/8 에는 인스타그램풍 카드 UI(PostFrame 컴포넌트)를 새로 설계했다. 실제 인스타그램 로고·워드마크나 실존 인물 사진은 쓰지 않고, 좋아요·댓글·공유 아이콘 같은 범용 소셜 UI 패턴만 차용하는 방향으로 저작권 문제를 스스로 정리했다. 이날 모바일 접속이 전혀 안 되는 문제를 4 단계(로컬호스트 특성 → API 주소 하드코딩 → CORS → 공유기 AP 격리)에 걸쳐 순차적으로 진단해 해결했다.

### 7.2.3 Phase 3 · 7/9 — 캡션 생성 기능과 최초 배포

GPT-4o-mini 를 붙여 캡션·해시태그·스토리 문구 생성 기능을 새로 만들었다. 처음엔 텍스트 힌트(무드·구도명)만으로 캡션을 생성했지만, 실제 이미지를 전혀 보지 않고 있다는 한계를 발견해 Vision 기반(결과 이미지를 base64 로 직접 전달)으로 전환했다.

▶ **내 판단/제안:** 팀장 피드백("내부 작업용 정보를 고객향 문구에 노출하지 말 것")을 반영해, 프롬프트를 [A] 내부 참고용 힌트(노출 금지)와 [B] 실제 반영 정보(메뉴명·게시목적)로 명확히 분리하는 설계를 만들었다. 또한 "이미지만 먼저 보고 싶다", "캡션만 다시 만들고 싶다"는 요구를 반영해 /generate 와 /caption 을 별도 엔드포인트로 분리했다.

오후에는 서비스를 실제로 배포했다. 팀 GitHub 계정(ysy-works)이 개인 계정이라 외부 배포 서비스(Render, Vercel)의 GitHub 앱 설치 승인 권한이 없다는 걸 확인하고, Docker Hub 에 이미지를 올려 Render 에 "기존 이미지로 배포"하는 방식과 Vercel CLI 직접 배포로 각각 우회했다. 이 권한 문제는 이후에도(Cloudflare Pages, 7/10) 반복돼, 개인 GitHub 저장소를 별도로 두고 두 원격 저장소에 동시에 push 하는 방식을 표준 운영 절차로 정착시켰다.(팀장님 제안)

## 7.2.4 Phase 4 · 7/10, 7/15 — 배포 안정화와 실제 모델(Flux) 연동

7/10 에는 Vercel 을 Cloudflare Pages 로 이전했다(휴면 없는 정적 호스팅이라는 장점). 레퍼런스 이미지·JSON 을 백엔드(Render)가 아니라 프론트와 함께 CDN 에 정적 배포하도록 바꿔, Render 콜드스타트가 레퍼런스 로딩에 영향을 주지 않게 만들었다 — "백엔드는 진짜 연산이 필요한 것만 담당한다"는 원칙을 실제로 적용한 사례다. 또한 Vivid 무드 레퍼런스 이미지를 Gemini 로 재생성하며, "느낌"이 아니라 프레임 대비 비율·각도 같은 수치로 프롬프트를 구체화해야 결과가 일관됨을 다시 확인했다.

7/15 에는 모델 제작 담당(김기현)이 GCP 에 띄운 Flux 모델 API 를 실제로 연동했다. 연동 도중 serving 브랜치가 단독으로는 아예 실행조차 안 되는 상태(핵심 파일 다수 누락)임을 뒤늦게 발견해 복구했고, 모델 서버 응답 시간이 예상보다 훨씬 길어(최대 5 분) 타임아웃 값을 180 초 → 300 초 → 420 초로 두 차례 재조정했다.

## 7.2.5 Phase 5 · 7/16 — 브랜드 확정, 랜딩페이지 신설, 다국어 지원

임시명 "moodcut"을 팀에서 "Picmood"로 공식 확정하며 코드 전반의 브랜드명을 일괄 교체했다. 또한 접속하자마자 도구 화면이 뜨던 구조를, 광고·안내 역할의 홈 화면과 "시작하기"를 눌러야 들어가는 도구 화면으로 분리해 랜딩 페이지(Home.jsx)를 처음부터 설계·제작했다.

▶ **내 판단/제안:** react-router 대신 상태값 하나로 화면을 전환하는 방식을 선택했다. Cloudflare Pages 에 별도 SPA 라우팅 설정이 필요 없어 지금 규모에 더 간단하다고 판단했기 때문이다. 히어로 배경도 "다크+반투명 패널"에서 "밝은 크림톤"으로 전면 전환했는데, 반투명 패널로 가독성 문제를 덮기보다 배경 자체를 바꿔 대비 문제가 구조적으로 생기지 않게 하는 편이 낫다고 판단한 결정이었다.

"도구 화면으로 넘어가면 외국인이 이용을 못 한다"는 생각이 들, 홈 화면에만 있던 언어 토글을 도구 화면까지 확장했다. 다만 GPT 가 실제로 생성하는 캡션은 이 시점엔 여전히 한국어 고정이라는 점을 "UI 다국어화와 산출물 다국어화는 다른 스코프의 작업"이라고 명확히 구분해 팀에 공유했다.

## 7.2.6 Phase 6 · 7/20 — ComfyUI Gateway 정식 연동

팀장님(유소연)이 실제로 완성한 ComfyUI Gateway 스펙(BACKEND\_HANDOFF.md)을 받아, 모델 서버를 동기 직접 호출하던 구조를 제출(202 Accepted) → 상태 폴링 → 결과 다운로드하는 비동기 구조로 전면 교체하고 실배포까지 마쳤다.

▶ **내 판단/제안:** 7/16 에 미리 만들어둔 WORKFLOW\_REGISTRY(워크플로별 핸들러 등록) 구조 덕분에, 실제 스펙이 왔을 때 함수 내부 구현만 교체하면 됐다. "스펙이 확정되기 전에 인터페이스만 먼저 고정해두는 설계"가 실제로 시간을 아껴준 것을 이날 체감했다. 기존 GCP 직접 호출 코드는 삭제하지 않고 legacy 함수로 보존해 롤백 가능성을 열어뒀다.

팀장님의 디자인 피드백(레퍼런스 그리드 지그재그 배치, 카드 테두리 제거, 헤드 문구 순화)을 반영했고, "다크모드에서 고딕체가 얇아 보인다"는 버그의 실제 원인(웹폰트 파일명과 CSS font-family 표기 불일치)을 추적해 근본적으로 해결했다.

## 7.2.7 Phase 7 · 7/21 — 다국어 완성과 발표 템플릿 제작

스타일 선택 화면의 카드 캡션·좋아요·선택됨 배지가 영문 모드에서도 한글로 고정되던 버그를 고치고, 캡션 생성(GPT-4o-mini) 프롬프트를 한/영 두 버전으로 분리해 실제 산출물까지 다국어화를 완료했다. 팀장 요청으로

촬영 가이드(i 아이콘 툴팁) 기능을 신규 추가했는데, 이 과정에서 겪은 4 단계 트러블슈팅은 이 기간 동안 가장 깊어 파고든 버그였다.

오후에는 팀 발표자료 Google Slides 템플릿을 직접 설계·제작했다. 사이트 색상·폰트 톤을 그대로 가져와 브랜드 일관성을 유지했고, 레이아웃 옵션 7 종(사진 2/3 개, Before→After+프롬프트, 코드 스니펫+결과 등)을 팀원별 결과물 형태(스크린샷/코드/생성이미지+프롬프트)에 맞춰 따로 설계했다. 실제로 내 파트 협업일지로 예시 슬라이드를 채워보며 "텍스트 카드 폭이 실제 문장 길이엔 좁다" 같은 문제를 발견해 템플릿 자체를 수정했다.

## 7.2.8 Phase 8 · 7/23 — OpenAI 신규 워크플로 연동과 결과 비율 선택

소연(팀장)님이 공유한 ComfyUI 재설계 문서를 검토해, gpt-image-2 기반 신규 프리셋 2 개(화이트·우드 미디엄샷)를 백엔드에 연결했다. reference\_id 를 Gateway 의 preset\_id 로 그대로 전달하는 구조, 활성 프리셋 화이트리스트, workflow 별 크롭 분기(신규 workflow 는 무크롭)를 설계·구현했다.

▶ **내 판단/제안:** 기본 workflow 를 코드 상수가 아니라 COMFYUI\_WORKFLOW\_ID 환경변수로 선택하도록 만들어, 실제 전환 시점에는 배포 환경변수 하나만 바꾸면 되는 구조로 미리 준비해뒀다. 이 설계 덕분에 이후 실제 연동 시점에 코드 수정 없이 스위치만 바꿔 전환할 수 있었다.

오후에 소연님으로부터 스펙 정정(무크롭, 4:5=1024×1280 / 1:1=1024×1024, 비율 필수 선택)을 받아, 처음엔 "준비 중"으로 막아뒀던 1:1 옵션을 실제로 열고 사용자가 반드시 하나를 선택해야 생성되도록 가드를 추가했다. 이후 배포 단계에서 실제로 발생한 문제 세 가지 (model-c-v1 500 에러, Gateway workflow 미등록, 캡션 API 키 오류)를 발견했고 소연님께 알려드려 해결했다.

## 7.2.9 Phase 9 · 7/27 — 프리셋 12 개 완성과 컵 선택 토글

소연님이 순차적으로 검증·배포한 프리셋(뉴트럴 화이트,우드 7 개 → 다크그레이 4 개 + 우드 나머지 1 개)을 받아, 최종적으로 12 개 전부를 최종 workflow(gpt-image-2-v1)에 연결했다. 회의에서 나온 '업로드한 컵/모델이 준비한 컵을 선택하는 토글'을 새로 설계했다.

▶ **내 판단/제안:** 이 토글의 배치를 고민할 때, "사용자가 자기 사진을 실제로 본 다음에야 의미가 생기는 선택"이라는 점에 착안해 업로드 → 미리보기 이후, 생성 버튼 직전에 두기로 결정했다. 백엔드 연동 기준이 아직 확정되지 않은 부분(cup\_source 파라미터명, 스타일별 지원 여부)은 검증 로직 없이 값만 전달하도록 열어두고, 연동 기준이 확정되면 기존 활성 프리셋 화이트리스트와 같은 패턴으로 검증을 추가하기로 구조를 잡아뒀다.

이날 references.json 이 백엔드(app/config)와 프론트(frontend/public) 두 곳에 이중으로 존재하는 구조 때문에, 활성화 작업 때마다 한쪽만 갱신하고 넘어가는 실수가 재발했음을 인지하고 앞으로 두 파일을 세트로 확인하는 체크리스트가 필요하다는 점을 스스로 짚어뒀다.

## 7.2.10 Phase 10 · 7/27(오후) — Langfuse 세션 추적 연동과 레퍼런스 이미지 실제 교체

소연님이 API 비용·모델 호출 시간·Gateway 소요 시간 등 런타임 지표를 관리하려고 Langfuse 를 GCP 에 연동하면서, 요청을 세션 단위로 묶어볼 수 있도록 프론트·백엔드에 익명 세션 ID(X-Session-ID) 전달 기능을 요청받아 구현했다. 프론트는 `crypto.randomUUID()`로 만든 세션 ID 를 `localStorage` 에 저장해 재사용하고 `/generate` 요청에만 헤더로 실어 보내며, 백엔드는 이 값을 선택값으로 받아 기존 `Authorization·Idempotency-Key` 로직은 그대로 둔 채 Gateway 요청에만 조건부로 추가하는 공통 헬퍼를 만들었다. 이메일·닉네임 등 사용자를 특정할 수 있는 값은 쓰지 않았고, 로그에도 남기지 않았다.

▶ **내 판단/제안:** `model-c-v1` 과 `gpt-image-2` 두 워크플로 핸들러가 둘 다 같은 Gateway 를 호출한다는 점에 착안해, 헤더 생성 로직을 공통 헬퍼 함수 하나로 분리해 두 핸들러 모두에 적용했다. 이렇게 하면 이후 워크플로가 추가돼도 같은 함수만 재사용하면 되는 구조가 된다.

이어서 그동안 화이트·우드는 Gemini 로 만든 임시 이미지였고 다크그레이는 명칭만 바뀌고 사진은 옛 Vivid 그대로였던 레퍼런스 이미지 12 장을, 모델 제작팀이 실제 생성한 결과로 전부 교체했다. `mood_id` 와 파일명은 그대로 유지해(7/21 에 세운 설계 원칙 그대로) 코드 수정 없이 이미지 내용만 바꾸는 방식으로 진행했다. 이 과정에서 탐색기의 확장자 숨김 설정 때문에 파일명이 이중 확장자(`.png.png`, `.png.jpg`)로 꼬이는 문제가 반복돼, 실제 파일명을 터미널에서 직접 확인해 바로잡았다.

홈 화면의 "Before & after" 섹션도 채웠다. 7/16 랜딩 페이지를 처음 만들 때부터 "촬영 그대로" 쪽이 빈 자리로 남아있던 부분인데, "같은 사진, 완전히 다른 무드"라는 카피에 맞춰 실제로 짝이 맞는 원본→결과 사진 페어로 고정했다. 두 사진이 붙어 한 장처럼 보이던 것을 8px 간격과 개별 둥근 모서리로 분리하고, 반투명이던 배지를 단색으로 바꿔 가독성을 높였다.

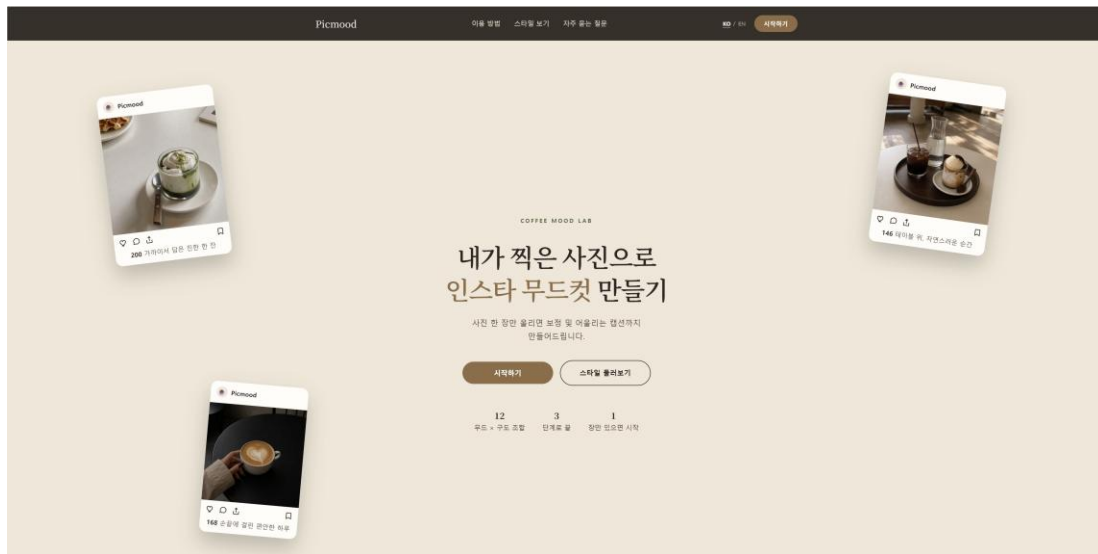
▶ **내 판단/제안:** "같은 사진"이라는 카피가 있는 만큼, 갤러리의 임의 이미지를 가져다 쓰던 기존 방식 대신 실제로 원본과 결과가 짝지어진 사진 쌍을 명시적으로 고정하는 쪽을 선택했다. 마케팅 카피와 실제 화면이 어긋나지 않도록 하기 위한 판단이었다.

배포 직전 로컬에서는 새 이미지가 잘 보이는데 실제 배포 사이트에는 반영이 안 되는 문제를 만났다. 배포 자체는 성공 상태였고 배포 전용 URL 로도 예전 이미지가 떠서 캐시 문제는 아니라고 판단했고, `git log` 로 추적한 끝에 이미지 교체 작업이 실제로는 한 번도 커밋된 적이 없었다는 걸 확인했다. 로컬 Vite 서버는 파일시스템을 직접 읽어 화면엔 보였지만, Cloudflare Pages 는 `git` 저장소 기준으로 빌드하기 때문에 커밋 안 된 변경은 배포에 반영되지 않았던 것이다. `local-dev` 에서 12 개 파일을 재커밋하고 `deploy` 에 병합·`push` 해 해결했고, 이후 `frontend` 브랜치에도 같은 커밋을 동기화해 브랜치 간 불일치를 정리했다.

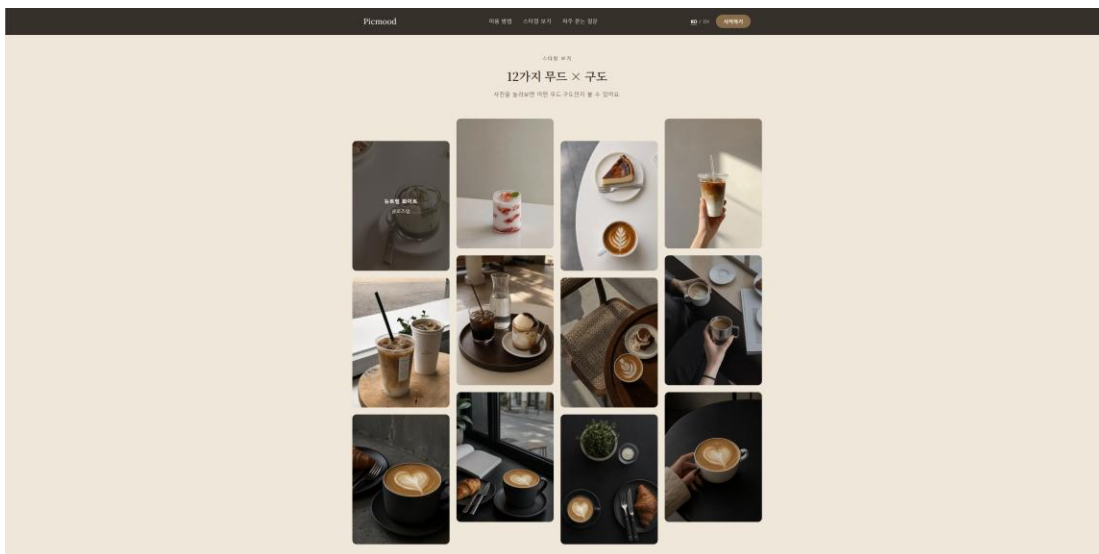
## 7.3 결과물

실제 서비스 화면과 파이프라인 진행 과정을 스크린샷으로 정리한다.

### 7.3.1 서비스 화면



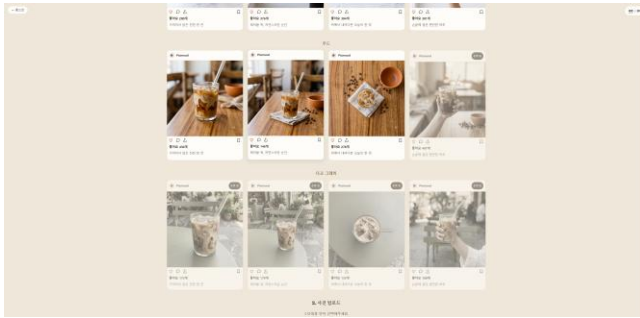
랜딩 페이지 히어로 (한국어)



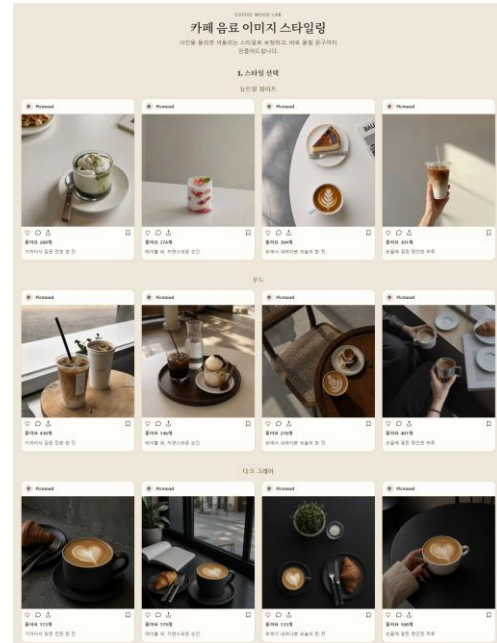
12 무드×구도 갤러리 (영어)

### 7.3.2 스타일 선택 화면 — 프리셋 활성화 진행 과정

12 개 프리셋이 소연의 검증·배포 순서에 맞춰 순차적으로 열렸다. 아래는 일부만 활성화됐던 중간 상태와, 12 개 전부가 열린 최종 상태를 나란히 비교한 것이다.

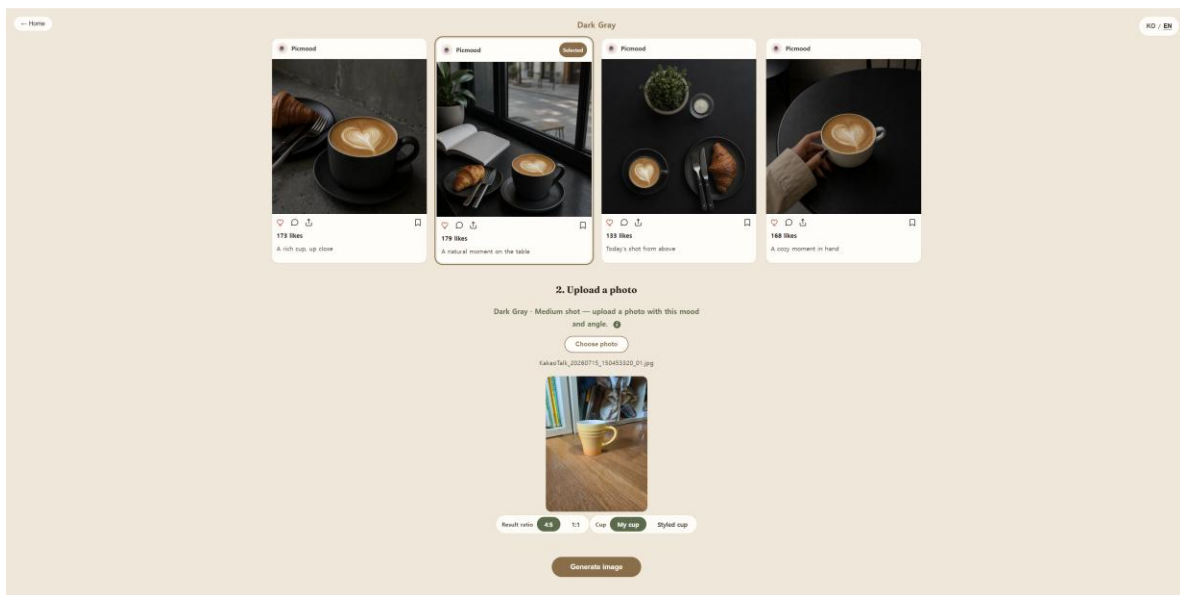


진행 중 — 우드·다크그레이 일부 "준비 중"



완료 — 12 개 전부 활성화

### 7.3.3 사진 업로드 화면 — 촬영 가이드 · 비율/컵 선택 토글



촬영 가이드 툴팁 + 4:5/1:1 결과 비율 + 컵 선택 토글 (영어)

### 7.3.4 생성 결과 — Before / After



After — Picmood 로 재보정된 결과

### 7.3.5 캡션 생성 결과 (다국어)

A screenshot of the Picmood app interface in Korean. It shows a form for generating a caption and hashtags. The form includes fields for 'Caption' (캡션) and 'Hashtags' (해시태그), and a 'Generate' (생성) button. Below the form, there is a section for 'Story text' (스토리 문구) with a 'Generate' button.

캡션·해시태그·스토리 문구 (한국어)

A screenshot of the Picmood app interface in English. It shows a form for generating a caption and hashtags. The form includes fields for 'Caption' (Caption) and 'Hashtags' (Hashtags), and a 'Generate' button. Below the form, there is a section for 'Story text' (Story text) with a 'Generate' button.

캡션·해시태그·스토리 문구 (영어)

## 7.4 주요 의사결정 및 회의 제안 총정리

---

위 타임라인 곳곳에 나온 결정·제안들을 한 곳에 모았다. 대부분 "지금 당장 필요한 것만 간단히"가 아니라 "다음 변경이 왔을 때 코드를 덜 건드리게"를 기준으로 판단했다는 공통점이 있다.

**7/1** 모델 없이도 서비스 전체 구조를 먼저 완성 — 모델 함수만 더미로 비워두는 구조 설계

**7/2** Streamlit → React+react-konva 전환 — 연속 드래그 인터랙션은 Streamlit 구조로 불가능하다고 판단

**7/6** 무드·구도 데이터를 코드가 아닌 references.json 으로 분리 설계 — 이후 실제로 구조가 3 번 바뀌어도 코드 무변경으로 대응

**7/7** 형용사 위주 프롬프트를 수치 기준(비율%, 각도)으로 다시 쓸 것을 설아님에 제안

**7/8** 인스타그램풍 UI 를 저작권 문제 없는 범용 소셜 패턴으로 직접 설계

**7/9** /generate·/caption 엔드포인트 분리 — 이미지 재생성 없이 캡션만 다시 만들 수 있도록

**7/9** 팀 GitHub 권한 문제를 Docker Hub·Vercel CLI 우회 배포로 직접 해결 방안 제시·실행

**7/10** 레퍼런스 정적 자원을 백엔드가 아닌 프론트+CDN 으로 직접 서빙하도록 구조 변경 제안

**7/16** WORKFLOW\_REGISTRY 패턴으로 모델 서빙 경로를 미리 추상화 — 이후 두 차례(ComfyUI, gpt-image-2) 실제 전환에서 이 구조가 그대로 재사용됨

**7/16** 홈 화면 신설 시 react-router 대신 상태값 전환 방식 채택 — 배포 환경(Cloudflare Pages)의 라우팅 제약을 고려한 결정

**7/21** 발표자료 템플릿을 팀원 역할별(스크린샷/코드/생성이미지+프롬프트) 전용 레이아웃으로 설계 제안

**7/23** 기본 workflow 를 환경변수로 분기하도록 설계 — 실제 전환 시점에 코드 수정 없이 스위치만 변경 가능하게 함

**7/27** 컵 선택 토글의 배치를 "사용자가 결과를 실제로 인지한 뒤 판단하는 선택"이라는 기준으로 업로드 미리보기 이후로 배치 결정, Gateway 헤더 생성을 공통 헬퍼로 분리해 두 워크플로 핸들러에 동일 적용 — 이후 워크플로가 늘어나도 재사용 가능한 구조로 설계, 홈 화면 Before/After 데모를 임의 이미지 대신 실제 원본-결과 짝으로 고정 — 마케팅 카피("같은 사진")와 화면 내용의 일치를 우선

## 7.5 오류 및 트러블슈팅 총정리

---

이 프로젝트 기간 동안 겪은 오류들을 성격별로 묶어 정리했다. 단순 오타 수준의 문제는 제외하고, 원인 진단에 시간이 들었거나 구조적인 교훈을 남긴 것 위주로 기록한다.

### 7.5.1 Git / 브랜치 운영

#### ■ **ModuleNotFoundError — 특정 브랜치 단독 실행 불가 (7/2, 7/9, 7/15 세 차례 반복)**

원인 역할별 브랜치(frontend/backend/serving)에 각자 담당 파일만 커밋해와서, 특정 브랜치 하나만으로는 서비스가 실행되지 않는 상태였음. 7/15에는 serving 브랜치에 app/main.py, router.py, references.json 등 핵심 파일이 애초에 커밋된 적이 없었다는 걸 뒤늦게 발견.

**해결** 세 역할 브랜치를 모두 합치는 local-dev 통합 테스트용 브랜치를 도입. 이후 배포·Docker 처럼 "전체 코드가 다 필요한" 작업은 항상 local-dev 기준으로 진행하는 규칙을 세움.

#### ■ **local-dev 에서 작업한 커밋이 frontend 브랜치에 반영 안 됨 (7/6)**

원인 레이아웃 수정을 frontend 가 아니라 local-dev 브랜치에 있는 상태에서 진행하고 커밋함. 이 상태로 git push origin frontend 를 실행해 'Everything up-to-date'만 뜨고 실제 반영 안 됨.

**해결** git cherry-pick 으로 해당 커밋만 frontend 브랜치에 옮겨 담아 정상 반영.

#### ■ **커밋 안 된 변경사항 때문에 브랜치 전환 실패/오염 (여러 차례: 7/6, 7/10, 7/16)**

원인 코드를 수정만 하고 커밋하지 않은 채 다른 브랜치로 checkout 을 시도하거나, deploy 브랜치에 그대로 있는 채로 다음 작업을 시작함.

**해결** 작업 시작 전 git status 로 현재 브랜치·변경사항부터 확인하는 것을 세션 시작 시 표준 절차로 고정. git stash 로 임시 보관 후 브랜치 이동하는 방식도 병행.

#### ■ **references.json 이중 관리 구조에서 한쪽만 갱신 (7/27)**

원인 백엔드용 app/config/references.json 만 갱신하고, 프론트가 실제로 읽는 frontend/public/references.json 은 그대로 둔 채 배포 지시를 내림.

**해결** frontend 브랜치에서 정적 사본도 동일하게 갱신 후 local-dev 재병합. 이후 두 파일을 세트로 확인하는 체크리스트 필요성을 인지.

#### ■ **이미지 교체 커밋 누락으로 로컬-배포 불일치 (7/27)**

원인 레퍼런스 이미지 12 장 교체 작업이 로컬 파일시스템에만 반영되고 실제로는 커밋되지 않음. 로컬 Vite 서버는 파일을 직접 읽어 정상으로 보였지만, Cloudflare Pages 는 git 저장소 기준으로 빌드해 반영되지 않았음.

**해결** git status 로 미커밋 상태 확인 → local-dev 에서 재커밋 → deploy 병합·push 로 정상 배포. 이후 frontend 브랜치에도 동일 커밋을 동기화해 브랜치 간 불일치 재발을 막음.

### 7.5.2 배포 · 인프라

#### ■ **팀 GitHub 계정 권한 문제로 배포 서비스 연동 불가 (7/9 Render·Vercel, 7/10 Cloudflare Pages)**

원인 저장소 소유 계정(ysy-works)이 조직이 아닌 개인 계정이라, 협업자 권한만으로는 외부 앱 설치 승인이 불가능함. 세 번의 서로 다른 배포 서비스에서 동일한 문제가 반복됨.

**해결** Docker Hub 에 이미지를 올려 "기존 이미지로 배포" 방식(Render), Vercel CLI 직접 배포로 각각 우회. 이후 개인 GitHub 저장소를 별도로 두고 팀 저장소(origin)·개인 저장소(personal) 양쪽에 동시 push 하는 방식을 표준 운영 절차로 정착.

### ■ ReadTimeout — 모델 서버 응답이 설정 시간을 초과 (7/15, 2 회)

**원인** 로컬 180 초 → ReadTimeout, Render 배포 환경 300 초로도 재발. Render 로그 확인 결과 Render 인프라 자체의 제한이 아니라, 설정한 300 초를 다 채운 뒤 우리 쪽에서 포기한 것이었고 실제 원인은 모델 서버의 이미지 생성이 5 분을 넘긴 것.

**해결** 타임아웃을 300 초 → 420 초로 재상향. "배포 환경의 타임아웃이 반드시 그 플랫폼 자체의 제한은 아닐 수 있다"는 걸 로그로 직접 구분해서 확인.

### ■ Docker 컨테이너 실행 시 ModuleNotFoundError: app.model.model (7/9)

**원인** Dockerfile·requirements.txt 를 backend 브랜치에서 작업했는데, model.py 는 serving 브랜치 소유라 backend 단독 브랜치엔 존재하지 않았음.

**해결** local-dev 로 이동해 재빌드 → 정상 작동. 이후 Docker/배포 관련 작업은 항상 local-dev 기준으로 진행.

### ■ model-c-v1 경로 500 Internal Server Error (7/23)

**원인** COMFYUI\_WORKFLOW\_ID 를 아직 전환하기 전이라 기존 model-c-v1 경로로 요청이 갔는데, 그 경로를 처리하는 서버 쪽에서 에러가 남 — 내 코드와 무관.

**해결** 에러 태그([MODEL\_C\_HTTP\_500])로 우리 백엔드가 아닌 하위 서버 문제임을 바로 판별해 소연님에게 전달. GCP 런타임 배포 완료 후 자연 해소.

### ■ Gateway "Unknown workflow\_id" 400 에러 (7/23)

**원인** 신규 workflow 가 코드상으로는 완성돼 있었지만, 실제 운영 Gateway 서버에는 아직 등록·배포 되지 않은 상태였음(기존 검증은 Gateway 를 거치지 않은 adapter 직접 호출이었음).

**해결** 에러 메시지가 영어·JSON 형식인 것으로 우리 백엔드 에러가 아니라 Gateway 가 직접 받은 응답임을 판별. 소연님이 API 키 주입, Health Check 기준 전환 후 정상 등록 확인.

### ■ 캡션 생성 401 Incorrect API key (7/23)

**원인** 이미지 생성(gpt-image-2)용으로 재발급된 sk-svcacct- 키가, 캡션 생성(GPT-4o-mini)용 OPENAI\_API\_KEY 환경변수 자리에 잘못 들어감. 로컬 .env 에서도 동일 문제 재발.

**해결** 캡션 생성용 정상 키로 값 재교체. 키 용도가 다르면 재발급 시 어디에 영향이 가는지 미리 확인해야 한다는 것을 깨달음.

## 7.5.3 UI / 레이아웃

### ■ PC 화면에서 레퍼런스 갤러리가 왼쪽으로 쏠림 (7/8)

**원인** body margin·flex 를 먼저 의심했으나 반영 후에도 재현. 실제 원인은 CSS Grid 의 grid-template-columns: repeat(auto-fill, ...) — 아이템이 4 개뿐인데 auto-fill 이 컨테이너 폭만큼 빈 트랙까지 미리 만들어 나머지 공간을 차지함.

**해결** auto-fill 을 auto-fit 으로 변경 — 빈 트랙을 접어 없애고 1fr 을 실제 아이템에만 재분배.

### ■ 모바일에서 사이트 접속 자체가 안 됨 (7/8, 4 단계)

**원인** ① localhost 는 기기 자기 자신만 가리키는 주소라 휴대폰에서 접속 불가 → ② App.jsx 에 localhost:8000 이 하드코딩돼 있어 API 요청이 실패 → ③ CORS 가 정확히 localhost:5173 만 허용해 사설망 IP 를 차단 → ④ 위 3 가지를 다 고쳐도 실패해, 공유기의 AP 격리(client isolation) 기능이 기기 간 통신을 차단하는 것으로 최종 확인.

**해결** ①②③은 각각 --host 0.0.0.0 바인딩, API\_BASE 를 window.location.hostname 기반 자동분기, allow\_origin\_regex 확장으로 해결. ④는 휴대폰 핫스팟으로 재시도해 원인을 최종 확정.

### ■ 다크모드에서 텍스트가 흰 글씨로 거의 안 보임 (7/16, 7/20 두 차례 유사 재발)

**원인** Vite 기본 index.css 의 h1,h2 { color: var(--text-h) } 규칙이 다크모드에서 거의 흰색으로 바뀌는데, 새로 만든 헤딩·타이틀에 색을 명시적으로 지정하지 않아 이 규칙을 그대로 상속받음. 7/20 엔 폰트 파일명(PretendardVariable)과 CSS font-family 표기(Pretendard) 불일치가 겹쳐 시스템 기본 글꼴로 계속 대체되고 있었던 것이 원인.

**해결** 모든 헤딩에 color 명시적 고정. 폰트명을 정확히 맞추고 -webkit-font-smoothing 등 렌더링 안정화 속성 추가. "과거에 겪은 버그 유형은 새 코드에서도 재발할 수 있다"는 걸 실감.

### ■ 한글 텍스트가 좁은 화면에서 단어 중간에 줄바꿈됨 (7/16)

**원인** 한글은 브라우저 기본값이 글자 단위 줄바꿈이라, 프로세스 제목·업로드 안내·카드 캡션 등 여러 곳에서 동시다발적으로 발생.

**해결** word-break: keep-all 을 두 파일의 최상위 컨테이너에 한 번씩 적용해 전체 일괄 해결 — 위치별로 대응하는 대신 전역 규칙 하나로 예방.

## 7.5.4 촬영 가이드 툴팁 — 4 단계에 걸친 위치 버그 (7/21)

이 기간 중 원인 추적이 가장 깊었던 버그다. 같은 증상(툴팁이 이상한 곳에 뜬다)이 수정할 때마다 다른 원인으로 재발해, 네 번에 걸쳐 접근 방식 자체를 바꿔야 했다.

### ■ 1 차 — 화면 밖으로 넘어감

**원인** 아이콘 중앙 기준 정렬(translateX(-50%))만 사용해, 아이콘이 화면 오른쪽 가장자리에 가까우면 툴팁 절반이 화면 밖으로 나감.

**해결** hover/focus 시점에 위치를 측정해 화면 여백을 넘지 않도록 보정하는 로직 추가.

### ■ 2 차 — 위치가 열 때마다 들쭉날쭉

**원인** 위치 계산이 '상대 보정값(offset)'을 누적하는 방식이라 이전 상태가 다음 계산에 영향을 줌.

**해결** open 을 React state 로 관리하고, 열릴 때마다 아이콘의 현재 위치만 새로 측정해 뷰포트 기준 절대좌표(position: fixed)로 직접 계산하도록 재작성.

### ■ 3 차 — 스크롤하면 화면에 고정되어 아이콘을 안 따라옴

**원인** position: fixed 는 뷰포트 기준으로 고정되는 성질이라, 스크롤해도 같은 화면 자리에 남아있음.

**해결** position: absolute 로 바꾸고 React Portal 로 document.body 에 렌더링, 좌표에 현재 스크롤 오프셋을 더해 '페이지 기준' 좌표로 계산 — 스크롤과 함께 자연스럽게 이동.

### ■ 4 차 — 포탈 적용 후 툴팁이 투명해지고 색이 바뀜

**원인** 배경/글자색을 var(--ink), var(--surface) 같은 CSS 변수로 지정했는데, 이 변수들이 .cafe-app 요소 안에서만 정의돼 있었음. 포탈로 body 에 렌더링되며 .cafe-app 밖으로 나가 변수를 상속받지 못함.

**해결** 실제 색상값(#2B2420, #FFFDF9)을 직접 하드코딩. 겉보기엔 사소한 UI 버그도 모바일 이벤트 특성·CSS 포지셔닝 기준·CSS 변수 상속 스코프까지 추적해야 근본적으로 해결된다는 걸 확인.

## 7.5.5 자산(이미지) 파일 관리 (7/27)

### ■ 파일명 이중 확장자 (.png.png)

원인 탐색기가 실제 확장자를 숨기고 있어, 이미 .png 로 끝나는 파일에 다시 .png 를 붙임.

해결 뒤쪽 진짜 확장자만 제거해 파일명을 원래 의도대로 정리.

### ■ jpg 파일이 webp 변환 스크립트에서 인식 안 됨

원인 convert\_to\_webp.py 가 \*.png 패턴만 찾는데, 파일명이 .png.jpg 로 되어 있어 스크립트가 대상에서 제외함.

해결 Rename-Item 으로 .jpg 만 제거해 .png 로 끝나게 맞춘 뒤 스크립트 재실행, 7 개 정상 변환 확인.

## 7.6 회고

---

- 모델 없이도 서비스 전체 구조를 먼저 완성할 수 있다. 입출력 형식만 팀원과 합의해두면 각자 독립적으로 작업이 가능하다는 걸 프로젝트 시작 시점부터 끝까지 반복적으로 확인했다.
- 데이터를 코드에 하드코딩하지 않고 별도 파일(references.json)로 분리해두면, 서비스 방향이 완전히 바뀌어도 이후 변경에 훨씬 유연하게 대응할 수 있다 — 실제로 무드·구도 구조가 세 차례나 바뀌었지만 코드 자체는 거의 안 바뀌었다.
- 스펙이 확정되기 전에 인터페이스(WORKFLOW\_REGISTRY, 환경변수 기반 분기)만 먼저 고정해두는 설계는, 실제 연동 시점의 작업량을 눈에 띄게 줄여줬다. 이 패턴은 로컬 모델→ComfyUI Gateway→OpenAI 신규 워크플로까지 세 번의 전환에서 전부 재사용됐다.
- 여러 터미널·브랜치를 오가며 작업할 때는 '지금 내가 어느 브랜치에 있는지' 확인하는 습관이 생각보다 큰 시간을 아껴준다. 이 확인을 건너뛴 순간마다 예외 없이 문제가 생겼다.
- 겉보기엔 사소해 보이는 UI 버그(다크모드 폰트, 툴팁 위치)일수록, 증상만 덮지 않고 실제 원인(파일명 불일치, CSS 상속 스코프, 이벤트 특성)까지 추적해야 재발을 막을 수 있다는 걸 이번 프로젝트에서 가장 여러 번 체감했다.
- 협업자 권한과 배포 플랫폼 연동 권한은 별개라는 것을 세 번(Render, Vercel, Cloudflare) 겪었다. 팀 공용 계정으로 저장소를 운영할 때는 이 권한 구조를 미리 확인해두는 것이 필요하다.
- 로컬 개발 서버는 파일시스템을 직접 읽고 배포는 git 커밋 기준으로 빌드한다는 차이 때문에, "로컬에서 잘 보인다"가 곧 "커밋됐다"를 보장하지 않는다는 걸 직접 겪었다. 자산 파일을 바꿀 때는 항상 git status 로 커밋 여부까지 확인하는 습관이 필요하다.
- 탐색기의 확장자 숨기기 설정 때문에 파일명 변경 작업에서 예상과 다른 이름이 되기 쉽다. 헛갈릴 땐 탐색기 대신 터미널로 실제 파일명을 직접 확인하는 게 더 정확했다.
- Langfuse 같은 외부 모니터링 툴도 GitHub 나 배포 서비스처럼 이메일 초대 기반으로 팀원 접근 권한을 나눠 관리한다는 공통 패턴을 다시 확인했다.

## 8. 현재 한계와 개선 과제

### 8.1 프리셋 제작의 수작업 의존

현재 프리셋은 레퍼런스를 수집하고, 장면을 분석하고, 프롬프트와 조명 시트를 작성한 뒤 사람이 결과를 반복 검수하는 방식으로 제작한다.

#### 8.1.1 한계

- 프리셋 1 개 제작에 많은 시간 소요
- 작업자의 판단과 경험에 따른 차이
- 수정할 때 프롬프트·힌트·구도 범위를 함께 검토해야 함
- 무드와 제품 카테고리가 늘어날수록 관리 비용 증가

#### 8.1.2 개선 방향

- 레퍼런스 자동 분석
- 카메라·빛·색감·제품 위치 자동 추출
- 프리셋 초안 자동 작성
- 실패 원인 분류와 수정 이력 데이터화
- 프리셋 승격 절차 자동화

### 8.2 자동 품질 평가의 미완성

현재 프리셋에는 `qa_criteria` 와 bbox 범위가 정의돼 있지만, 모든 결과를 자동 차단하는 공용 QA 는 아직 적용하지 않았다.

#### 8.2.1 후속 설계

- 제품 영역과 잘림 확인

- OCR 을 통한 핵심 문자 검증
- 제품 정체성 비교
- 피사체 수와 bbox 확인
- 자연스러움과 무드 일치도 비교
- 초기에는 shadow mode 로 기록만 수행
- 충분한 표본을 확보한 뒤 자동 차단 기준 적용

## 8.3 단일 Gateway 와 임시 저장소

현재 Gateway 는 worker 1 개와 SQLite 를 사용한다. MVP 와 팀 프로젝트 규모에서는 단순하고 관리하기 쉽지만, 사용자가 늘어날 경우 다음 개선이 필요하다.

- Redis 또는 관리형 큐 도입
- 분산 락과 worker 확장
- 작업 상태 DB 의 관리형 저장소 전환
- 결과 이미지의 GCS 영구 저장
- 장애 복구와 만료 정책 개선

## 8.4 브랜드 로고와 편집 기능

현재 별도의 로고 파일을 입력해 컵이나 포장지에 적용하는 기능은 비활성화돼 있다. 또한 생성 이후 소품 위치나 배경 일부를 직접 수정하는 기능도 제공하지 않는다.

### 8.4.1 확장 기능

- 사용자 로고 업로드와 적용 위치 선택
- 배경·제품·소품 분리
- 소품 위치 이동
- 일부 영역만 다시 생성

- 색상과 밝기 조정
- 레이어 단위 편집

## 8.5 비용과 품질의 지속적인 관리

기본 품질은 Medium 으로 설정했지만, 제품 유형과 프리셋에 따라 품질 차이가 다를 수 있다. 모든 요청을 High 로 올리기보다 실제 로그를 바탕으로 비용 대비 효과를 확인해야 한다.

### 8.5.1 관리 항목

- 프리셋별 평균 비용
- 컵 모드별 비용 차이
- 입력 이미지 크기와 usage 의 관계
- Low·Medium·High 품질 비교
- 재생성률을 포함한 실질 비용
- 프론트엔드·백엔드 익명 세션 전달 완료
- 중간 점검 DB 접근 오류 재검증
- canary 와 실제 완료 작업 1 건의 trace 전송 확인
- Langfuse 대시보드 기반 프리셋·품질·비용·모델 생성 시간·사용자 체감 완료 시간 추적

## 8.6 검증 범위의 한계

12 개 프리셋 × 2 개 컵 모드 × 3 개 품질의 72 개 유료 생성 조합은 모두 검증했지만, 이는 가능한 모든 음료·컵·촬영 상태에서 동일한 성공률을 보장한다는 뜻은 아니다. 프리셋별 성공률, 제품 정체성 유지율, 품질별 재생성률은 입력 데이터와 운영 표본을 더 축적해 검증해야 한다.

- 프리셋별 실패가 자주 발생하는 제품 유형 수집
- 4:5:1:1 과 두 컵 모드의 조합별 성공률 집계
- 프롬프트 변경 전후의 동일 입력 회귀 비교

- Low·Medium·High 품질별 비용·재생산률의 운영 데이터 기반 지속 비교
- 사람의 재생성 사유를 정형화해 프리셋 개선에 환류

현재 단계에서는 12 개 프리셋×2 개 컵 모드×3 개 품질의 72 개 유료 생성 조합을 전수 검증하고 서비스 가능한 공통 설계 규칙을 완성했다. 다만 실제 사용자 입력 전반에 대한 통계적 품질 보장은 운영 데이터가 쌓인 이후의 과제이다.

## 8.7 서비스 확장 방향

### 8.7.1 제품 카테고리 확장

현재는 음료에 집중했지만 같은 구조를 다음 카테고리로 확장할 수 있다.

- 디저트와 베이커리
- 음식
- 소품과 잡화
- 의류
- 포장 제품
- 가구와 인테리어 제품

카테고리가 달라지면 제품의 재질, 형태, 촬영 거리, 배경 관계가 달라지므로 별도의 장면 설계 규칙과 평가 기준이 필요하다.

### 8.7.2 브랜드 무드 관리

사용자가 매번 프리셋을 직접 선택하지 않아도 자신의 매장 분위기에 맞는 결과를 받을 수 있도록 확장할 수 있다.

- 선호 색상 저장
- 자주 사용하는 구도 저장
- 매장 인테리어 사진 등록
- 선호 프리셋과 생성 이력 관리

- 계절·신메뉴·행사별 장면 제안
- 브랜드에 맞는 캡션 톤 관리

### 8.7.3 프리셋 자동 제작 서비스

장기적으로는 사용자에게 이미 만들어진 프리셋을 제공하는 것에 그치지 않고, 사용자가 좋아하는 사진을 바탕으로 새로운 프리셋을 제안하는 서비스로 발전시킬 수 있다.

사용자 레퍼런스 업로드→ 장면 분석→ 카메라·빛·구도 설계 규칙 생성→ 비식별 힌트 제작→ 프리셋 초안 생성→ 시험 이미지 생성→ 사람 검수→ 개인 브랜드 프리셋 등록

## 9. 종합 결론

Pickmood 는 카페 운영자가 직접 촬영한 제품 사진을 인스타그램용 광고 이미지와 게시 문구로 전환하는 서비스다. 프로젝트의 핵심 문제는 이미지를 생성할 수 있는가가 아니라, 제품의 정체성을 고려하면서 카메라·구도·빛·재질을 반복 가능한 방식으로 적용할 수 있는가에 있었다.

초기 오픈소스 모델링에서는 SD1.5, SDXL, FLUX.1 Kontext 와 다중 모델 조합을 거치며 전체 재생성의 제품 변형과 결정론적 합성의 이질감을 확인했다. 이 시행착오는 최종 구조의 실패가 아니라, 제품 정보와 장면 정보를 분리하고 각 요소의 적용 범위를 명확히 해야 한다는 설계 근거가 됐다.

약 700 장의 카페 SNS 레퍼런스를 색감·공간 무드와 카메라 시점·피사체 점유율 기준으로 분류해 3 개 무드와 4 개 촬영 구도의 12 개 프리셋을 구성했다. 프리셋은 하나의 긴 프롬프트가 아니라, 제품 정체성·장면·빛·출력 비율·금지 조건을 역할별 파일과 힌트 이미지로 나눈 패키지이다.

ComfyUI 에서는 실행 그래프를 프리셋마다 복제하지 않고 하나의 공용 런타임이 preset\_id 를 해석하도록 통합했다. 프론트엔드와 백엔드는 이 실행 구조를 실제 서비스에 연결하고 GCP 배포와 Langfuse 관측까지 구현했다. 그 결과 프리셋을 추가할 때 실행 그래프 전체를 다시 만들지 않고, 설계 파일 등록과 검증으로 확장할 수 있는 기반을 확보했다.

현재 결과는 모든 제품과 모든 촬영 조건에서 통계적으로 품질을 보장하는 단계는 아니다. 프리셋 제작과 시각 검수는 수작업 비중이 높고, 자동 품질 검사·로그 합성·선택 영역 재생성·부분 편집과 안정적인 다중 작업 처리는 후속 과제로 남아 있다. 향후에는 검증된 구조를 자동화하고 매장별 선호 색감·구도·문구와 브랜드 요소를 저장해 매장 전용 콘텐츠로 확장할 수 있다.

이번 프로젝트는 더 많은 기능을 제공하는 범용 디자인 도구를 만드는 대신, 사용자 촬영본을 카페 SNS 에 활용할 제품 사진으로 바꾸는 좁고 반복적인 문제에 집중했다. 모델 선택, 프리셋 설계, 시각 힌트, 공용 런타임과 운영 관측을 하나의 흐름으로 연결함으로써, 생성 품질을 우연한 한 장이 아니라 관리 가능한 서비스 과정으로 만들 수 있다는 가능성을 확인했다.